

Algèbre générale

Module ECUE111 — S1, 2025–2026

Filière Mathématiques — Option Sciences des données

1^{ère} année

Aymen Ben Brik

6 mai 2026

Code UE	UEF110
Code ECUE	ECUE111
Volume horaire	84 h présentiel (42 h cours + 42 h TD)
Coefficient	3.5
Crédits ECTS	7
Régime	CC mixte
Responsable du module	Moufid Chaima
Rédaction du support	Aymen Ben Brik

Avant-propos

Ce document constitue le support complet du module *Algèbre générale* (ECUE111), en première année de la filière Mathématiques (option Sciences des données). Il couvre les six chapitres du programme officiel :

1. Calculs algébriques (sommes, produits, formule du binôme)
2. Vocabulaire ensembliste (logique, ensembles, dénombrement, relations)
3. Arithmétique dans $\mathbb{Z}$
4. Structures algébriques (groupes, anneaux, corps)
5. Polynômes
6. Fractions rationnelles

Pédagogie. Chaque chapitre suit une structure unifiée : une *motivation* concrète (en informatique, science des données, cryptographie ou ingénierie), une *approche par problème*, des *définitions et théorèmes* avec démonstrations, des *exemples résolus* à difficulté ascendante, du *code Python* (`sympy`, `numpy`, `itertools`) et des *exercices à faire*. Chaque chapitre se conclut par un *TP Python structuré* (annotations Bloom + difficulté), construisant progressivement des bibliothèques réutilisables.

Acquis d'apprentissage.

- **AA1** : Maîtriser les opérations algébriques fondamentales et leurs propriétés.
- **AA2** : Utiliser logique et théorie des ensembles pour structurer un raisonnement.
- **AA3** : Appliquer les principes d'arithmétique (division euclidienne, premiers, congruences).
- **AA4** : Manipuler les polynômes (opérations, factorisation, racines).
- **AA5** : Manipuler les fractions rationnelles et simplifications algébriques.
- **AA6** : Rédiger et justifier des arguments mathématiques de manière rigoureuse et claire.

Code source. Le repo GitHub [Aymenbenbrik/Algebre1-2026](https://github.com/Aymenbenbrik/Algebre1-2026) contient les fichiers $\text{\LaTeX}$ sources de chaque chapitre, ainsi que les notes SmartBoard des séances.

Table des matières

Avant-propos	1
1 Calculs algébriques	7
1.1 Sommes et produits finis	7
1.1.1 Motivation	7
1.1.2 Approche par problème : L’astuce du jeune Gauss	7
1.1.3 Définitions	7
1.1.4 Exemples résolus (Difficulté ascendante)	8
1.1.5 Exercices pratiques avec Python	8
1.1.6 Exercices à faire	9
1.2 Sommes doubles	9
1.2.1 Motivation	9
1.2.2 Approche par problème : Lignes ou colonnes en premier ?	9
1.2.3 Définitions	9
1.2.4 Exemples résolus (Difficulté ascendante)	10
1.2.5 Exercices pratiques avec Python	10
1.2.6 Exercices à faire	11
1.3 Formule du Binôme	11
1.3.1 Motivation : Analyse de séquences binaires	11
1.3.2 Approche par problème : L’arbre des passes	11
1.3.3 Définitions	12
1.3.4 Exemples résolus (Difficulté ascendante)	13
1.3.5 Exercices pratiques avec Python	13
1.3.6 Exercices à faire	14
Pour aller plus loin : L’Algèbre au cœur de la Data Science	14
Travaux Dirigés : Calculs Algébriques	16
Travaux Pratiques (TP) Python	17
2 Vocabulaire ensembliste	20
2.1 Éléments de logique	20
2.1.1 Motivation : L’ordinateur face à l’ambiguïté	20
2.1.2 Approche par problème : Le contrat trompeur	20
2.1.3 Définitions des connecteurs logiques	20
2.1.4 Les Quantificateurs	21
2.1.5 Exemples résolus (Difficulté ascendante)	21
2.1.6 Exercices pratiques avec Python	21
2.1.7 Exercices à faire	22
2.2 Éléments de la théorie des ensembles	22

2.2.1	Motivation : L'agrégation de données extraites	22
2.2.2	Approche par problème : Le paradoxe de l'appartenance	23
2.2.3	Définitions fondamentales	23
2.2.4	Opérations sur les ensembles	23
2.2.5	Exemples résolus (Difficulté ascendante)	23
2.2.6	Exercices pratiques avec Python	24
2.2.7	Exercices à faire	24
2.3	Ensembles finis et dénombrement	25
2.3.1	Motivation : L'explosion combinatoire en informatique	25
2.3.2	Approche par problème : L'ordre a-t-il de l'importance ?	25
2.3.3	Définitions fondamentales	25
2.3.4	Les outils de dénombrement	25
2.3.5	Exemples résolus (Difficulté ascendante)	26
2.3.6	Exercices pratiques avec Python	26
2.3.7	Exercices à faire	27
2.4	Applications et relations : Ordre, équivalence et quotient	27
2.4.1	Motivation : Regroupement et classification de données	27
2.4.2	Approche par problème : Le tri des doublons	27
2.4.3	Définitions des Relations	28
2.4.4	Définitions des Applications (Fonctions)	28
2.4.5	Exemples résolus (Difficulté ascendante)	29
2.4.6	Exercices pratiques avec Python	29
2.4.7	Exercices à faire	30
	Pour aller plus loin : Ensembles, Relations et Intelligence Artificielle	30
	Travaux Pratiques (TP) Python	33
3	Rappels d'arithmétique dans l'ensemble des entiers relatifs	36
3.1	Division euclidienne, congruence	36
3.1.1	Motivation	36
3.1.2	Approche par problème	36
3.1.3	Définitions et théorèmes	36
3.1.4	Exemples de difficulté croissante	37
3.1.5	Exercice pratique avec Python	38
3.2	PGCD et PPCM	38
3.2.1	Motivation	38
3.2.2	Approche par problème	39
3.2.3	Définitions et théorèmes	39
3.2.4	Exemples de difficulté croissante	39
3.2.5	Exercice pratique avec Python	40
3.3	Théorème de Gauss, Identité de Bézout, Algorithme d'Euclide	41
3.3.1	Motivation	41
3.3.2	Approche par problème	41
3.3.3	Définitions et théorèmes	41
3.3.4	Exemples de difficulté croissante	42
3.3.5	Exercice pratique avec Python	42
	Travaux Dirigés : Arithmétique dans $\mathbb{Z}$	43
	Travaux Pratiques (TP) : Implémentations Algorithmiques	45
	Aller plus loin : L'arithmétique au cœur de la Data Science	47

Projet d'Application : Le « Hashing Trick » en Analyse de Données	48
4 Structures algébriques	51
4.1 Structure de groupe	51
4.1.1 Motivation	51
4.1.2 Approche par problème : l'horloge et le chiffrement de César	51
4.1.3 Loi de composition interne et définition du groupe	52
4.1.4 Sous-groupes	52
4.1.5 Les sous-groupes de $(\mathbb{Z}, +)$	53
4.1.6 Sous-groupe engendré, groupes monogènes et cycliques	53
4.1.7 Ordre d'un élément	53
4.1.8 Exemples résolus (Difficulté ascendante)	54
4.1.9 Exercices pratiques avec Python	54
4.1.10 Exercices à faire	55
4.1.11 Théorème de Lagrange	55
4.1.12 Morphismes de groupes	56
4.1.13 Le groupe symétrique S_n	57
4.1.14 Le groupe $\mathbb{Z}/n\mathbb{Z}$	58
4.1.15 Exemples résolus pour le §1.b (Difficulté ascendante)	59
4.1.16 Exercices Python pour le §1.b	59
4.1.17 Exercices à faire (§1.b)	60
4.2 Structures d'anneau et de corps	60
4.2.1 Motivation	60
4.2.2 Approche par problème : une anomalie dans $\mathbb{Z}/6\mathbb{Z}$	60
4.2.3 Définition d'un anneau	61
4.2.4 Sous-anneau	61
4.2.5 Anneau intègre, diviseurs de zéro	62
4.2.6 Idéaux (notion brève)	62
4.2.7 Corps	62
4.2.8 Caractéristique d'un anneau	63
4.2.9 Morphismes d'anneaux	63
4.2.10 Exemples résolus (Difficulté ascendante)	63
4.2.11 Exercices pratiques avec Python	64
4.2.12 Exercices à faire	65
Travaux Pratiques (TP) Python	65
5 Polynômes	68
5.1 Anneaux $\mathbb{R}[X]$ et $\mathbb{C}[X]$	68
5.1.1 Motivation	68
5.1.2 Approche par problème : polynôme nul vs fonction nulle	68
5.1.3 Construction formelle de $K[X]$	68
5.1.4 Degré et valuation	69
5.1.5 Inversibles de $K[X]$	70
5.1.6 Cas particuliers : $\mathbb{R}[X]$ et $\mathbb{C}[X]$	70
5.1.7 Composition de polynômes	70
5.1.8 Exemples résolus (Difficulté ascendante)	70
5.1.9 Exercices pratiques avec Python	71
5.1.10 Exercices à faire	71
5.2 Fonctions polynomiales et racines	72

5.2.1	Motivation	72
5.2.2	Approche par problème : reconstruire un polynôme à partir de ses racines	72
5.2.3	Évaluation et fonction polynomiale	72
5.2.4	Racines et factorisation par $(X - a)$	73
5.2.5	Multiplicité d'une racine	73
5.2.6	Polynôme dérivé	73
5.2.7	Borne sur le nombre de racines	74
5.2.8	Identification polynôme-fonction	74
5.2.9	Polynômes à coefficients réels	74
5.2.10	Exemples résolus (Difficulté ascendante)	75
5.2.11	Exercices pratiques avec Python	75
5.2.12	Exercices à faire	76
5.3	Arithmétique dans $K[X]$	76
5.3.1	Motivation	76
5.3.2	Approche par problème : PGCD de deux polynômes	76
5.3.3	Divisibilité dans $K[X]$	77
5.3.4	Division euclidienne	77
5.3.5	Procédé de Hörner et division par $(X - a)$	78
5.3.6	PGCD dans $K[X]$	78
5.3.7	Identité de Bézout	78
5.3.8	Théorème de Gauss	79
5.3.9	PPCM	79
5.3.10	Exemples résolus (Difficulté ascendante)	79
5.3.11	Exercices pratiques avec Python	80
5.3.12	Exercices à faire	81
5.4	Polynômes irréductibles, décomposition, relations racines-coefficients	81
5.4.1	Motivation	81
5.4.2	Approche par problème : où $X^2 + 1$ est-il irréductible ?	81
5.4.3	Polynômes irréductibles	82
5.4.4	Décomposition unique en facteurs irréductibles	82
5.4.5	Irréductibles de $\mathbb{C}[X]$	82
5.4.6	Irréductibles de $\mathbb{R}[X]$	83
5.4.7	Relations entre coefficients et racines (formules de Viète)	83
5.4.8	Exemples résolus (Difficulté ascendante)	84
5.4.9	Exercices pratiques avec Python	85
5.4.10	Exercices à faire	85
	Travaux Pratiques (TP) Python	86
6	Fractions rationnelles	89
6.1	Corps des fractions $K(X)$	89
6.1.1	Motivation	89
6.1.2	Approche par problème : à quelles conditions $A/B = C/D$?	89
6.1.3	Construction de $K(X)$	90
6.1.4	Opérations sur $K(X)$	90
6.1.5	Structure de corps	90
6.1.6	Inclusion $K[X] \hookrightarrow K(X)$	91
6.1.7	Cas particuliers : $\mathbb{R}(X)$ et $\mathbb{C}(X)$	91

6.1.8	Fonction rationnelle associée	91
6.1.9	Exemples résolus (Difficulté ascendante)	92
6.1.10	Exercices pratiques avec Python	92
6.1.11	Exercices à faire	93
6.2	Forme irréductible et fonctions rationnelles	93
6.2.1	Motivation	93
6.2.2	Approche par problème : trouver la « plus simple » écriture	93
6.2.3	Forme irréductible	94
6.2.4	Algorithme de réduction	94
6.2.5	Domaine de la fonction rationnelle	95
6.2.6	Opérations sur les formes irréductibles	95
6.2.7	Exemples résolus (Difficulté ascendante)	95
6.2.8	Exercices pratiques avec Python	96
6.2.9	Exercices à faire	97
6.3	Degré, pôles, zéros, multiplicités	97
6.3.1	Motivation	97
6.3.2	Approche par problème	98
6.3.3	Degré d'une fraction rationnelle	98
6.3.4	Fractions propres et partie entière	98
6.3.5	Multiplicité d'un zéro et d'un pôle	99
6.3.6	Comportement local près d'un pôle	99
6.3.7	Cas $\mathbb{C}(X)$: relation entre degré, zéros et pôles	99
6.3.8	Exemples résolus (Difficulté ascendante)	100
6.3.9	Exercices pratiques avec Python	100
6.3.10	Exercices à faire	101
6.4	Décomposition en éléments simples	102
6.4.1	Motivation	102
6.4.2	Approche par problème : intégrale d'une fraction simple	102
6.4.3	Théorème de décomposition sur $\mathbb{C}$	103
6.4.4	Théorème de décomposition sur $\mathbb{R}$	103
6.4.5	Méthodes pratiques de calcul des coefficients	104
6.4.6	Exemples résolus (Difficulté ascendante)	104
6.4.7	Application : calcul de primitives	106
6.4.8	Exercices pratiques avec Python	106
6.4.9	Exercices à faire	107
	Travaux Pratiques (TP) Python	107

Chapitre 1

Calculs algébriques

1.1 Sommes et produits finis

1.1.1 Motivation

Dans de nombreux domaines tels que l'analyse de données, l'algorithmique ou les probabilités, nous sommes confrontés à la nécessité d'agréger ou de multiplier de grandes quantités de valeurs. L'écriture explicite de ces opérations (ex : $x_1 + x_2 + \dots + x_{1000}$) devient vite illisible et impraticable. L'introduction des symboles de sommation ($\sum$) et de produit ($\prod$) permet non seulement de compacter l'écriture, mais surtout de manipuler ces blocs de données de manière formelle et rigoureuse pour en extraire de nouvelles propriétés.

1.1.2 Approche par problème : L'astuce du jeune Gauss

Problème : Comment calculer rapidement la somme des 100 premiers nombres entiers : $S = 1 + 2 + 3 + \dots + 99 + 100$?

Résolution : Une approche naïve consisterait à additionner terme à terme. Cependant, si l'on écrit cette somme à l'endroit, puis à l'envers, et qu'on les additionne verticalement :

$$\begin{array}{r} S = 1 + 2 + \dots + 99 + 100 \\ S = 100 + 99 + \dots + 2 + 1 \\ \hline 2S = 101 + 101 + \dots + 101 + 101 \end{array}$$

Puisqu'il y a 100 termes, on obtient $2S = 100 \times 101$, d'où $S = 5050$. Ce problème met en évidence le besoin d'un formalisme (les changements d'indices) pour généraliser ce type de raisonnement à tout entier n .

1.1.3 Définitions

Soient $n, p \in \mathbb{N}$ tels que $p \leq n$, et $(a_k)_{p \leq k \leq n}$ une suite de nombres réels ou complexes.

Définition 1.1 (Somme) : On note par la lettre grecque majuscule Sigma ($\sum$) la somme des éléments a_k pour l'indice k allant de p à n :

$$\sum_{k=p}^n a_k = a_p + a_{p+1} + \dots + a_n$$

Remarque : La variable k est une variable "muette". Elle n'existe qu'à l'intérieur de la somme. $\sum_{k=1}^n a_k = \sum_{i=1}^n a_i$.

Définition 1.2 (Produit) : On note par la lettre grecque majuscule Pi ($\prod$) le produit de ces mêmes éléments :

$$\prod_{k=p}^n a_k = a_p \times a_{p+1} \times \cdots \times a_n$$

Convention : Une somme vide (si $p > n$) vaut 0. Un produit vide vaut 1.

1.1.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Développement simple - Facile) : Développer $\sum_{k=2}^5 k^2$. *Correction :* On remplace successivement k par 2, 3, 4 et 5.

$$\sum_{k=2}^5 k^2 = 2^2 + 3^2 + 4^2 + 5^2 = 4 + 9 + 16 + 25 = 54$$

Exemple 2 (Changement d'indice - Intermédiaire) : Exprimer $S = \sum_{k=1}^n a_{k+2}$ sous la forme d'une somme dont le terme général est a_j . *Correction :* Posons $j = k + 2$. Quand $k = 1$, $j = 3$. Quand $k = n$, $j = n + 2$. Donc, $S = \sum_{j=3}^{n+2} a_j$.

Exemple 3 (Somme télescopique - Difficile) : Calculer $S_n = \sum_{k=1}^n \left(\frac{1}{k} - \frac{1}{k+1}\right)$. *Correction :* En développant les premiers et derniers termes, on observe des annulations en cascade :

$$S_n = \left(1 - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \cdots + \left(\frac{1}{n} - \frac{1}{n+1}\right)$$

Tous les termes intermédiaires s'annulent. Il ne reste que le premier et le dernier :

$$S_n = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

1.1.5 Exercices pratiques avec Python

Les sommes mathématiques se traduisent naturellement par des boucles `for` en développement logiciel. Python propose également des fonctions intégrées très performantes pour l'analyse de tableaux de données.

```

1  # Exercice : Calculer la somme des carrés des 10 premiers entiers
2
3  # Methode 1 : Approche algorithmique classique (boucle for)
4  somme_boucle = 0
5  for k in range(1, 11): # k va de 1 a 10
6      somme_boucle += k**2
7  print(f"Resultat avec boucle : {somme_boucle}")
8
9  # Methode 2 : Approche "Pythonique" (comprehension de liste)
10 # Tres utile pour manipuler rapidement des donnees
11 somme_rapide = sum([k**2 for k in range(1, 11)])
12 print(f"Resultat avec sum() : {somme_rapide}")

```

1.1.6 Exercices à faire

1. **Exercice 1** : Calculer la valeur exacte de $\prod_{k=1}^4 (2k)$.
2. **Exercice 2** : En utilisant la relation de Chasles, simplifier l'expression : $\sum_{k=1}^{100} k^3 - \sum_{k=1}^{98} k^3$.
3. **Exercice 3** : Montrer par le calcul que $\sum_{k=0}^n (2k+1) = (n+1)^2$.
4. **Exercice 4 (Code)** : Écrire une fonction Python `factorielle(n)` qui calcule $n! = \prod_{k=1}^n k$ à l'aide d'une boucle.

1.2 Sommes doubles

1.2.1 Motivation

Jusqu'à présent, nous avons sommé les éléments d'une liste à une dimension. Cependant, en algèbre linéaire (avec les matrices) ou en analyse de données (avec les tableaux et les images numériques), les données sont souvent organisées en deux dimensions : par lignes et par colonnes. Pour agréger ou manipuler ces données, nous devons itérer sur deux indices simultanément. La maîtrise des sommes doubles permet de naviguer mathématiquement dans ces grilles avec une précision absolue.

1.2.2 Approche par problème : Lignes ou colonnes en premier ?

Problème : Imaginons un tableau de nombres rectangulaire comportant n lignes et p colonnes. On note $a_{i,j}$ le nombre situé à la ligne i et à la colonne j . Comment calculer la somme totale de tous les nombres de ce tableau ?

Résolution : Deux stratégies intuitives s'offrent à nous :

1. **Par lignes** : On calcule d'abord la somme de chaque ligne i (soit $L_i = \sum_{j=1}^p a_{i,j}$), puis on additionne le total de toutes les lignes : $\sum_{i=1}^n L_i$.
2. **Par colonnes** : On calcule d'abord la somme de chaque colonne j (soit $C_j = \sum_{i=1}^n a_{i,j}$), puis on additionne ces totaux : $\sum_{j=1}^p C_j$.

Ces deux méthodes comptent exactement les mêmes éléments. On en déduit une propriété fondamentale d'interversion :

$$\sum_{i=1}^n \left(\sum_{j=1}^p a_{i,j} \right) = \sum_{j=1}^p \left(\sum_{i=1}^n a_{i,j} \right)$$

1.2.3 Définitions

Définition 2.1 (Somme sur un rectangle) : Si les indices i et j varient indépendamment l'un de l'autre ($1 \leq i \leq n$ et $1 \leq j \leq p$), on note la somme double :

$$\sum_{i=1}^n \sum_{j=1}^p a_{i,j} \quad \text{ou plus simplement} \quad \sum_{\substack{1 \leq i \leq n \\ 1 \leq j \leq p}} a_{i,j}$$

Définition 2.2 (Somme sur un triangle / Indices dépendants) : Si la borne d'un indice dépend de l'autre (par exemple $1 \leq i \leq j \leq n$), on somme uniquement sur

une "moitié" du tableau (au-dessus ou en dessous de la diagonale). L'ordre de sommation devient crucial :

$$\sum_{1 \leq i \leq j \leq n} a_{i,j} = \sum_{j=1}^n \left(\sum_{i=1}^j a_{i,j} \right) = \sum_{i=1}^n \left(\sum_{j=i}^n a_{i,j} \right)$$

Règle d'or : La somme extérieure ne doit jamais avoir d'indice variable dans ses bornes.

1.2.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Variables séparables - Facile) : Calculer $S = \sum_{i=1}^n \sum_{j=1}^p (i \times j)$.
Correction : Puisque les termes dépendants de i sont des constantes par rapport à j , on peut factoriser :

$$S = \sum_{i=1}^n \left(i \sum_{j=1}^p j \right) = \left(\sum_{i=1}^n i \right) \times \left(\sum_{j=1}^p j \right) = \frac{n(n+1)}{2} \times \frac{p(p+1)}{2}$$

Exemple 2 (Somme triangulaire simple - Intermédiaire) : Calculer $S = \sum_{i=1}^n \sum_{j=i}^n 1$. (Cela revient à compter le nombre d'éléments dans un triangle). *Correction* : On commence par la somme intérieure (sur j) :

$$\sum_{j=i}^n 1 = (n - i + 1) \quad (\text{nombre de termes})$$

On remplace dans la somme extérieure :

$$S = \sum_{i=1}^n (n - i + 1) = \sum_{i=1}^n n - \sum_{i=1}^n i + \sum_{i=1}^n 1 = n^2 - \frac{n(n+1)}{2} + n = \frac{n(n+1)}{2}$$

Exemple 3 (Inversion de l'ordre de sommation - Difficile) : Soit $S = \sum_{i=1}^n \sum_{j=i}^n \frac{i}{j}$. Il est difficile de sommer d'abord sur j . Invertissons les sommes. *Correction* : Le domaine est $1 \leq i \leq j \leq n$. Si on somme d'abord sur i , pour un j fixé, i varie de 1 à j . L'indice j varie de 1 à n .

$$S = \sum_{j=1}^n \sum_{i=1}^j \frac{i}{j} = \sum_{j=1}^n \frac{1}{j} \left(\sum_{i=1}^j i \right) = \sum_{j=1}^n \frac{1}{j} \frac{j(j+1)}{2} = \frac{1}{2} \sum_{j=1}^n (j+1)$$

$$S = \frac{1}{2} \left(\frac{n(n+1)}{2} + n \right) - \frac{1}{2} \quad (\text{en ajustant l'indice pour } j=1)$$

1.2.5 Exercices pratiques avec Python

En programmation, une somme double se traduit par des boucles imbriquées. En analyse de données, la bibliothèque **numpy** permet d'optimiser ces calculs matriciels.

```
1 import numpy as np
2
3 # Creation d'un tableau (matrice) 3x3
4 matrice = np.array([
5     [1, 2, 3],
6     [4, 5, 6],
```

```

7         [7, 8, 9]
8     ])
9
10    # Methode 1 : Somme double avec boucles imbriquees (Algorithmique
      classique)
11    somme_totale = 0
12    for i in range(3):          # Lignes
13        for j in range(3):      # Colonnes
14            somme_totale += matrice[i][j]
15    print(f"Somme_via_boucles : {somme_totale}")
16
17    # Methode 2 : Somme triangulaire (i <= j, au-dessus de la diagonale
      )
18    somme_triangle = 0
19    for i in range(3):
20        for j in range(i, 3): # j démarre a i
21            somme_triangle += matrice[i][j]
22    print(f"Somme_triangulaire : {somme_triangle}")
23
24    # Methode 3 : L'approche matricielle (Python pour la data)
25    print(f"Somme_rapide_avec_numpy : {np.sum(matrice)}")

```

1.2.6 Exercices à faire

1. **Exercice 1** : Calculer la valeur de $\sum_{i=1}^n \sum_{j=1}^n (i + j)$.
2. **Exercice 2** : Inverser l'ordre des sommes dans l'expression suivante et la calculer : $\sum_{i=1}^n \sum_{j=i}^n 2^j$.
3. **Exercice 3** : Que vaut la somme double $\sum_{1 \leq i, j \leq n} \min(i, j)$? (Indice : séparez les cas $i \leq j$ et $j < i$).

1.3 Formule du Binôme

1.3.1 Motivation : Analyse de séquences binaires

En probabilités et en analyse de données, il est fréquent d'étudier des séquences d'événements binaires. Prenons l'exemple de l'analyse des performances dans un match de football : une passe tentée par un joueur est soit réussie (notons-la R), soit manquée (notons-la M). Si l'on souhaite analyser les statistiques d'un joueur qui tente n passes durant un match, il existe une multitude de scénarios possibles menant à exactement k passes réussies. La formule du binôme est l'outil algébrique qui permet de dénombrer instantanément ces combinaisons et de modéliser cette distribution sans avoir à lister manuellement chaque trajectoire possible.

1.3.2 Approche par problème : L'arbre des passes

Problème : Un milieu de terrain tente 3 passes consécutives. En représentant algébriquement chaque tentative par la somme $(R + M)$, comment déduire tous les scénarios possibles ?

- 1 passe : $(R + M)^1 = R + M$ (1 scénario 100% réussi, 1 scénario 100% raté)
- 2 passes : $(R + M)^2 = R^2 + 2RM + M^2$ (1 avec 2 réussites, 2 avec 1 réussite/1 échec, 1 avec 2 échecs)
- 3 passes : $(R + M)^3 = R^3 + 3R^2M + 3RM^2 + M^3$

$n = 0$

1

$n = 1$

1 1

$n = 2$

1 2 1

$n = 3$

1 3 3 1

$\swarrow + \searrow$

$n = 4$

1 4 6 4 1

Chaque nombre y est la somme des deux nombres situés juste au-dessus de lui. Pour généraliser cela à n passes, il nous faut définir rigoureusement ces coefficients.

Définition 3.1 (Factorielle) : Soit $n \in \mathbb{N}$. On appelle factorielle n , notée $n!$, le produit des n premiers entiers naturels non nuls :

$$n! = 1 \times 2 \times \cdots \times n = \prod_{k=1}^n k$$

Définition 3.2 (Coefficient binomial) : Pour $n \in \mathbb{N}$ et $k \in \mathbb{N}$ tels que $0 \leq k \leq n$, on définit le coefficient binomial "k parmi n", noté $\binom{n}{k}$, par :

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Théorème (Formule du Binôme de Newton) : Pour tous nombres complexes (ou réels) a et b , et pour tout entier naturel n :

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k}$$

1.3.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Développement direct - Facile) : Développer $(x + 1)^4$. *Correction :* En appliquant la formule avec $a = x$, $b = 1$ et $n = 4$:

$$(x + 1)^4 = \sum_{k=0}^4 \binom{4}{k} x^k 1^{4-k} = \binom{4}{0} x^0 + \binom{4}{1} x^1 + \binom{4}{2} x^2 + \binom{4}{3} x^3 + \binom{4}{4} x^4$$

En calculant les coefficients (ligne $n = 4$ du triangle : 1, 4, 6, 4, 1), on obtient :

$$(x + 1)^4 = 1 + 4x + 6x^2 + 4x^3 + x^4$$

Exemple 2 (Recherche de coefficient - Intermédiaire) : Quel est le coefficient de x^3 dans le développement de $(2x - 3)^5$? *Correction :* Le terme général du développement est $\binom{5}{k} (2x)^k (-3)^{5-k}$. Pour isoler x^3 , on choisit $k = 3$. Le terme correspondant est $\binom{5}{3} (2x)^3 (-3)^2 = 10 \times 8x^3 \times 9 = 720x^3$. Le coefficient recherché est 720.

Exemple 3 (Calcul de somme via le binôme - Difficile) : Calculer la somme $S = \sum_{k=0}^n \binom{n}{k}$. *Correction :* On pose astucieusement $a = 1$ et $b = 1$ dans la formule du binôme :

$$(1 + 1)^n = \sum_{k=0}^n \binom{n}{k} 1^k 1^{n-k} = \sum_{k=0}^n \binom{n}{k}$$

On en déduit immédiatement que $S = 2^n$.

1.3.5 Exercices pratiques avec Python

Les factorielles grandissent de manière exponentielle. En Python, plutôt que de coder la division des factorielles à la main, il est recommandé d'utiliser les fonctions optimisées de la bibliothèque standard pour éviter les lenteurs et les dépassements de capacité sur de grandes données.

```
1 import math
2
3 # Calcul du nombre de combinaisons (ex: 3 passes reussies parmi 10)
4 n_tentatives = 10
5 k_succes = 3
6 combinaisons = math.comb(n_tentatives, k_succes)
7 print(f"Scenarios possibles ({k_succes} succes sur {n_tentatives}) : {combinaisons}")
8
9 # Fonction pour generer et afficher le Triangle de Pascal
10 def afficher_pascal(lignes):
11     for n in range(lignes):
12         ligne_actuelle = [math.comb(n, k) for k in range(n + 1)]
13         # Affichage centre pour visualiser la symetrie
14         print(" ".join(map(str, ligne_actuelle)).center(40))
15
16 print("\nTriangle de Pascal (n=0 a n=5) :")
17 afficher_pascal(6)
```

1.3.6 Exercices à faire

1. **Exercice 1** : Calculer la somme alternée : $\sum_{k=0}^n (-1)^k \binom{n}{k}$.
2. **Exercice 2** : Développer l'expression $(x - \frac{1}{x})^6$ et simplifier les termes.
3. **Exercice 3** : Démontrer la formule de Pascal : $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (pour $1 \leq k \leq n-1$) en utilisant la définition algébrique avec les factorielles.

Pour aller plus loin : L'Algèbre au cœur de la Data Science

Bien que les concepts de sommes doubles et de coefficients binomiaux puissent sembler purement théoriques de prime abord, ils constituent en réalité le moteur algorithmique des technologies modernes d'analyse de données. Pour clôturer ce premier chapitre, je vous invite à explorer les deux pistes de recherche suivantes.

Piste 1 : Sommes doubles et Vision par ordinateur (Computer Vision)

Lorsqu'un algorithme de vision par ordinateur analyse un flux vidéo (par exemple, pour détecter automatiquement la balle ou traquer les déplacements des joueurs sur un terrain de football), il traite chaque image comme une immense matrice de pixels.

Pour détecter des contours, des lignes ou des mouvements, les algorithmes utilisent une opération mathématique appelée **convolution**. Appliquer un filtre de convolution sur un pixel de coordonnées (x, y) revient exactement à calculer une double somme sur un voisinage de ce pixel, pondérée par une petite matrice appelée "noyau" (F) :

$$Nouvelle_Valeur_{x,y} = \sum_{i=-1}^1 \sum_{j=-1}^1 F_{i,j} \times Pixel_{x+i,y+j}$$

Sujet de recherche : Renseignez-vous sur les "Noyaux de convolution" (*Convolution Kernels*) tels que les filtres de Sobel. Vous découvrirez comment de simples sommes doubles en algèbre permettent aux ordinateurs de "voir" les formes.

Piste 2 : Combinaisons, Scraping et "Fléau de la dimension"

Imaginez que vous ayez constitué une vaste base de données (par exemple, en automatisant l'extraction web de milliers d'offres d'emploi). Chaque offre est décrite par n caractéristiques ou variables (salaire, localisation, mots-clés des compétences, etc.).

Si vous souhaitez entraîner un modèle d'Intelligence Artificielle pour prédire si une offre trouvera rapidement preneur, vous devrez choisir le meilleur sous-ensemble de variables. Or, si vous avez $n = 50$ variables et que vous souhaitez tester toutes les combinaisons possibles de $k = 5$ variables, le nombre de modèles à entraîner est exactement le coefficient binomial :

$$\binom{50}{5} = 2\,118\,760 \text{ combinaisons}$$

C'est ce que l'on appelle en Data Science le **fléau de la dimension** (*Curse of dimensionality*) ou l'explosion combinatoire.

Sujet de recherche : Explorez les algorithmes de **sélection de caractéristiques** (*Feature Selection*) en Machine Learning. Comment les data scientists font-ils pour trouver le meilleur modèle sans avoir à calculer ces millions de combinaisons dictées par la formule du binôme ?

Travaux Dirigés : Calculs Algébriques

Partie A : Entraînement technique

Exercice 1 : Manipulations de sommes simples

1. Calculer la somme télescopique suivante en fonction de n :

$$S_n = \sum_{k=1}^n \ln \left(\frac{k+1}{k} \right)$$

2. En utilisant un changement d'indice, démontrer que :

$$\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0$$

Exercice 2 : Sommes doubles et inversions

1. Calculer la somme double sur un rectangle :

$$A_n = \sum_{i=1}^n \sum_{j=1}^n (2i + j)$$

2. Calculer la somme sur un domaine triangulaire en inversant l'ordre de sommation :

$$B_n = \sum_{i=1}^n \sum_{j=i}^n j$$

Exercice 3 : Autour du Binôme et de Pascal

1. En utilisant la formule du binôme de Newton, simplifier la somme : $\sum_{k=0}^n \binom{n}{k} 3^k$.
2. En dérivant la fonction $f(x) = (1+x)^n$, démontrer la formule suivante (très utile en probabilités) :

$$\sum_{k=0}^n k \binom{n}{k} = n 2^{n-1}$$

Partie B : Modélisation et Algorithmique

Exercice 4 : Extraction de données (Web Scraping)

Un script Python automatisé parcourt n pages d'un portail web pour extraire des offres d'emploi. L'algorithme est conçu tel que sur la page numéro k , il parvient à extraire exactement $3k - 1$ offres valides.

1. Écrire la somme E_n représentant le nombre total d'offres d'emploi extraites après le parcours de n pages.
2. Calculer cette somme en fonction de n .
3. Combien d'offres seront stockées dans la base de données si le script parcourt 50 pages ?

Exercice 5 : Vision par ordinateur et analyse sportive

Lors de l'analyse vidéo d'un match de football par vision par ordinateur, on isole une zone spécifique du terrain représentée par une matrice de pixels M de taille $n \times n$. Pour détecter le mouvement des joueurs, l'algorithme calcule un "poids analytique" pour chaque coordonnée (i, j) défini par $P_{i,j} = i \times j$. Afin d'optimiser le temps de calcul, le script ne balaie que le triangle supérieur de l'image, c'est-à-dire les pixels où $i \leq j$.

1. Écrire la somme double W_n représentant le poids analytique total de la zone balayée.
2. Calculer cette somme en commençant par sommer sur l'indice i . *Indice : Rappelez-vous que $\sum_{i=1}^j i = \frac{j(j+1)}{2}$ et $\sum_{j=1}^n j^3 = \left(\frac{n(n+1)}{2}\right)^2$.*

Exercice 6 : Combinaisons de variables

Dans la phase de préparation de nos données d'analyse, nous disposons d'un ensemble de n variables distinctes (vitesse, accélération, position, etc.). Nous voulons tester des modèles prédictifs en essayant toutes les combinaisons possibles de ces variables (des modèles incluant 0 variable jusqu'aux modèles incluant les n variables).

1. Exprimer le nombre de modèles contenant exactement k variables.
2. Montrer, à l'aide de la formule du binôme, que le nombre total de modèles à entraîner sera exactement de 2^n . (Ce résultat illustre la notion d'explosion combinatoire en machine learning).

Travaux Pratiques (TP) Python

L'objectif de ce TP est de transformer les outils théoriques de ce chapitre (sommes, produits, binôme) en algorithmes opérationnels, et de les appliquer à des problèmes de probabilités, d'analyse numérique et de calcul symbolique. Toutes les implémentations seront réalisées en Python (modules `math`, `itertools`, `matplotlib` optionnel).

Exercice 1 : Sommes et produits *from scratch*

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire deux fonctions `somme(L)` et `produit(L)` qui prennent une liste de nombres et retournent respectivement leur somme et leur produit, sans utiliser `sum()` ni `math.prod()`.

2. **[Bloom : Application | Difficulté : Facile]**

En déduire deux fonctions `somme_indices(f, p, n)` et `produit_indices(f, p, n)` qui calculent respectivement $\sum_{k=p}^n f(k)$ et $\prod_{k=p}^n f(k)$ pour une fonction f donnée et des bornes $p \leq n$. Tester avec $f(k) = k^2$, $p = 1$, $n = 10$.

Exercice 2 : Sommes télescopiques

1. **[Bloom : Application | Difficulté : Facile]**

Calculer $S_n = \sum_{k=1}^n \frac{1}{k(k+1)}$ numériquement pour $n = 10, 100, 1000$ et 10^6 .

2. **[Bloom : Analyse | Difficulté : Moyenne]**

Démontrer (à la main) que $S_n = 1 - \frac{1}{n+1}$, puis vérifier numériquement la formule en comparant les deux calculs et en mesurant l'erreur relative.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

Comparer le temps d'exécution des deux méthodes pour $n = 10^7$. Quelle est la complexité de chaque approche ?

Exercice 3 : Sommes doubles et ordre des boucles

1. **[Bloom : Application | Difficulté : Moyenne]**

Implémenter $T(n) = \sum_{i=1}^n \sum_{j=1}^i (i+j)$ avec deux fonctions : `T_externe_i(n)` (boucle externe sur i) et `T_externe_j(n)` (boucle externe sur j , en réorganisant le domaine de sommation).

2. **[Bloom : Évaluation | Difficulté : Moyenne]**

Vérifier numériquement que les deux versions donnent le même résultat pour $n = 100$. Comparer les temps d'exécution pour $n = 5000$.

3. **[Bloom : Synthèse | Difficulté : Difficile]**

Démontrer la formule fermée $T(n) = \frac{n(n+1)(2n+1)}{6} + \frac{n^2(n+1)}{4}$ (en utilisant les formules de $\sum k$ et $\sum k^2$) et vérifier numériquement.

Exercice 4 : Triangle de Pascal

1. **[Bloom : Application | Difficulté : Facile]**

Écrire une fonction `pascal(N)` qui retourne les $N+1$ premières lignes du triangle de Pascal sous forme de liste de listes, en utilisant uniquement la relation $\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}$.

2. **[Bloom : Application | Difficulté : Moyenne]**

En utilisant le triangle, écrire une fonction `coefficient_binomial(n, k)` qui ne calcule pas les factorielles. Comparer son temps d'exécution à `math.comb(n, k)` pour $n = 1000$.

3. **[Bloom : Analyse | Difficulté : Moyenne]**

Vérifier numériquement les deux propriétés sur les lignes du triangle : symétrie $\binom{n}{k} = \binom{n}{n-k}$, et somme $\sum_{k=0}^n \binom{n}{k} = 2^n$.

Exercice 5 : Récursion vs formule fermée pour $\binom{n}{k}$

1. **[Bloom : Application | Difficulté : Moyenne]**

Implémenter `binom_recuratif(n, k)` avec la relation de Pascal et les conditions aux bords ($\binom{n}{0} = \binom{n}{n} = 1$). Implémenter aussi `binom_factoriel(n, k)` via $\frac{n!}{k!(n-k)!}$.

2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Mesurer le temps d'exécution des deux versions pour $\binom{30}{15}$. La version récursive sans mémorisation est-elle utilisable ?
3. **[Bloom : Synthèse | Difficulté : Difficile]**
Optimiser la version récursive avec `functools.lru_cache` et comparer les performances. Donner la complexité asymptotique de chaque version.

Exercice 6 : Identité de Vandermonde

1. **[Bloom : Synthèse | Difficulté : Moyenne]**
L'identité de Vandermonde affirme :

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}.$$

Écrire une fonction `verifier_vandermonde(m, n, r)` qui calcule les deux membres et retourne `True` ssi ils sont égaux.

2. **[Bloom : Création | Difficulté : Difficile]**
Tester l'identité pour 50 triplets aléatoires (m, n, r) avec $m, n \leq 30$ et $0 \leq r \leq m+n$. Conclure.

Exercice 7 : Loi binomiale et application en probabilités

1. **[Bloom : Application | Difficulté : Moyenne]**
Si X suit une loi binomiale $\mathcal{B}(n, p)$, alors $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$. Implémenter `proba_binomiale(n, k, p)`.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier numériquement que $\sum_{k=0}^n P(X = k) = 1$ pour $(n, p) \in \{(10, 0.3), (50, 0.7), (100, 0.5)\}$. Cette vérification est une conséquence directe de quelle formule ?
3. **[Bloom : Création | Difficulté : Difficile]**
Avec `matplotlib`, tracer la distribution $P(X = k)$ en fonction de k pour $(n, p) = (20, 0.3)$. Calculer aussi l'espérance théorique $E[X] = np$ et la variance $V[X] = np(1-p)$.

Exercice 8 : Développement symbolique du binôme

1. **[Bloom : Création | Difficulté : Difficile]**
Écrire une fonction `developeur_binome(n)` qui retourne, sous forme de chaîne de caractères, le développement de $(a+b)^n$ selon la formule du binôme. Exemple attendu : `developeur_binome(3)` retourne `"a^3 + 3*a^2*b + 3*a*b^2 + b^3"`.
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Comparer la sortie avec `sympy.expand((a+b)**n)` pour $n = 5$. Discuter des limites de votre implémentation comparée au moteur symbolique.

Chapitre 2

Vocabulaire ensembliste

2.1 Éléments de logique

2.1.1 Motivation : L'ordinateur face à l'ambiguïté

Dans la vie de tous les jours, le langage humain est riche mais souvent imprécis. L'intonation ou le contexte suffisent à nous faire comprendre. À l'inverse, en mathématiques et en programmation, une instruction ne peut souffrir d'aucune ambiguïté. Si vous programmez le système de validation d'un site de e-commerce, l'ordinateur doit savoir exactement dans quelles conditions appliquer une réduction. La logique formelle est le langage universel qui permet de traduire des règles métier complexes en assertions mathématiques irréfutables (vraies ou fausses, sans entre-deux).

2.1.2 Approche par problème : Le contrat trompeur

Problème : Une assurance auto propose la clause suivante : *"Vous êtes remboursé si le conducteur a plus de 25 ans OU s'il a son permis depuis plus de 3 ans ET qu'il n'a pas eu d'accident."* Un conducteur de 26 ans, ayant son permis depuis 1 an et ayant eu un accident réclame son remboursement. L'assurance doit-elle payer ?

Résolution : La phrase en français est ambiguë à cause de l'absence de parenthèses.

- Interprétation 1 : (Plus de 25 ans OU Permis > 3 ans) ET (Pas d'accident). Ici, le conducteur a eu un accident, l'assertion totale est Fausse, il n'est pas remboursé.
- Interprétation 2 : (Plus de 25 ans) OU (Permis > 3 ans ET Pas d'accident). Ici, le conducteur a plus de 25 ans, la première condition est Vraie, l'assertion totale est Vraie, il est remboursé !

Pour éviter les litiges, les mathématiques imposent l'usage strict de connecteurs logiques hiérarchisés.

2.1.3 Définitions des connecteurs logiques

Une **proposition** (ou assertion) est un énoncé mathématique qui prend une et une seule valeur de vérité : Vrai (V) ou Faux (F). Soient P et Q deux propositions.

- **Négation (NON)** : Notée $\neg P$. Elle est vraie si P est fausse.
- **Conjonction (ET)** : Notée $P \wedge Q$. Elle est vraie uniquement si P et Q sont toutes les deux vraies.
- **Disjonction (OU)** : Notée $P \vee Q$. Elle est vraie si *au moins* l'une des deux est vraie. (Il s'agit d'un "ou" inclusif).

- **Implication** ($\implies$) : $P \implies Q$ ("Si P alors Q "). Elle n'est fausse que dans un seul cas : quand P est vraie et Q est fausse. Elle est logiquement équivalente à $(\neg P) \vee Q$.
- **Équivalence** ($\iff$) : $P \iff Q$ signifie que P et Q ont toujours la même valeur de vérité.

2.1.4 Les Quantificateurs

Pour formuler des propriétés sur les éléments d'un ensemble, on utilise des quantificateurs :

- **Le quantificateur universel** ($\forall$) : "Pour tout" ou "Quel que soit". Exemple : $\forall x \in \mathbb{R}, x^2 \geq 0$.
- **Le quantificateur existentiel** ($\exists$) : "Il existe au moins un". Exemple : $\exists x \in \mathbb{R}, x^2 = 2$.

Règle de négation : Prendre la négation d'une phrase quantifiée inverse les quantificateurs.

$$\neg(\forall x \in E, P(x)) \iff \exists x \in E, \neg P(x)$$

$$\neg(\exists x \in E, P(x)) \iff \forall x \in E, \neg P(x)$$

2.1.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Tables de vérité - Facile) : Démontrer que l'implication $P \implies Q$ est équivalente à sa contraposée $\neg Q \implies \neg P$. *Correction* : On dresse la table de vérité. Dans les cas où (P, Q) valent respectivement $(V, V), (V, F), (F, V), (F, F)$, l'expression $P \implies Q$ vaut V, F, V, V . L'expression $\neg Q \implies \neg P$ (qui se lit "Si $\neg Q$ est vrai, alors $\neg P$ doit être vrai") donne exactement les mêmes résultats : V, F, V, V . Les deux assertions sont donc équivalentes.

Exemple 2 (Négation d'une phrase courante - Intermédiaire) : Écrire la négation de : "Toutes les voitures de cette marque sont rouges ou noires." *Correction* : Formellement : $\forall v \in \text{Voitures}, (\text{Rouge}(v) \vee \text{Noire}(v))$. La négation transforme le $\forall$ en $\exists$, et par la loi de De Morgan, la négation du OU devient un ET : $\exists v \in \text{Voitures}, (\neg \text{Rouge}(v) \wedge \neg \text{Noire}(v))$. En français : "Il existe au moins une voiture de cette marque qui n'est ni rouge ni noire."

Exemple 3 (Raisonnement par l'absurde - Difficile) : Démontrer que $\sqrt{2}$ est irrationnel ($\sqrt{2} \notin \mathbb{Q}$). *Correction* : On suppose par l'absurde la négation de l'assertion, c'est-à-dire que $\sqrt{2} \in \mathbb{Q}$. Il existe donc deux entiers p et q (avec $q \neq 0$) premiers entre eux tels que $\sqrt{2} = \frac{p}{q}$. En élevant au carré : $2 = \frac{p^2}{q^2} \implies p^2 = 2q^2$. p^2 est pair, donc p est pair. On peut écrire $p = 2k$. En remplaçant : $(2k)^2 = 2q^2 \implies 4k^2 = 2q^2 \implies q^2 = 2k^2$. q^2 est donc pair, ce qui implique que q est pair. Nous avons trouvé que p et q sont tous deux pairs (divisibles par 2), ce qui contredit notre hypothèse initiale qu'ils sont premiers entre eux. L'hypothèse de départ est donc fausse : $\sqrt{2}$ est irrationnel.

2.1.6 Exercices pratiques avec Python

Les opérateurs logiques (**and**, **or**, **not**) sont les piliers des structures conditionnelles en algorithmique. Voici comment vérifier des règles métier strictes à l'aide de la logique booléenne.

```
1 def est_eligible_bourse(etudiant):
2     """
```

```

3     Regle : Un etudiant est eligible s'il a plus de 14 de moyenne
4     OU (s'il a plus de 12 ET qu'il est boursier sur criteres
5         sociaux).
6     """
7     moyenne = etudiant['moyenne']
8     critere_social = etudiant['critere_social']
9
10    # Application stricte de la logique formelle avec parentheses
11    if moyenne >= 14 or (moyenne >= 12 and critere_social == True):
12        return True
13    else:
14        return False
15
16    # Test des propositions
17    etudiant_A = {'nom': 'Alice', 'moyenne': 13, 'critere_social':
18        False}
19    etudiant_B = {'nom': 'Bob', 'moyenne': 12.5, 'critere_social': True
20        }
21
22    print(f"Alice est eligible : {est_eligible_bourse(etudiant_A)}") #
23        Renvoie False
24    print(f"Bob est eligible : {est_eligible_bourse(etudiant_B)}") #
25        Renvoie True

```

2.1.7 Exercices à faire

1. **Exercice 1** : À l'aide d'une table de vérité, démontrer la loi de De Morgan : $\neg(P \wedge Q) \iff (\neg P) \vee (\neg Q)$.
2. **Exercice 2** : Traduire la définition suivante de la limite d'une suite avec des quantificateurs : "Pour tout réel ε strictement positif, il existe un rang N à partir duquel la distance entre u_n et L est strictement inférieure à ε ". Écrire ensuite sa négation.
3. **Exercice 3** : Démontrer par contraposée l'assertion suivante pour tout entier n : "Si $n^2 - 1$ n'est pas divisible par 8, alors n est pair".

2.2 Éléments de la théorie des ensembles

2.2.1 Motivation : L'agrégation de données extraites

Lors de l'extraction automatisée d'offres d'emploi par web scraping sur des portails professionnels (comme *Tunisie Travail* par exemple), les scripts récupèrent souvent de vastes listes d'annonces classées par catégories. Pour croiser ces informations — par exemple, identifier les annonces qui requièrent à la fois des compétences en analyse de données et en vision par ordinateur — il est indispensable de modéliser ces listes comme des ensembles mathématiques. L'algèbre ensembliste permet de formaliser ces intersections, unions et exclusions pour concevoir des algorithmes de filtrage robustes et garantis sans doublons.

2.2.2 Approche par problème : Le paradoxe de l'appartenance

Problème : Considérons l'ensemble de toutes les offres d'emploi d'une plateforme, noté E . Une offre spécifique pour un poste de développeur, notons-la o_1 , appartient à E . On écrit $o_1 \in E$. Si l'on prend maintenant la catégorie regroupant toutes les offres en "Data Analysis", notée D . Peut-on écrire $D \in E$?

Résolution : C'est une erreur fondamentale en algèbre. La catégorie D n'est pas une offre d'emploi unique ; c'est une *sous-collection* d'offres. Il faut donc distinguer strictement l'**appartenance** (un élément dans un ensemble, symbole $\in$) de l'**inclusion** (un ensemble contenu dans un autre, symbole $\subset$). On écrira rigoureusement $D \subset E$.

2.2.3 Définitions fondamentales

Définition 2.1 (Inclusion et Égalité) : Soient A et B deux ensembles.

— A est inclus dans B , noté $A \subset B$, si tout élément de A est aussi un élément de B .

$$\forall x, (x \in A \implies x \in B)$$

— Deux ensembles sont égaux ($A = B$) s'ils ont exactement les mêmes éléments. Pour le prouver en exercice, on démontre presque toujours la **double inclusion** : ($A \subset B$) et ($B \subset A$).

Définition 2.2 (Ensemble des parties) : L'ensemble des parties d'un ensemble E , noté $\mathcal{P}(E)$, est l'ensemble dont les éléments sont tous les sous-ensembles possibles de E . Si $E = \{a, b\}$, alors $\mathcal{P}(E) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$. *Remarque :* L'ensemble vide, noté $\emptyset$, est inclus dans n'importe quel ensemble.

2.2.4 Opérations sur les ensembles

Soient A et B deux parties d'un ensemble de référence E .

- **Union** ($A \cup B$) : Éléments appartenant à A ou à B . ($x \in A \cup B \iff x \in A \vee x \in B$).
- **Intersection** ($A \cap B$) : Éléments appartenant à A et à B . ($x \in A \cap B \iff x \in A \wedge x \in B$).
- **Complémentaire** ($C_E(A)$ ou $\overline{A}$) : Éléments de E n'appartenant pas à A . ($x \in \overline{A} \iff x \notin A$).
- **Différence** ($A \setminus B$) : Éléments de A qui ne sont pas dans B . ($A \setminus B = A \cap \overline{B}$).

2.2.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Opérations simples - Facile) : Soit $E = \{1, 2, 3, 4, 5\}$, $A = \{1, 2, 3\}$ et $B = \{3, 4\}$. Déterminer $A \cap B$, $A \cup B$, et $A \setminus B$. *Correction :* $A \cap B = \{3\}$ (l'élément commun). $A \cup B = \{1, 2, 3, 4\}$ (on fusionne sans répéter le 3). $A \setminus B = \{1, 2\}$ (les éléments de A privés de ceux de B).

Exemple 2 (Preuve d'inclusion - Intermédiaire) : Montrer que pour tous ensembles A et B , on a $A \cap B \subset A$. *Correction :* Soit $x \in A \cap B$. Par définition de l'intersection, cela signifie que la proposition ($x \in A \wedge x \in B$) est vraie. Par conséquent, $x \in A$ est vrai. Tout élément de $A \cap B$ appartient bien à A . Donc $A \cap B \subset A$.

Exemple 3 (Lois de De Morgan pour les ensembles - Difficile) : Démontrer que $\overline{A \cup B} = \overline{A} \cap \overline{B}$. *Correction :* Procédons par équivalence logique en utilisant les résultats

de la Section I. Soit $x \in E$.

$$\begin{aligned}x \in \overline{A \cup B} &\iff \neg(x \in A \cup B) \\&\iff \neg(x \in A \vee x \in B)\end{aligned}$$

D'après les lois de De Morgan en logique :

$$\begin{aligned}&\iff \neg(x \in A) \wedge \neg(x \in B) \\&\iff x \notin A \wedge x \notin B \\&\iff x \in \overline{A} \wedge x \in \overline{B} \\&\iff x \in \overline{A} \cap \overline{B}\end{aligned}$$

L'égalité ensembliste est ainsi démontrée par équivalence.

2.2.6 Exercices pratiques avec Python

Python dispose du type `set`, spécialement conçu pour l'algèbre ensembliste. Il est particulièrement efficace pour croiser des résultats sans avoir à programmer de complexes boucles imbriquées.

```
1  # Identifiants d'offres d'emploi recuperees sur differentes
   # categories d'un site
2  offres_data_analysis = {"ID101", "ID102", "ID103", "ID105"}
3  offres_computer_vision = {"ID103", "ID104", "ID105", "ID106"}
4
5  # 1. Intersection : Offres exigeant simultanement les deux
   # competences
6  offres_cibles = offres_data_analysis.intersection(
   offres_computer_vision)
7  # Syntaxe alternative plus concise : offres_data_analysis &
   offres_computer_vision
8  print(f"Offres correspondantes aux deux : {offres_cibles}")
9
10 # 2. Union : Fusion des deux extractions en supprimant les doublons
11 toutes_offres = offres_data_analysis.union(offres_computer_vision)
12 print(f"Total des annonces uniques extraites : {len(toutes_offres)}")
13
14 # 3. Difference : Offres en Data Analysis sans rapport avec la
   # Vision
15 offres_pures_data = offres_data_analysis - offres_computer_vision
16 print(f"Offres exclusivement Data Analysis : {offres_pures_data}")
```

2.2.7 Exercices à faire

1. **Exercice 1** : Soient A , B et C trois ensembles. Démontrer la distributivité de l'intersection sur l'union : $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.
2. **Exercice 2** : La différence symétrique de deux ensembles, notée $A \Delta B$, est l'ensemble des éléments qui appartiennent à A ou à B , mais pas aux deux à la fois. Montrer que $A \Delta B = (A \cup B) \setminus (A \cap B)$.

3. **Exercice 3** : Sachant que $\mathcal{P}(E)$ est l'ensemble des parties de E , déterminer $\mathcal{P}(\emptyset)$ et $\mathcal{P}(\{1\})$.

2.3 Ensembles finis et dénombrement

2.3.1 Motivation : L'explosion combinatoire en informatique

Le dénombrement est l'art de compter mathématiquement, sans avoir à énumérer chaque cas un par un. En sécurité informatique (cryptographie, mots de passe), en optimisation d'algorithmes, ou en analyse de données, il est indispensable de savoir calculer la taille d'un espace de recherche. Savoir qu'un algorithme de "brute force" devra tester des milliards de combinaisons justifie la recherche de solutions plus élégantes et explique la notion d'explosion combinatoire.

2.3.2 Approche par problème : L'ordre a-t-il de l'importance ?

Problème : Vous disposez d'un groupe de 10 étudiants développeurs.

1. Cas A : Vous devez élire un président, un trésorier et un secrétaire.
2. Cas B : Vous devez former une équipe de 3 développeurs pour un projet.

Combien y a-t-il de choix possibles dans chaque cas ?

Résolution : Dans le Cas A, l'ordre de tirage importe : (Ali=Président, Sara=Trésorière) est différent de (Sara=Présidente, Ali=Trésorier). C'est ce qu'on appelle un **arrangement**. Dans le Cas B, l'équipe Ali, Sara, Karim est rigoureusement identique à l'équipe Sara, Karim, Ali. L'ordre n'a pas d'importance. C'est une **combinaison**. Le dénombrement nous fournit les formules pour calculer ces deux cas distincts instantanément.

2.3.3 Définitions fondamentales

Définition 3.1 (Cardinal) : On dit qu'un ensemble E est fini s'il contient un nombre fini d'éléments. Ce nombre d'éléments est appelé le cardinal de E , noté $\text{Card}(E)$ ou $|E|$. Exemple : Si $E = \{a, b, c\}$, alors $|E| = 3$.

Théorème (Principe d'inclusion-exclusion) : Soient A et B deux ensembles finis. Le cardinal de leur union est donné par :

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Remarque : Si A et B sont disjoints ($A \cap B = \emptyset$), alors $|A \cup B| = |A| + |B|$.

Définition 3.2 (Produit cartésien) : Le produit cartésien de deux ensembles A et B , noté $A \times B$, est l'ensemble des couples (x, y) tels que $x \in A$ et $y \in B$.

$$|A \times B| = |A| \times |B|$$

2.3.4 Les outils de dénombrement

Soit un ensemble fini E de cardinal n , et un entier k tel que $1 \leq k \leq n$.

- **Permutations ($n!$)** : C'est le nombre de façons d'ordonner tous les éléments de E .

$$P_n = n! = 1 \times 2 \times \cdots \times n$$

- **Arrangements** (A_n^k) : C'est le nombre de façons de choisir et d'ordonner k éléments parmi n . (L'ordre compte).

$$A_n^k = \frac{n!}{(n-k)!}$$

- **Combinaisons** ($\binom{n}{k}$) : C'est le nombre de sous-ensembles à k éléments que l'on peut former à partir de E . (L'ordre ne compte pas).

$$\binom{n}{k} = \frac{A_n^k}{k!} = \frac{n!}{k!(n-k)!}$$

2.3.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Cardinal d'une union - Facile) : Dans une promotion de 100 étudiants, 60 étudient Python, 50 étudient Java, et 20 étudient les deux. Combien d'étudiants étudient au moins l'un des deux langages ? *Correction* : On cherche $|P \cup J|$.

$$|P \cup J| = |P| + |J| - |P \cap J| = 60 + 50 - 20 = 90$$

Il y a 90 étudiants. (Il en reste donc 10 qui n'étudient ni l'un ni l'autre).

Exemple 2 (Arrangements ou combinaisons ? - Intermédiaire) : Un code PIN est composé de 4 chiffres distincts (parmi 10). Combien de codes PIN existe-t-il ? *Correction* : L'ordre des chiffres change le code (1234 est différent de 4321). On choisit 4 éléments parmi 10, avec ordre et sans répétition. C'est un arrangement :

$$A_{10}^4 = \frac{10!}{(10-4)!} = \frac{10!}{6!} = 10 \times 9 \times 8 \times 7 = 5040$$

Exemple 3 (Dénombrement complexe - Difficile) : Combien y a-t-il d'anagrammes du mot "DATA" ? *Correction* : Le mot contient 4 lettres, on pourrait penser qu'il y a $4! = 24$ permutations. Cependant, la lettre "A" est répétée 2 fois. Permuter le premier "A" avec le deuxième "A" ne change pas le mot. Il faut donc diviser le nombre total de permutations par le nombre de permutations des lettres identiques :

$$\frac{4!}{2!} = \frac{24}{2} = 12$$

2.3.6 Exercices pratiques avec Python

Le module `itertools` de la bibliothèque standard de Python est l'outil parfait pour générer et manipuler les arrangements et les combinaisons.

```

1 import itertools
2 import math
3
4 etudiants = ["Ali", "Sara", "Karim"]
5
6 # 1. Permutations : Tous les ordres possibles de passage à l'oral
7 ordre_passage = list(itertools.permutations(etudiants))
8 print(f"Permutations_({math.factorial(len(etudiants))})_cas:")
9 for cas in ordre_passage:
```

```

10     print(cas)
11
12 # 2. Combinaisons : Former un binome de projet (l'ordre ne compte
    pas)
13 binomes = list(itertools.combinations(etudiants, 2))
14 print(f"\nCombinaisons_(binomes_possibles):")
15 for b in binomes:
16     print(b)
17
18 # Verification mathematique du nombre de binomes (3 parmi 3 -> 3! /
    (2!*1!))
19 print(f"Nombre_mathematique_de_binomes:{math.comb(3,2)}")

```

2.3.7 Exercices à faire

1. **Exercice 1** : Dans un jeu de 32 cartes, combien y a-t-il de "mains" de 5 cartes possibles ?
2. **Exercice 2** : Démontrer algébriquement la formule de symétrie : $\binom{n}{k} = \binom{n}{n-k}$. Que signifie cette formule concrètement en termes de choix ?
3. **Exercice 3** : Vous devez constituer un mot de passe de 8 caractères comprenant exactement 3 chiffres et 5 lettres. Sachant que vous avez le choix entre 10 chiffres et 26 lettres (sans répétition), combien de mots de passe pouvez-vous créer ? (*Indice : Choisissez d'abord les positions des chiffres*).

2.4 Applications et relations : Ordre, équivalence et quotient

2.4.1 Motivation : Regroupement et classification de données

Lorsqu'un algorithme de vision par ordinateur analyse la vidéo d'un match de football, il détecte des dizaines d'objets (les joueurs). Pour que l'analyse ait un sens, il faut que la machine comprenne que certains joueurs appartiennent à la même équipe. En mathématiques, on ne va pas créer un ensemble pour chaque équipe de manière arbitraire : on va définir une **relation** (ex : "porte un maillot de la même couleur"). Cette relation va naturellement découper l'ensemble de tous les joueurs en sous-groupes disjoints (les équipes). C'est ce que l'on appelle une classe d'équivalence et un ensemble quotient, concepts fondamentaux pour tout algorithme de *clustering* (regroupement) en Data Science.

2.4.2 Approche par problème : Le tri des doublons

Problème : Vous avez écrit un script web scraping pour aspirer des offres d'emploi sur un portail web. Souvent, la même offre est republiée plusieurs fois à des dates différentes. Comment nettoyer cette base de données ?

Résolution : On définit une relation $\mathcal{R}$ entre deux offres A et B : " $A\mathcal{R}B$ si elles ont exactement le même titre et la même entreprise". Cette relation possède trois propriétés évidentes :

- Une offre est identique à elle-même (Réflexivité).

- Si A est identique à B , alors B est identique à A (Symétrie).
- Si A est identique à B , et B identique à C , alors A est identique à C (Transitivité).

Toute relation vérifiant ces trois critères est une **relation d'équivalence**. Elle permet de regrouper toutes les annonces identiques dans un même "sac" (la classe d'équivalence), et de ne garder qu'un seul représentant par sac.

2.4.3 Définitions des Relations

Soit E un ensemble. Une relation binaire $\mathcal{R}$ sur E relie certains éléments de E entre eux. On écrit $x\mathcal{R}y$ pour dire que " x est en relation avec y ".

Définition 4.1 (Relation d'équivalence) : Une relation $\mathcal{R}$ sur E est une relation d'équivalence si elle est :

1. **Réflexive** : $\forall x \in E, x\mathcal{R}x$
2. **Symétrique** : $\forall (x, y) \in E^2, (x\mathcal{R}y \implies y\mathcal{R}x)$
3. **Transitive** : $\forall (x, y, z) \in E^3, ((x\mathcal{R}y \text{ et } y\mathcal{R}z) \implies x\mathcal{R}z)$

Définition 4.2 (Classe d'équivalence et Ensemble Quotient) : Soit $\mathcal{R}$ une relation d'équivalence sur E . La classe d'équivalence d'un élément $x \in E$, notée $\bar{x}$ ou $cl(x)$, est l'ensemble de tous les éléments de E qui sont en relation avec x .

$$\bar{x} = \{y \in E \mid y\mathcal{R}x\}$$

L'ensemble de toutes ces classes forme une **partition** de E (elles recouvrent tout E et sont deux à deux disjointes). Cet ensemble des classes est appelé **Ensemble Quotient**, noté $E/\mathcal{R}$.

Définition 4.3 (Relation d'ordre) : Une relation $\leq$ est une relation d'ordre si elle est réflexive, transitive et **antisymétrique** :

$$\forall (x, y) \in E^2, ((x \leq y \text{ et } y \leq x) \implies x = y)$$

Un ordre est dit *total* si tous les éléments sont comparables ($x \leq y$ ou $y \leq x$). Sinon, il est *partiel*.

2.4.4 Définitions des Applications (Fonctions)

Soit f une application d'un ensemble de départ E vers un ensemble d'arrivée F .

- **Injective** : Deux éléments distincts de E ont toujours des images distinctes dans F .

$$\forall (x, x') \in E^2, (f(x) = f(x') \implies x = x')$$

- **Surjective** : Tout élément de F possède au moins un antécédent dans E .

$$\forall y \in F, \exists x \in E, f(x) = y$$

- **Bijective** : L'application est à la fois injective et surjective. Tout élément de l'arrivée possède un *unique* antécédent. Cela implique l'existence d'une fonction réciproque f^{-1} .

2.4.5 Exemples résolus (Difficulté ascendante)

Exemple 1 (Relation d'ordre - Facile) : Dans l'ensemble des parties $\mathcal{P}(E)$, la relation d'inclusion ($\subset$) est-elle une relation d'ordre total ou partiel? *Correction* : C'est un ordre. Mais il est **partiel**. Par exemple, si $E = \{1, 2\}$, les ensembles $A = \{1\}$ et $B = \{2\}$ ne sont pas comparables : on n'a ni $A \subset B$, ni $B \subset A$.

Exemple 2 (Relation d'équivalence et modulo - Intermédiaire) : Sur $\mathbb{Z}$, on définit $x \mathcal{R} y \iff x - y$ est un multiple de 3. (On note $x \equiv y \pmod{3}$). Quelles sont les classes d'équivalence? *Correction* : Il y a exactement 3 classes d'équivalence, correspondant aux restes possibles de la division euclidienne par 3 : $\bar{0} = \{\dots, -3, 0, 3, 6, \dots\}$ $\bar{1} = \{\dots, -2, 1, 4, 7, \dots\}$ $\bar{2} = \{\dots, -1, 2, 5, 8, \dots\}$ L'ensemble quotient $\mathbb{Z}/3\mathbb{Z}$ est donc $\{\bar{0}, \bar{1}, \bar{2}\}$.

Exemple 3 (Preuve de bijectivité - Difficile) : Soit $f : \mathbb{R} \setminus \{1\} \rightarrow \mathbb{R} \setminus \{2\}$ définie par $f(x) = \frac{2x+1}{x-1}$. Montrer que f est bijective. *Correction* : On va montrer que pour tout $y \in \mathbb{R} \setminus \{2\}$, l'équation $f(x) = y$ admet une unique solution $x \in \mathbb{R} \setminus \{1\}$.

$$\frac{2x+1}{x-1} = y \iff 2x+1 = y(x-1) \iff 2x - yx = -y - 1 \iff x(2-y) = -y - 1$$

Puisque $y \neq 2$, on peut diviser par $(2-y)$:

$$x = \frac{-y-1}{2-y} = \frac{y+1}{y-2}$$

Cette expression est unique et bien définie (le dénominateur ne s'annule pas). f est donc bijective, et sa réciproque est $f^{-1}(y) = \frac{y+1}{y-2}$.

2.4.6 Exercices pratiques avec Python

Les classes d'équivalence sont le fondement de l'opération GROUP BY en bases de données. En Python, on utilise souvent des dictionnaires pour construire l'ensemble quotient (partitionner la donnée).

```
1 # Simulation de donnees (ex: resultats d'un algorithme d'analyse
   # video de football)
2 # Chaque joueur est detecte avec une coordonnee et la couleur
   # dominante de son maillot
3 joueurs = [
4     {"id": 1, "couleur": "Rouge"},
5     {"id": 2, "couleur": "Bleu"},
6     {"id": 3, "couleur": "Rouge"},
7     {"id": 4, "couleur": "Bleu"}
8 ]
9
10 # Creation de l'ensemble quotient : on regroupe les joueurs (
   # classes d'equivalence)
11 # selon la relation "a la meme couleur de maillot que"
12 equipes = {}
13
14 for j in joueurs:
15     c = j["couleur"]
16     if c not in equipes:
```

```

17     equipes[c] = [] # Creation d'une nouvelle classe d'
18         equivalence
19     equipes[c].append(j["id"])
20 print("Ensemble_Quotient_(Equipes_formees):")
21 for couleur, membres in equipes.items():
22     print(f"Classe_{couleur}:_Joueurs_{membres}")

```

2.4.7 Exercices à faire

1. **Exercice 1** : Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ définie par $f(x) = x^2$. Cette fonction est-elle injective ? Surjective ? Bijective ? Justifiez rigoureusement.
2. **Exercice 2** : Sur l'ensemble des mots de la langue française, on définit la relation $\mathcal{R}$ par : " $mot_1 \mathcal{R} mot_2$ s'ils ont la même première lettre". Montrer que c'est une relation d'équivalence et décrire l'ensemble quotient.
3. **Exercice 3** : Démontrer que la composée de deux applications injectives est une application injective.

Pour aller plus loin : Ensembles, Relations et Intelligence Artificielle

Les notions de logique, d'ensembles et de relations ne sont pas de simples abstractions mathématiques : elles forment l'architecture logique sur laquelle reposent les bases de données modernes et les algorithmes d'Intelligence Artificielle. Pour clore ce chapitre, voici comment ces concepts prennent vie en Data Science.

Piste 1 : Théorie des Ensembles, Algèbre de Boole et Web Scraping

Lorsqu'un script d'extraction automatisée (web scraping) parcourt des plateformes d'emploi pour construire une base de données, il récolte un volume massif de texte brut. Transformer ce texte en données exploitables repose sur la théorie des ensembles.

Si l'on définit l'ensemble A comme les offres exigeant la compétence "Python", et B comme les offres proposant du "Télétravail", la requête permettant de trouver le job idéal est une simple **intersection** ($A \cap B$). À grande échelle, les moteurs de recherche (comme Elasticsearch) ou les bases de données relationnelles optimisent ces filtres en utilisant l'algèbre de Boole. Les lois de De Morgan, que nous avons démontrées, sont d'ailleurs utilisées par les optimiseurs de requêtes SQL pour réduire le temps de calcul lors du croisement de millions de tables.

Sujet de recherche : Explorez le concept d'**Indexation Inversée** (*Inverted Index*). Comment les moteurs de recherche utilisent-ils les opérations ensemblistes (unions et intersections de listes d'identifiants) pour renvoyer des résultats en quelques millisecondes ?

Piste 2 : Relations d'équivalence et Clustering (Apprentissage non supervisé)

En Machine Learning, on distingue l'apprentissage supervisé (où l'on connaît la réponse attendue) de l'apprentissage non supervisé. L'une des tâches phares du non super-

visé est le **Clustering** (ou partitionnement de données).

Prenons l'exemple de l'analyse vidéo d'un match de football par vision par ordinateur. L'algorithme détecte des dizaines de "points" en mouvement sur le terrain. Pour que l'analyse soit exploitable, l'IA doit grouper ces points. Mathématiquement, l'algorithme cherche à construire une **relation d'équivalence** $\mathcal{R}$ du type : " $x\mathcal{R}y$ si le joueur x et le joueur y portent une couleur de maillot similaire ou courent dans une formation similaire". En appliquant cette relation, l'algorithme génère un **ensemble quotient** : il sépare automatiquement l'ensemble des joueurs en classes d'équivalence distinctes (Équipe 1, Équipe 2, et les Arbitres).

Sujet de recherche : Renseignez-vous sur l'algorithme des **K-Means** (K-Moyennes) ou sur **DBSCAN**. Comment ces algorithmes créent-ils des partitions (classes d'équivalence) dans un espace de données à N dimensions en se basant sur une relation de "distance" mathématique ?

Piste 3 : Applications (Fonctions) et Modélisation

Tout modèle de Machine Learning est, fondamentalement, une **application** mathématique f qui relie un espace de départ E (les données d'entrée, ou *features*) à un espace d'arrivée F (les prédictions, ou *labels*).

- Si l'on souhaite que notre algorithme identifie de manière unique chaque utilisateur à partir d'une photo de son visage, nous exigeons que notre modèle f se rapproche le plus possible d'une application **injective** (deux visages différents ne doivent jamais donner le même identifiant).
- Si l'application ne peut prendre que des valeurs discrètes dans l'ensemble d'arrivée (ex : $F = \{\text{Spam}, \text{Non Spam}\}$), on parle d'un problème de **Classification**.

Sujet de recherche : Comment les réseaux de neurones artificiels agissent-ils comme des fonctions mathématiques complexes, et pourquoi étudie-t-on la notion d'espace vectoriel (notre prochain grand chapitre !) pour comprendre la dimensionnalité de E et F ?

Partie A : Entraînement technique et théorique

Exercice 1 : Logique formelle

1. Dresser la table de vérité de la proposition suivante : $P \implies (Q \vee R)$.
2. Soit f une fonction de $\mathbb{R}$ dans $\mathbb{R}$. Écrire la négation formelle de la proposition suivante :

$$\forall M \in \mathbb{R}, \exists A \in \mathbb{R}, \forall x \in \mathbb{R}, (x > A \implies f(x) > M)$$

3. Démontrer par l'absurde que si $A \cap B = A \cup B$, alors $A = B$.

Exercice 2 : Opérations sur les ensembles

Soient A , B et C trois sous-ensembles d'un ensemble de référence E .

1. Démontrer algébriquement la propriété d'absorption : $A \cup (A \cap B) = A$.
2. Montrer que : $(A \setminus B) \setminus C = A \setminus (B \cup C)$.
3. La différence symétrique est définie par $A \Delta B = (A \cup B) \setminus (A \cap B)$. Résoudre l'équation $A \Delta X = \emptyset$, d'inconnue $X \subset E$.

Exercice 3 : Dénombrement et Cardinaux

Soit E un ensemble fini de cardinal n .

1. Combien y a-t-il d'applications possibles de E vers un ensemble F de cardinal p ?
2. Combien y a-t-il de bijections possibles de E vers lui-même ?
3. Démontrer que le nombre total de sous-ensembles de E (le cardinal de $\mathcal{P}(E)$) est égal à 2^n . (*Indice : utiliser la formule du binôme de Newton vue au chapitre précédent*).

Exercice 4 : Applications et bijectivité

Soit l'application $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ définie par $f(x, y) = (x + y, x - y)$.

1. Montrer que f est injective.
2. Montrer que f est surjective.
3. En déduire l'expression de sa bijection réciproque $f^{-1}(x, y)$.

Partie B : Modélisation Algorithmique

Exercice 5 : Algèbre ensembliste et Web Scraping

Un script d'extraction automatisée est déployé pour récupérer des offres d'emploi sur le site *Tunisie Travail*. On note E l'ensemble total des offres extraites. On définit les sous-ensembles suivants :

- P : l'ensemble des offres mentionnant "Python".
 - D : l'ensemble des offres mentionnant "Data Analysis".
 - A : l'ensemble des offres mentionnant "Power Automate".
1. Exprimer à l'aide des opérations ensemblistes ($\cap, \cup, \setminus, \overline{}$) l'ensemble des offres qui requièrent Python et l'analyse de données, mais qui ne mentionnent pas Power Automate.
 2. Traduire en français la signification de l'ensemble suivant : $(P \cup D) \cap \overline{A}$.
 3. Si le script a extrait $|E| = 500$ offres, sachant que $|P| = 120$, $|A| = 40$ et que $P \cap A = \emptyset$, combien d'offres ne mentionnent ni Python ni Power Automate ?

Exercice 6 : Relations d'équivalence en Vision par ordinateur

Lors de l'analyse vidéo d'un match de football par un algorithme de traitement d'images, on note J l'ensemble des joueurs détectés sur le terrain à l'instant t . L'algorithme analyse les pixels de la zone délimitant chaque joueur et calcule une valeur $c(x)$ représentant le code couleur dominant de son maillot. On définit sur J la relation $\mathcal{R}$ par :

$$\text{Pour tous joueurs } x \text{ et } y, \quad x\mathcal{R}y \iff c(x) = c(y)$$

1. Démontrer que $\mathcal{R}$ est une relation d'équivalence sur J .
2. Que représente concrètement la classe d'équivalence d'un joueur x (notée $\bar{x}$) sur le terrain ?
3. De combien d'éléments l'ensemble quotient $J/\mathcal{R}$ est-il composé lors d'un match classique (en supposant que les maillots des deux équipes et des arbitres sont parfaitement identifiés) ?

Travaux Pratiques (TP) Python

L'objectif de ce TP est d'implémenter les concepts de logique formelle, théorie des ensembles, dénombrement et relations sur machine. On exploitera les modules `itertools`, `math.comb`, `collections` et `matplotlib` (optionnel).

Exercice 1 : Tables de vérité automatisées

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire des fonctions `NEG(P)`, `ET(P, Q)`, `OU(P, Q)`, `IMP(P, Q)`, `EQUIV(P, Q)` qui prennent des booléens et retournent le résultat des connecteurs logiques.
2. **[Bloom : Application | Difficulté : Facile]**
Écrire `table_verite(formule, n)` qui prend une formule logique sous forme d'une fonction Python à n variables (ex. `lambda P, Q: IMP(P, Q)`) et imprime sa table de vérité complète sur les 2^n combinaisons.
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier automatiquement que `IMP(P, Q)` et `OU(NEG(P), Q)` ont la même table de vérité (équivalence logique). Vérifier aussi la contraposée : $P \implies Q$ et $\neg Q \implies \neg P$.

Exercice 2 : Détection de tautologies

1. **[Bloom : Synthèse | Difficulté : Moyenne]**
Une *tautologie* est une formule logique vraie pour toutes les valuations de ses variables. Écrire `est_tautologie(formule, n)` qui retourne `True` si la formule est une tautologie sur ses n variables.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier que les formules suivantes sont des tautologies :
 - $P \vee \neg P$ (tiers exclu) ;
 - $\neg(P \wedge Q) \iff (\neg P \vee \neg Q)$ (loi de De Morgan) ;
 - $((P \implies Q) \wedge (Q \implies R)) \implies (P \implies R)$ (transitivité de l'implication).

Exercice 3 : Opérations ensemblistes sans set

1. **[Bloom : Application | Difficulté : Facile]**
En représentant un ensemble par une liste sans doublons, écrire les fonctions `union(A, B)`, `intersection(A, B)`, `difference(A, B)`, `difference_symetrique(A, B)` sans utiliser le type `set` de Python.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier numériquement la *distributivité* de $\cap$ sur $\cup$: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ pour 100 triplets aléatoires d'ensembles d'entiers.
3. **[Bloom : Synthèse | Difficulté : Moyenne]**
Écrire `parties(E)` qui retourne l'ensemble des parties $\mathcal{P}(E)$ d'un ensemble fini. Vérifier que $|\mathcal{P}(E)| = 2^{|E|}$ pour $|E| \leq 8$.

Exercice 4 : Test des propriétés d'une relation

1. **[Bloom : Application | Difficulté : Moyenne]**
Une relation binaire sur un ensemble fini E peut être codée comme une fonction $r : E \times E \rightarrow \{0, 1\}$ ou comme un ensemble de couples. Écrire `est_reflexive(r, E)`, `est_symetrique(r, E)`, `est_transitive(r, E)`, `est_antisymetrique(r, E)`.

2. **[Bloom : Synthèse | Difficulté : Moyenne]**
En déduire `est_equivalence(r, E)` et `est_ordre(r, E)` (ordre = réflexive, antisymétrique, transitive).
3. **[Bloom : Création | Difficulté : Difficile]**
Tester sur $E = \{1, 2, 3, 4, 5, 6\}$:
— la relation « x divise y » : ordre ? équivalence ?
— la relation « $x \equiv y \pmod{3}$ » : ordre ? équivalence ?

Exercice 5 : Dénombrement et combinatoire

1. **[Bloom : Compréhension | Difficulté : Facile]**
Implémenter `factorielle(n)`, `arrangement(n, k)` ($A_n^k = n!/(n-k)!$) et `combinaison(n, k)` ($\binom{n}{k}$) sans utiliser `math`.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Comparer le résultat de votre `combinaison(20, 10)` à `math.comb(20, 10)`.
3. **[Bloom : Application | Difficulté : Moyenne]**
En utilisant `itertools.permutations`, `permutations(L, k)` (pour les arrangements) et `combinations(L, k)`, énumérer effectivement tous les arrangements et combinaisons de $\{a, b, c, d\}$ par $k = 2$ et compter pour vérifier les formules.

Exercice 6 : Principe d'inclusion-exclusion

1. **[Bloom : Application | Difficulté : Moyenne]**
Implémenter `cardinal_union(A, B)` qui calcule $|A \cup B|$ *uniquement* via la formule $|A| + |B| - |A \cap B|$ (sans construire l'union).
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Généraliser à trois ensembles : $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$. Écrire `cardinal_union_3(A, B, C)` et tester sur trois sous-ensembles aléatoires de $\{1, \dots, 100\}$.

Exercice 7 : Classes d'équivalence et partitions

1. **[Bloom : Application | Difficulté : Moyenne]**
Étant donnée une relation d'équivalence $\mathcal{R}$ sur E (codée comme un ensemble de couples ou une fonction), écrire `classes_equivalence(R, E)` qui retourne la liste des classes d'équivalence (= la *partition* associée).
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester sur $E = \{0, 1, 2, \dots, 11\}$ avec la relation $x \mathcal{R} y \iff x \equiv y \pmod{3}$.
Combien de classes ? Quels sont leurs cardinaux ?

Exercice 8 : Application — détection de doublons (web scraping)

1. **[Bloom : Création | Difficulté : Difficile]**
Vous récupérez une liste d'offres d'emploi sous forme de dictionnaires `{titre, entreprise, lieu, salaire}`. Vous voulez *regrouper les offres identiques* en utilisant la relation : « deux offres sont équivalentes si leurs (titre, entreprise) sont identiques après normalisation ».
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Écrire `normaliser(s)` qui passe une chaîne en minuscules et supprime les espaces superflus, puis `regrouper_offres(offres)` qui retourne la partition de

la liste sous forme de dictionnaire {(titre_norm, entreprise_norm) : [liste des offres associées]}.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

Tester avec une liste fictive de 10 offres dont 3 paires de doublons (par variation de casse ou d'espaces). Combien de classes après regroupement ?

Chapitre 3

Rappels d'arithmétique dans l'ensemble des entiers relatifs

Ce chapitre a pour objectif de consolider les bases de l'arithmétique dans l'ensemble des entiers relatifs ($\mathbb{Z}$).

3.1 Division euclidienne, congruence

3.1.1 Motivation

En informatique et en sciences de l'ingénieur, les nombres entiers ne sont pas de simples compteurs. L'arithmétique modulaire, qui repose sur la division euclidienne et les congruences, est le moteur de la cryptographie moderne (comme les protocoles qui sécurisent nos communications), mais aussi des algorithmes de hachage et de la génération de nombres pseudo-aléatoires. Maîtriser ces concepts permet de comprendre comment les machines traitent, transforment et sécurisent l'information de manière cyclique.

3.1.2 Approche par problème

Imaginons un capteur dans un système industriel qui enregistre une mesure de température toutes les 7 heures. Si le capteur est activé un lundi à 00h00, quel jour de la semaine la 100^{ème} mesure sera-t-elle prise ?

Ce type de problème cyclique ne nécessite pas de calculer le temps total écoulé de manière absolue, mais plutôt de s'intéresser au *reste* de la division du temps total par la durée d'un cycle. C'est précisément l'objet de la division euclidienne et de la notion de congruence, qui permettent de simplifier les grands nombres en se concentrant uniquement sur leur comportement périodique.

3.1.3 Définitions et théorèmes

La division euclidienne

Théorème 3.1.1 (Division euclidienne dans $\mathbb{Z}$). *Soit $a \in \mathbb{Z}$ et $b \in \mathbb{Z}^*$. Il existe un unique couple d'entiers $(q, r) \in \mathbb{Z} \times \mathbb{N}$ tel que :*

$$a = bq + r \quad \text{avec} \quad 0 \leq r < |b|$$

Remarque 3.1.1. Dans cette écriture, a est appelé le **dividende**, b le **diviseur**, q le **quotient** et r le **reste**. Il est crucial de noter que le reste r est toujours **positif ou nul**, même si a ou b sont négatifs.

Congruences

La notion de congruence permet de formaliser mathématiquement ces phénomènes cycliques en travaillant directement sur les restes.

Définition 3.1.1 (Congruence modulo n). Soit $n \in \mathbb{N}^*$. Deux entiers relatifs a et b sont dits congrus modulo n si et seulement si n divise leur différence $a - b$. On note :

$$a \equiv b \pmod{n}$$

Ceci est équivalent à dire que a et b ont le même reste dans la division euclidienne par n .

Propriété 3.1.1 (Opérations sur les congruences). La relation de congruence est une relation d'équivalence compatible avec l'addition et la multiplication. Si $a \equiv b \pmod{n}$ et $c \equiv d \pmod{n}$, alors :

- $a + c \equiv b + d \pmod{n}$
- $ac \equiv bd \pmod{n}$
- Pour tout entier naturel k , $a^k \equiv b^k \pmod{n}$

3.1.4 Exemples de difficulté croissante

Exemple 3.1.1 (Division avec des entiers négatifs). Effectuer la division euclidienne de $a = -17$ par $b = 5$.

Correction : On cherche $q \in \mathbb{Z}$ et $r \in \mathbb{N}$ tels que $-17 = 5q + r$ avec $0 \leq r < 5$. Si l'on prend $q = -3$, on a $5 \times (-3) = -15$. Le reste serait de -2 , ce qui est impossible car le reste doit être positif. On doit donc « descendre » au multiple inférieur : on prend $q = -4$. On obtient : $-17 = 5 \times (-4) + 3$. Le quotient est donc $q = -4$ et le reste est $r = 3$.

Exemple 3.1.2 (Calcul modulaire sur de grandes puissances). Déterminer le reste de la division euclidienne de 3^{2026} par 7.

Correction : L'objectif est de trouver une puissance de 3 qui soit congrue à 1 ou -1 modulo 7. Calculons les premières puissances :

- $3^1 \equiv 3 \pmod{7}$
- $3^2 = 9 \equiv 2 \pmod{7}$
- $3^3 = 27 \equiv -1 \pmod{7}$

C'est ce qu'il nous faut ! Exprimons l'exposant 2026 en fonction de 3. La division euclidienne de 2026 par 3 donne : $2026 = 3 \times 675 + 1$. On peut donc écrire :

$$3^{2026} = 3^{3 \times 675 + 1} = (3^3)^{675} \times 3^1$$

En passant aux congruences modulo 7 :

$$3^{2026} \equiv (-1)^{675} \times 3 \pmod{7}$$

Comme 675 est impair, $(-1)^{675} = -1$.

$$3^{2026} \equiv -1 \times 3 \pmod{7} \equiv -3 \pmod{7}$$

Puisque le reste doit être positif, on ajoute 7 : $-3 + 7 = 4$.

$$3^{2026} \equiv 4 \pmod{7}$$

Le reste de la division euclidienne de 3^{2026} par 7 est donc 4.

3.1.5 Exercice pratique avec Python

Objectif : Implémenter la division euclidienne et vérifier des congruences.

En Python, les opérateurs `//` (quotient) et `%` (modulo) calculent la division euclidienne en respectant la règle mathématique du reste positif, même pour les entiers négatifs. C'est un avantage majeur par rapport à d'autres langages comme le C ou le Java.

```
def division_euclidienne(a, b):
    """Calcule et affiche le quotient et le reste de a par b."""
    if b == 0:
        return "Erreur : division par zéro"
    q = a // b
    r = a % b
    print(f"Dividende {a} = {b} * {q} + {r}")
    return q, r

def sont_congrus(a, b, n):
    """Vérifie si a et b sont congrus modulo n."""
    return (a % n) == (b % n)

# --- Tests ---
print("--- Division euclidienne ---")
division_euclidienne(17, 5)
division_euclidienne(-17, 5) # Python gère le reste positif !

print("\n--- Congruences ---")
# Vérifions le résultat de l'exemple 2 : 3^2026 = 4 mod 7
# En Python, 3**2026 calcule la puissance exacte instantanément.
resultat = sont_congrus(3**2026, 4, 7)
print(f"3^2026 est-il congru à 4 modulo 7 ? {resultat}")
```

3.2 PGCD et PPCM

3.2.1 Motivation

Dans de nombreux domaines de l'ingénierie et de l'informatique, nous sommes confrontés à des problèmes d'optimisation et de synchronisation. Comment diviser une surface de stockage en blocs carrés les plus grands possibles sans perte d'espace ? Comment déterminer le moment où deux processus exécutés en parallèle, avec des temps de cycle différents,

vont se superposer ? Le PGCD (Plus Grand Commun Diviseur) et le PPCM (Plus Petit Commun Multiple) sont les outils mathématiques fondamentaux pour répondre à ces questions.

3.2.2 Approche par problème

Imaginons deux tâches dans un système d'exploitation embarqué. La tâche A s'exécute toutes les 15 millisecondes et la tâche B toutes les 25 millisecondes. Si elles démarrent exactement en même temps à $t = 0$, au bout de combien de temps s'exécuteront-elles à nouveau simultanément ?

Pour résoudre cela, on cherche un multiple commun à 15 et 25, et idéalement le plus petit possible pour anticiper le premier conflit : c'est le PPCM. À l'inverse, si l'on souhaite diviser une mémoire de 1500 octets et une autre de 2500 octets en segments de taille identique, la plus grande taille possible pour ces segments sera donnée par le PGCD.

3.2.3 Définitions et théorèmes

Le Plus Grand Commun Diviseur (PGCD)

Définition 3.2.1 (PGCD). Soient a et b deux entiers relatifs non tous les deux nuls. L'ensemble des diviseurs communs à a et b est une partie finie et non vide de $\mathbb{Z}$. Le plus grand élément de cet ensemble est un entier naturel appelé Plus Grand Commun Diviseur de a et b . On le note $\text{PGCD}(a, b)$ ou simplement $a \wedge b$.

Définition 3.2.2 (Nombres premiers entre eux). Deux entiers relatifs a et b sont dits premiers entre eux si et seulement si leur seul diviseur commun positif est 1, c'est-à-dire :

$$\text{PGCD}(a, b) = 1$$

Le Plus Petit Commun Multiple (PPCM)

Définition 3.2.3 (PPCM). Soient a et b deux entiers relatifs non nuls. L'ensemble des multiples communs strictement positifs de a et b possède un plus petit élément. Cet entier naturel est appelé Plus Petit Commun Multiple de a et b . On le note $\text{PPCM}(a, b)$ ou simplement $a \vee b$.

Relation fondamentale

Il existe un lien analytique très fort entre le PGCD et le PPCM de deux nombres.

Théorème 3.2.1 (Lien entre PGCD et PPCM). *Pour tous entiers relatifs non nuls a et b , le produit de leur PGCD et de leur PPCM est égal à la valeur absolue de leur produit :*

$$\text{PGCD}(a, b) \times \text{PPCM}(a, b) = |ab|$$

3.2.4 Exemples de difficulté croissante

Exemple 3.2.1 (Calcul par décomposition en facteurs premiers). Déterminer le PGCD et le PPCM de $a = 60$ et $b = 84$.

Correction : On décompose d'abord en produit de facteurs premiers. $60 = 2^2 \times 3^1 \times 5^1$
 $84 = 2^2 \times 3^1 \times 7^1$

- Pour le **PGCD**, on prend les facteurs premiers *communs* avec la *plus petite* puissance :

$$\text{PGCD}(60, 84) = 2^2 \times 3^1 = 12$$

- Pour le **PPCM**, on prend *tous* les facteurs premiers présents avec la *plus grande* puissance :

$$\text{PPCM}(60, 84) = 2^2 \times 3^1 \times 5^1 \times 7^1 = 420$$

On peut vérifier le théorème : $12 \times 420 = 5040$ et $60 \times 84 = 5040$.

Exemple 3.2.2 (Démonstration algébrique). Montrer que pour tout entier naturel n , les nombres $a = n$ et $b = n + 1$ sont premiers entre eux.

Correction : Soit d un diviseur commun positif à n et $n + 1$. Par définition, il existe des entiers k_1 et k_2 tels que $n = d \cdot k_1$ et $n + 1 = d \cdot k_2$. Si d divise n et $n + 1$, il divise aussi toute combinaison linéaire de ces deux nombres, et en particulier leur différence :

$$(n + 1) - n = d \cdot k_2 - d \cdot k_1 = d(k_2 - k_1)$$

Donc $1 = d(k_2 - k_1)$. Le seul entier naturel qui divise 1 est 1. Donc $d = 1$. Leur seul diviseur commun étant 1, on a bien $\text{PGCD}(n, n + 1) = 1$. Ils sont premiers entre eux.

3.2.5 Exercice pratique avec Python

Objectif : Utiliser la bibliothèque `math` de Python pour calculer le PGCD et déduire le PPCM.

Depuis Python 3.9, les fonctions `gcd` (Greatest Common Divisor) et `lcm` (Least Common Multiple) sont directement incluses dans le module mathématique standard.

```
import math

def calculer_pgcd_ppcm(a, b):
    """Affiche le PGCD et le PPCM de deux entiers."""
    if a == 0 and b == 0:
        return "Le PGCD de 0 et 0 n'est pas défini."

    # Utilisation des fonctions natives de Python
    pgcd_val = math.gcd(a, b)

    # Si on utilise une version de Python < 3.9, on peut déduire le PPCM :
    # ppcm_val = abs(a * b) // pgcd_val
    # Sinon, on utilise math.lcm :
    ppcm_val = math.lcm(a, b)

    print(f"Pour a={a} et b={b} :")
    print(f" -> PGCD = {pgcd_val}")
    print(f" -> PPCM = {ppcm_val}")
    print(f" -> Vérification PGCD * PPCM = {pgcd_val * ppcm_val} (|ab| = {abs(a*b)})")

# --- Tests ---
calculer_pgcd_ppcm(60, 84)
print("-" * 20)
calculer_pgcd_ppcm(15, 25) # Retour à notre problème de synchronisation
```

3.3 Théorème de Gauss, Identité de Bézout, Algorithme d'Euclide

3.3.1 Motivation

La théorie des nombres ne se contente pas d'étudier les propriétés abstraites des entiers ; elle fournit les algorithmes qui sécurisent le monde numérique. L'algorithme d'Euclide est l'un des plus vieux algorithmes au monde, et sa version "étendue" (liée à l'identité de Bézout) permet de calculer des inverses modulaires. Ces calculs sont le cœur mathématique du chiffrement RSA, utilisé quotidiennement pour sécuriser les transactions bancaires et les communications sur Internet.

3.3.2 Approche par problème

Imaginez que vous disposiez d'un récipient de 5 litres et d'un autre de 3 litres, sans aucune graduation. Comment faire pour obtenir exactement 1 litre d'eau ?

Mathématiquement, cela revient à chercher deux entiers u et v (le nombre de fois où l'on remplit ou vide chaque récipient) tels que $5u + 3v = 1$. Cette équation, appelée équation diophantienne linéaire, possède des solutions entières uniquement grâce aux relations profondes qui lient le nombre 5, le nombre 3, et leur PGCD. Le théorème de Bézout et l'algorithme d'Euclide nous donnent la méthode systématique pour trouver ces solutions.

3.3.3 Définitions et théorèmes

L'Algorithme d'Euclide

L'algorithme d'Euclide permet de calculer le PGCD de deux entiers sans avoir à chercher tous leurs diviseurs. Il repose sur le lemme suivant : si $a = bq + r$, alors $\text{PGCD}(a, b) = \text{PGCD}(b, r)$.

Théorème 3.3.1 (Algorithme d'Euclide). *Soient a et b deux entiers naturels non nuls avec $a > b$. En effectuant des divisions euclidiennes successives :*

$$\begin{aligned}a &= bq_1 + r_1 \quad (0 < r_1 < b) \\b &= r_1q_2 + r_2 \quad (0 < r_2 < r_1) \\r_1 &= r_2q_3 + r_3 \quad (0 < r_3 < r_2) \\&\dots \\r_{n-2} &= r_{n-1}q_n + r_n \quad (0 < r_n < r_{n-1}) \\r_{n-1} &= r_nq_{n+1} + 0\end{aligned}$$

Le dernier reste non nul, r_n , est exactement le $\text{PGCD}(a, b)$.

[Image of flowchart of the Euclidean algorithm]

Identité de Bézout

Théorème 3.3.2 (Identité de Bézout). *Deux entiers relatifs a et b sont premiers entre eux si et seulement s'il existe deux entiers relatifs u et v tels que :*

$$au + bv = 1$$

Propriété 3.3.1 (Généralisation - Théorème de Bachet-Bézout). Soient a et b deux entiers non nuls. Si $d = \text{PGCD}(a, b)$, alors il existe deux entiers relatifs u et v tels que :

$$au + bv = d$$

De plus, l'équation $ax + by = c$ admet des solutions entières (x, y) si et seulement si c est un multiple de d .

Théorème de Gauss

Théorème 3.3.3 (Théorème de Gauss). Soient a , b et c trois entiers relatifs non nuls. Si a divise le produit bc et si a et b sont premiers entre eux ($\text{PGCD}(a, b) = 1$), alors a divise c .

3.3.4 Exemples de difficulté croissante

Exemple 3.3.1 (Recherche de PGCD par l'algorithme d'Euclide). Déterminer le PGCD de 420 et 135.

Correction : On applique l'algorithme d'Euclide par divisions successives :

$$— 420 = 135 \times 3 + 15 \text{ (le reste est 15)}$$

$$— 135 = 15 \times 9 + 0 \text{ (le reste est 0)}$$

Le dernier reste non nul est 15. Donc, $\text{PGCD}(420, 135) = 15$.

Exemple 3.3.2 (Résolution d'une équation avec Bézout et Gauss). Résoudre dans $\mathbb{Z}^2$ l'équation $(E) : 5x - 3y = 2$.

Correction : 1. Recherche d'une solution particulière : On remarque de manière évidente que $5 \times 1 - 3 \times 1 = 2$. Donc le couple $(1, 1)$ est une solution particulière. 2.

Soustraction : Soit (x, y) une solution. On a :

$$5x - 3y = 2$$

$$5(1) - 3(1) = 2$$

En soustrayant ligne à ligne : $5(x - 1) - 3(y - 1) = 0$, soit $5(x - 1) = 3(y - 1)$. 3.

Application du théorème de Gauss : 5 divise $3(y - 1)$. Or $\text{PGCD}(5, 3) = 1$. D'après le théorème de Gauss, 5 divise $(y - 1)$. Il existe donc un entier k tel que $y - 1 = 5k$, d'où $y = 5k + 1$. 4. **Substitution :** On remplace $y - 1$ par $5k$ dans l'égalité $5(x - 1) = 3(y - 1)$:

$$5(x - 1) = 3(5k) \implies x - 1 = 3k \implies x = 3k + 1$$

Les solutions sont donc les couples de la forme $(3k + 1, 5k + 1)$ avec $k \in \mathbb{Z}$.

3.3.5 Exercice pratique avec Python

Objectif : Implémenter l'algorithme d'Euclide étendu.

Cet algorithme ne se contente pas de trouver le PGCD, il "remonte" les calculs pour trouver les coefficients u et v de l'identité de Bézout ($au + bv = \text{PGCD}$). C'est indispensable pour calculer des clés cryptographiques.

```

def euclide_etendu(a, b):
    """
    Retourne (pgcd, u, v) tels que  $a*u + b*v = \text{pgcd}$ 
    basé sur l'algorithme d'Euclide étendu.
    """
    if b == 0:
        return a, 1, 0
    else:
        # Appel récursif
        d, u_prime, v_prime = euclide_etendu(b, a % b)

        # On reconstitue les coefficients u et v
        u = v_prime
        v = u_prime - (a // b) * v_prime

        return d, u, v

# --- Tests ---
a, b = 420, 135
pgcd, u, v = euclide_etendu(a, b)

print(f"Pour a={a} et b={b} :")
print(f"PGCD trouvé : {pgcd}")
print(f"Coefficients de Bézout : u={u}, v={v}")
print(f"Vérification : {a}*({u}) + {b}*({v}) = {a*u + b*v}")

# Test avec le problème d'introduction (récipients de 5L et 3L)
d, u, v = euclide_etendu(5, 3)
print("\nProblème des récipients (5L et 3L) :")
print(f"5*({u}) + 3*({v}) = {d}")
# Interprétation : Remplir 2 fois le récipient de 5L (10L),
# et vider 3 fois son contenu dans celui de 3L (-9L) laisse 1L.

```

Travaux Dirigés : Arithmétique dans $\mathbb{Z}$

Exercice 1 : Restitution et définitions (*Taxonomie : Connaissance / Difficulté : Facile*)

1. Énoncer avec précision le théorème de la division euclidienne dans $\mathbb{Z}$, en insistant sur la condition portant sur le reste.
2. Soit $n \in \mathbb{N}^*$. Donner la définition mathématique exacte de la proposition : « a et b sont congrus modulo n ».
3. Rappeler l'énoncé du théorème de Gauss.

Exercice 2 : Propriétés fondamentales (*Taxonomie : Compréhension / Difficulté : Facile*)

L'objectif de cet exercice est de démontrer des propriétés du cours à partir des définitions.

1. Soient $a, b, c, d \in \mathbb{Z}$ et $n \in \mathbb{N}^*$. Démontrer que si $a \equiv b \pmod{n}$ et $c \equiv d \pmod{n}$, alors $a + c \equiv b + d \pmod{n}$.
2. Démontrer que si $a \equiv b \pmod{n}$, alors pour tout entier naturel k , $a^k \equiv b^k \pmod{n}$.

Exercice 3 : PGCD et littéral (*Taxonomie : Application / Difficulté : Moyenne*)

Soit n un entier naturel. On considère les entiers $A = n$ et $B = n + 1$.

1. En utilisant la définition des diviseurs communs, démontrer que $\text{PGCD}(A, B) = 1$.
2. En déduire, en appliquant l'identité de Bézout, que pour tout $n \in \mathbb{N}$, les nombres $2n + 1$ et $n(n + 1)$ sont premiers entre eux.

Exercice 4 : Réduction de PGCD (*Taxonomie : Analyse / Difficulté : Moyenne*)

Soient a et b deux entiers relatifs non nuls. On pose $d = \text{PGCD}(a, b)$.

1. Justifier l'existence de deux entiers a' et b' tels que $a = da'$ et $b = db'$.
2. Analyser la relation entre a' et b' en démontrant rigoureusement que $\text{PGCD}(a', b') = 1$.
1. (*Indication : Utiliser le théorème de Bachet-Bézout*).

Exercice 5 : Conséquences du théorème de Gauss (*Taxonomie : Synthèse / Difficulté : Difficile*)

Soient a, b et c trois entiers relatifs non nuls. L'objectif est de démontrer une propriété de divisibilité simultanée.

1. On suppose que a divise c , que b divise c , et que $\text{PGCD}(a, b) = 1$. Construire une démonstration prouvant que le produit ab divise c . (*Indication : Commencer par écrire $c = ak = bl$, puis utiliser le théorème de Gauss*).
2. Montrer par un contre-exemple que la condition $\text{PGCD}(a, b) = 1$ est strictement nécessaire.

Exercice 6 : Évaluation de divisibilité (*Taxonomie : Évaluation / Difficulté : Difficile*)

On considère l'expression $E(n) = n(n + 1)(2n + 1)$ pour tout entier naturel n .

1. Évaluer la divisibilité de $E(n)$ par 2 en distinguant les cas de parité de n .
2. Évaluer la divisibilité de $E(n)$ par 3 en utilisant les congruences modulo 3.
3. En déduire, à l'aide du résultat de l'Exercice 5, que $E(n)$ est toujours un multiple de 6.

Exercice 7 : Somme et produit d'entiers premiers entre eux (*Taxonomie : Synthèse / Difficulté : Très Difficile*)

Soient a et b deux entiers relatifs non nuls tels que $\text{PGCD}(a, b) = 1$.

1. Démontrer que $\text{PGCD}(a + b, a) = 1$ et que $\text{PGCD}(a + b, b) = 1$.
2. En déduire rigoureusement que $\text{PGCD}(a + b, ab) = 1$.
3. Cette propriété est-elle conservée pour la somme des carrés ? Démontrer que $\text{PGCD}(a^2 + b^2, ab) = 1$.

Exercice 8 : Construction du système des restes chinois (cas $n=2$) (*Taxonomie : Création / Difficulté : Très Difficile*)

Soient p et q deux entiers naturels non nuls et premiers entre eux ($\text{PGCD}(p, q) = 1$). On s'intéresse au système de congruences suivant, où a et b sont des entiers quelconques :

$$(S) \begin{cases} x \equiv a \pmod{p} \\ x \equiv b \pmod{q} \end{cases}$$

1. Justifier l'existence de deux entiers u et v tels que $pu + qv = 1$.
2. Créer une solution particulière x_0 au système (S) en fonction de p, q, u, v, a et b . Démontrer que cette valeur x_0 satisfait bien les deux équations du système.
3. Formuler et démontrer la forme générale de toutes les solutions x du système (S) . Sur quel modulo l'unicité de la solution est-elle garantie ?

Travaux Pratiques (TP) : Implémentations Algorithmiques

L'objectif de cette séance de Travaux Pratiques est de traduire les théorèmes d'arithmétique dans l'ensemble $\mathbb{Z}$ en algorithmes fonctionnels. Les implémentations pourront être réalisées en langage Python.

Exercice 1 : La division euclidienne *from scratch*

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire une fonction `division_naïve(a, b)` qui prend deux entiers naturels a et b ($b > 0$) et qui retourne le quotient q et le reste r en utilisant uniquement des soustractions successives (sans utiliser les opérateurs `//` et `%`).
2. **[Bloom : Application | Difficulté : Moyenne]**
Adapter cette fonction pour qu'elle respecte le théorème mathématique de la division euclidienne dans $\mathbb{Z}$, c'est-à-dire en gérant correctement les cas où a et/ou b sont des entiers strictement négatifs (rappel : le reste r doit toujours vérifier $0 \leq r < |b|$).

Exercice 2 : Congruences et tests de primalité

1. **[Bloom : Application | Difficulté : Facile]**
Écrire une fonction `est_premier(n)` qui utilise l'opérateur modulo pour déterminer si un entier $n \geq 2$ est un nombre premier. L'algorithme devra s'arrêter intelligemment à $\lfloor \sqrt{n} \rfloor$ pour optimiser le calcul.
2. **[Bloom : Analyse | Difficulté : Moyenne]**
Implémenter une fonction `crible_eratosthene(N)` qui retourne la liste de tous les nombres premiers inférieurs ou égaux à N , en appliquant le principe d'élimination des multiples.

Exercice 3 : Les deux visages du PGCD

1. **[Bloom : Application | Difficulté : Facile]**
Implémenter une fonction `pgcd_soustraction(a, b)` basée sur l'algorithme original d'Euclide utilisant les soustractions successives.

2. **[Bloom : Application | Difficulté : Moyenne]**
Implémenter une fonction `pgcd_modulo(a, b)` utilisant les divisions euclidiennes successives (opérateur %).
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Écrire un script qui compare le temps d'exécution de ces deux fonctions pour $a = 123456789$ et $b = 123$. Conclure sur la complexité algorithmique.

Exercice 4 : Calcul du PPCM optimisé

1. **[Bloom : Compréhension | Difficulté : Facile]**
Déduire du théorème vu en cours une fonction `ppcm(a, b)` qui calcule le Plus Petit Commun Multiple de deux entiers en faisant appel à la fonction `pgcd_modulo(a, b)`.
2. **[Bloom : Synthèse | Difficulté : Moyenne]**
Écrire une fonction `ppcm_liste(L)` qui prend en paramètre une liste de n entiers relatifs et qui retourne le PPCM global de tous les éléments de cette liste.

Exercice 5 : L'Algorithme d'Euclide Étendu

1. **[Bloom : Analyse | Difficulté : Difficile]**
Implémenter la fonction `euclide_etendu(a, b)` qui retourne un triplet (d, u, v) tel que $d = \text{PGCD}(a, b)$ et $au + bv = d$. Vous pouvez utiliser une approche récursive ou itérative au choix.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester la fonction avec $a = 240$ et $b = 46$, puis afficher la chaîne de caractères vérifiant l'équation de Bézout correspondante.

Exercice 6 : Résolution algorithmique d'équations diophantiennes

1. **[Bloom : Synthèse | Difficulté : Difficile]**
Écrire une fonction `resoudre_diophantienne(a, b, c)` qui cherche à résoudre l'équation $ax + by = c$. La fonction devra vérifier si une solution existe grâce au théorème de Bachet-Bézout, et si oui, utiliser la fonction `euclide_etendu` pour retourner une solution particulière (x_0, y_0) .
2. **[Bloom : Création | Difficulté : Difficile]**
Enrichir la fonction pour qu'elle prenne en compte deux bornes $X_{\min}$ et $X_{\max}$, et qu'elle retourne la liste de toutes les solutions (x, y) telles que $X_{\min} \leq x \leq X_{\max}$.

Exercice 7 : Application Cryptographique (Chiffrement Affine)

1. **[Bloom : Application | Difficulté : Moyenne]**
Le chiffrement affine transforme une lettre en lui associant un rang x (de 0 pour A à 25 pour Z), puis calcule un nouveau rang $y \equiv ax + b \pmod{26}$. Écrire une fonction `chiffrement_affine(texte, a, b)` qui retourne le texte chiffré. Que doit vérifier la clé a pour que le déchiffrement soit possible?
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Pour déchiffrer, il faut trouver l'inverse modulaire de a modulo 26, c'est-à-dire un entier a' tel que $a \times a' \equiv 1 \pmod{26}$. Utiliser l'algorithme d'Euclide étendu pour écrire une fonction `inverse_modulaire(a, n)` et en déduire la fonction `dechiffrement_affine(texte_chiffre, a, b)`.

Exercice 8 : Introduction à RSA (Génération de clés)

1. [Bloom : Création | Difficulté : Très Difficile]

L'algorithme de cryptographie asymétrique RSA repose fortement sur l'arithmétique dans $\mathbb{Z}$. Écrire une fonction globale `generer_cles_rsa(p, q)` où p et q sont deux nombres premiers distincts donnés en paramètre. L'algorithme devra :

- Calculer le module $n = p \times q$.
- Calculer l'indicatrice d'Euler $\phi(n) = (p - 1)(q - 1)$.
- Choisir un entier e tel que $1 < e < \phi(n)$ et $\text{PGCD}(e, \phi(n)) = 1$.
- Calculer d , l'inverse modulaire de e modulo $\phi(n)$ (à l'aide de l'exercice 7).
- Retourner la clé publique (n, e) et la clé privée (n, d) .

Aller plus loin : L'arithmétique au cœur de la Data Science

L'arithmétique dans $\mathbb{Z}$, bien qu'étudiée depuis l'Antiquité, est aujourd'hui l'un des piliers de l'ingénierie des données (Data Science) et de l'intelligence artificielle. Au-delà de la cryptographie (RSA) que nous avons évoquée, les concepts de division euclidienne et de congruence sont indispensables pour optimiser la mémoire et les temps de calcul dans les algorithmes d'apprentissage automatique (Machine Learning).

Le défi des données textuelles en NLP

Dans le domaine du traitement du langage naturel (NLP), les algorithmes ne comprennent pas les mots : ils ont besoin de nombres. Pour entraîner un modèle de classification de textes ou analyser des séquences de mots (ce qu'on appelle des *N-grammes*), on transforme généralement le vocabulaire en un immense tableau (un dictionnaire).

Cependant, sur un corpus de texte réel (comme l'ensemble des articles Wikipédia), le nombre de mots uniques et de combinaisons de *N-grammes* peut atteindre des dizaines de millions. Créer une matrice où chaque colonne représente un *N-gramme* spécifique demande une quantité de mémoire RAM gigantesque, souvent indisponible sur des machines standards. Comment alors représenter ces données spatialement sans saturer la mémoire ?

La solution : Le « Feature Hashing » (L'astuce du hachage)

C'est ici qu'intervient l'arithmétique modulaire à travers une technique appelée le **Feature Hashing** (ou Hashing Trick). Plutôt que de maintenir un dictionnaire exact, on utilise une fonction mathématique (une fonction de hachage) qui transforme chaque chaîne de caractères en un grand nombre entier abstrait.

Ensuite, pour forcer ces grands nombres à rentrer dans un espace mémoire de taille fixe et contrôlée (par exemple M colonnes), on applique une simple congruence modulo M :

$$\text{Index du vecteur} = \text{Fonction_Hachage}(\text{mot}) \pmod{M}$$

Exemple conceptuel : Supposons que nous fixions la taille de notre mémoire à $M = 1000$ dimensions. Si la fonction de hachage convertit le bi-gramme "*intelligence artificielle*" en l'entier 4598273, sa position dans notre vecteur mémoire sera simplement le reste de sa division euclidienne par 1000 :

$$4598273 \equiv 273 \pmod{1000}$$

Le modèle stockera l'information de ce bi-gramme à l'index 273.

Gestion des collisions et propriétés statistiques

Que se passe-t-il si un autre mot possède également un reste de 273 (ce qu'on appelle une collision) ? Mathématiquement, le modèle va additionner les fréquences de ces deux mots au même endroit.

Bien que cela semble être une perte d'information (deux mots différents mélangés), les mathématiques statistiques démontrent que si M est suffisamment grand (et souvent choisi comme un nombre premier pour de meilleures propriétés de répartition arithmétique), l'impact de ces collisions sur les performances du modèle de Machine Learning est négligeable.

Ainsi, grâce à la théorie des congruences, des modèles très complexes peuvent s'entraîner sur des flux de données continus et infinis avec une empreinte mémoire fixe et strictement maîtrisée en temps constant $\mathcal{O}(1)$.

*Pour aller plus loin sur ce sujet, vous pouvez vous documenter sur l'algorithme **MurmurHash**, très utilisé dans les bibliothèques de Machine Learning comme Scikit-Learn (via **FeatureHasher**), ou sur la manière dont les architectures de plongement lexical (Word Embeddings) comme **GloVe** gèrent les matrices de co-occurrence massives.*

Projet d'Application : Le « Hashing Trick » en Analyse de Données

Contexte du Projet

Imaginez que vous avez mis en place un système de *web scraping* pour récupérer en direct les commentaires textuels de milliers de matchs de football (ex : « Tir cadré de l'attaquant », « Faute dangereuse au milieu du terrain », etc.). Votre objectif est d'entraîner un modèle de Machine Learning capable d'analyser ces flux de textes pour calculer des statistiques en temps réel ou prédire l'issue d'une action.

Le problème : le vocabulaire total généré par ces milliers de matchs est gigantesque. Construire un dictionnaire classique en mémoire vive (RAM) pour associer chaque mot ou N-gramme à un index unique va saturer vos serveurs.

Vous allez donc utiliser l'arithmétique dans $\mathbb{Z}$, et plus précisément la division euclidienne, pour implémenter de zéro la technique du **Feature Hashing** étudiée dans la section précédente.

Objectifs

1. Implémenter une vectorisation de texte classique (avec dictionnaire) pour en montrer les limites.
2. Implémenter une fonction de *Feature Hashing* basée sur l'opération modulo.
3. Analyser mathématiquement le taux de collision en fonction de la taille de l'espace mémoire M .

Cahier des charges et Travail demandé

Étape 1 : Simulation des données textuelles

Pour simuler notre flux de données, créez un script Python qui génère une liste de phrases ou récupérez un corpus de texte volumineux (par exemple, un fichier texte contenant un grand nombre de commentaires sportifs ou d'articles). Développez une fonction `extraire_mots(texte)` qui nettoie le texte (minuscules, ponctuation) et retourne une liste de mots.

Étape 2 : L'approche classique (Dictionnaire exact)

Écrivez une fonction `vectoriseur_classique(corpus)` qui :

- Parcourt tous les mots du corpus.
- Assigne un identifiant unique (un entier de 0 à $V - 1$, où V est la taille du vocabulaire) à chaque nouveau mot rencontré en utilisant un dictionnaire Python (`dict`).
- **Question :** Quelle est la complexité spatiale de cette approche si le flux de mots est infini ?

Étape 3 : L'approche arithmétique (Feature Hashing)

Ici, nous n'utilisons plus de dictionnaire pour mémoriser les mots. Nous fixons une taille de mémoire maximale, par exemple $M = 1024$ colonnes.

Écrivez une fonction `vectoriseur_hachage(corpus, M)` qui :

- Utilise une fonction de hachage standard pour convertir chaque mot en un entier abstrait. Vous pouvez utiliser la fonction native `hash()` de Python ou importer `hashlib`.
- Calcule l'index du mot dans le vecteur mémoire en utilisant la congruence :

$$\text{Index} \equiv \text{hash}(\text{mot}) \pmod{M}$$

- Construit un vecteur de taille M comptant la fréquence d'apparition des index.

Étape 4 : Analyse arithmétique des collisions

Une collision survient lorsque deux mots différents m_1 et m_2 vérifient :

$$\text{hash}(m_1) \equiv \text{hash}(m_2) \pmod{M}$$

1. Modifiez votre fonction de hachage pour qu'elle compte le nombre exact de mots différents qui "tombent" sur le même index.
2. Testez votre algorithme avec $M = 100$, $M = 1009$ (un nombre premier) et $M = 10000$.
3. Tracez un graphique (avec `matplotlib`) représentant le nombre de collisions en fonction de la taille M .
4. **Analyse :** Pourquoi recommande-t-on souvent d'utiliser un nombre premier pour la dimension M en Data Science ? (Faites le lien avec les diviseurs communs vus dans le Chapitre III).

Bonus (Pour aller très loin)

Remplacez les mots simples par des bi-grammes (paires de mots adjacents). Comparez l'explosion de la mémoire RAM avec l'approche classique par rapport à la stabilité absolue de la mémoire avec votre approche par modulo.

Chapitre 4

Structures algébriques

Ce chapitre introduit les structures fondamentales de l'algèbre : groupes, anneaux et corps. Ces structures unifient sous un même formalisme l'arithmétique entière, le calcul matriciel, les permutations, l'arithmétique modulaire de la cryptographie, ainsi que de nombreuses transformations en informatique graphique et en science des données. La force de cette abstraction est qu'un théorème démontré sur une structure s'applique automatiquement à toutes ses incarnations concrètes.

4.1 Structure de groupe

4.1.1 Motivation

Un *groupe* est l'abstraction la plus simple qui capture la notion de symétrie réversible. Que vous mélangiez un paquet de cartes, fassiez tourner un cube de Rubik, chiffriez un message par un protocole asymétrique (Diffie-Hellman, RSA), ou appliquiez une transformation géométrique en infographie, vous manipulez sans le savoir un groupe. Cette unification est précieuse : un théorème démontré une fois sur les groupes (par exemple « tout sous-groupe d'un groupe cyclique est cyclique ») s'applique automatiquement à des dizaines de situations très différentes.

4.1.2 Approche par problème : l'horloge et le chiffrement de César

Problème : Dans le chiffrement de César de pas k , on remplace chaque lettre par celle qui se trouve k positions plus loin dans l'alphabet (modulo 26).

1. Si je chiffre avec $k_1 = 5$ puis $k_2 = 9$, par quel décalage déchiffrer ?
2. Si je chiffre par $k = 13$, que se passe-t-il en chiffrant deux fois ?
3. Quelles propriétés possède l'opération « composer deux décalages » sur $\{0, 1, \dots, 25\}$?

Résolution : La composition correspond à l'addition modulo 26. Donc $k_1 + k_2 = 14$: pour déchiffrer il faut décaler de $-14 \equiv 12 \pmod{26}$. Pour $k = 13$: $13 + 13 = 26 \equiv 0$. Chiffrer deux fois revient à l'identité, c'est ROT13. La loi $+$ (mod 26) est associative, possède un neutre (0) et tout élément k a un inverse $26 - k$. C'est exactement la structure de *groupe* sur l'ensemble $\mathbb{Z}/26\mathbb{Z}$.

4.1.3 Loi de composition interne et définition du groupe

Définition 4.1.1 (Loi de composition interne). Une **loi de composition interne** (l.c.i) sur un ensemble non vide G est une application $\star : G \times G \rightarrow G$ qui à un couple (a, b) associe l'élément noté $a \star b$.

Définition 4.1.2 (Groupe). Un couple $(G, \star)$ est un **groupe** si $\star$ est une l.c.i sur G vérifiant :

1. (**Associativité**) $\forall a, b, c \in G, (a \star b) \star c = a \star (b \star c)$;
2. (**Élément neutre**) $\exists e \in G, \forall a \in G, a \star e = e \star a = a$;
3. (**Inverse**) $\forall a \in G, \exists a' \in G, a \star a' = a' \star a = e$.

Si de plus $\forall a, b \in G, a \star b = b \star a$, le groupe est dit **abélien** (ou commutatif).

Proposition 4.1.1 (Unicité du neutre et de l'inverse). *Dans tout groupe $(G, \star)$:*

- l'élément neutre e est unique ;
- pour tout $a \in G$, l'inverse a' est unique. On le note a^{-1} (notation multiplicative) ou $-a$ (notation additive).

Démonstration. Si e et e' sont neutres : $e = e \star e' = e'$. Si a' et a'' sont inverses de a : $a' = a' \star e = a' \star (a \star a'') = (a' \star a) \star a'' = e \star a'' = a''$. $\square$

Exemple 4.1.1 (Groupes usuels). — $(\mathbb{Z}, +), (\mathbb{Q}, +), (\mathbb{R}, +), (\mathbb{C}, +)$ sont des groupes abéliens.

- $(\mathbb{Q}^*, \times), (\mathbb{R}^*, \times), (\mathbb{C}^*, \times)$ sont des groupes abéliens.
- $(\mathbb{N}, +)$ n'est pas un groupe : pas d'inverse pour $n \geq 1$.
- L'ensemble S_n des permutations de $\{1, \dots, n\}$ muni de la composition est un groupe, non abélien dès $n \geq 3$.
- $GL_n(\mathbb{R})$, l'ensemble des matrices réelles $n \times n$ inversibles muni du produit, est un groupe (non abélien dès $n \geq 2$).

4.1.4 Sous-groupes

Définition 4.1.3 (Sous-groupe). Soit $(G, \star)$ un groupe d'élément neutre e . Une partie $H \subseteq G$ est un **sous-groupe** de G , noté $H \leq G$, si :

1. $e \in H$;
2. $\forall a, b \in H, a \star b \in H$ (stabilité par la loi) ;
3. $\forall a \in H, a^{-1} \in H$ (stabilité par inverse).

Proposition 4.1.2 (Caractérisation pratique d'un sous-groupe). $H \subseteq G$ est un sous-groupe de G si et seulement si :

$$H \neq \emptyset \quad \text{et} \quad \forall a, b \in H, a \star b^{-1} \in H.$$

Démonstration. $(\Rightarrow)$ immédiat. $(\Leftarrow)$ H non vide contient un élément a , donc $a \star a^{-1} = e \in H$. Puis $\forall b \in H, e \star b^{-1} = b^{-1} \in H$ donc H est stable par inverse. Enfin $\forall a, b \in H, a \star (b^{-1})^{-1} = a \star b \in H$ donc H est stable par la loi. $\square$

Remarque 4.1.1. Tout sous-groupe contient le neutre et est lui-même un groupe pour la loi induite. Pour tout groupe G , $\{e\}$ et G sont les sous-groupes *triviaux*.

4.1.5 Les sous-groupes de $(\mathbb{Z}, +)$

Les sous-groupes du groupe additif $\mathbb{Z}$ se décrivent intégralement.

Théorème 4.1.3 (Sous-groupes de $\mathbb{Z}$). *Soit H un sous-groupe de $(\mathbb{Z}, +)$. Il existe un unique entier $n \in \mathbb{N}$ tel que*

$$H = n\mathbb{Z} = \{nk : k \in \mathbb{Z}\}.$$

Démonstration. Si $H = \{0\}$, on prend $n = 0$. Sinon H contient un entier non nul, donc un entier strictement positif (en prenant son opposé si besoin). Posons $n = \min(H \cap \mathbb{N}^*)$.

Inclusion $n\mathbb{Z} \subseteq H$: par stabilité par addition et passage à l'opposé, $n \in H$ entraîne $kn \in H$ pour tout $k \in \mathbb{Z}$.

Inclusion $H \subseteq n\mathbb{Z}$: soit $h \in H$. La division euclidienne donne $h = nq + r$ avec $0 \leq r < n$. Comme $h \in H$ et $nq \in H$, on a $r = h - nq \in H$. Mais $r < n$ et n est le plus petit élément strictement positif de H , donc $r = 0$. Ainsi $h = nq \in n\mathbb{Z}$.

L'unicité est claire : $n\mathbb{Z} = m\mathbb{Z}$ avec $n, m \geq 0$ implique $n = m$ (plus petit élément > 0 commun). $\square$

Remarque 4.1.2. Ce résultat est central. Il dit que tous les sous-groupes de $\mathbb{Z}$ sont *cycliques* (engendrés par un seul élément, voir ci-dessous). On retrouvera ce phénomène dans tout groupe cyclique.

4.1.6 Sous-groupe engendré, groupes monogènes et cycliques

Définition 4.1.4 (Sous-groupe engendré par un élément). Soit $(G, \star)$ un groupe et $a \in G$. On note $\langle a \rangle$ le **sous-groupe engendré par a** :

$$\langle a \rangle = \{a^n : n \in \mathbb{Z}\}$$

en notation multiplicative, où $a^0 = e$, $a^n = \underbrace{a \star \cdots \star a}_{n \text{ fois}}$ et $a^{-n} = (a^{-1})^n$ pour $n \geq 1$. En notation additive, $\langle a \rangle = \{na : n \in \mathbb{Z}\}$. C'est le plus petit sous-groupe de G contenant a .

Définition 4.1.5 (Groupe monogène, groupe cyclique). Un groupe G est **monogène** s'il existe $a \in G$ tel que $G = \langle a \rangle$. Un tel a est appelé *générateur*. Si de plus G est fini, on dit que G est **cyclique**.

Exemple 4.1.2 (Premiers exemples). — $(\mathbb{Z}, +)$ est monogène, engendré par 1 (ou par -1). Il est infini, donc non cyclique.

— $(\mathbb{Z}/n\mathbb{Z}, +)$ est cyclique d'ordre n , engendré par $\bar{1}$. Plus généralement, $\bar{k}$ est générateur si et seulement si $\text{pgcd}(k, n) = 1$.

— Le groupe S_3 a 6 éléments mais aucun n'engendre les six : il n'est pas cyclique.

4.1.7 Ordre d'un élément

Définition 4.1.6 (Ordre d'un élément). Soit $(G, \star)$ un groupe (noté multiplicativement) et $a \in G$. L'**ordre** de a , noté $o(a)$, est :

- $o(a) = +\infty$ si $\langle a \rangle$ est infini ;
- $o(a) =$ le plus petit entier $n \geq 1$ tel que $a^n = e$, sinon.

Lorsque $o(a)$ est fini, on a $|\langle a \rangle| = o(a)$.

Proposition 4.1.4 (Caractérisation de l'ordre). Soit $a \in G$ d'ordre fini n . Pour tout $k \in \mathbb{Z}$:

$$a^k = e \iff n \mid k.$$

Démonstration. Division euclidienne $k = nq + r$ avec $0 \leq r < n$: $a^k = (a^n)^q \star a^r = e^q \star a^r = a^r$. Donc $a^k = e \iff a^r = e$. Comme $0 \leq r < n$ et n est le plus petit entier ≥ 1 vérifiant $a^n = e$, ceci équivaut à $r = 0$, soit $n \mid k$. $\square$

Exemple 4.1.3 (Ordres dans $(\mathbb{Z}/12\mathbb{Z}, +)$). On cherche le plus petit $k \geq 1$ tel que $k \cdot a \equiv 0 \pmod{12}$, ce qui donne $k = 12/\text{pgcd}(a, 12)$.

a	0	1	2	3	4	5	6	7	8	9	10	11
$o(a)$	1	12	6	4	3	12	2	12	3	4	6	12

On observe que $o(a) \mid 12$ pour tout a : c'est un cas particulier du théorème de Lagrange (à venir au §1.b).

4.1.8 Exemples résolus (Difficulté ascendante)

Exemple 1 (Vérification d'un groupe — Facile). Montrer que $(\mathbb{R}^*, \times)$ est un groupe abélien.

Correction : la multiplication réelle est interne sur $\mathbb{R}^*$ (un produit de réels non nuls est non nul), associative et commutative. L'élément 1 est neutre. Tout $x \in \mathbb{R}^*$ admet l'inverse $1/x \in \mathbb{R}^*$.

Exemple 2 (Sous-groupe par caractérisation — Intermédiaire). Montrer que $H = \{2^n : n \in \mathbb{Z}\}$ est un sous-groupe de $(\mathbb{R}_+^*, \times)$.

Correction : $H \neq \emptyset$ (il contient $1 = 2^0$). Pour $a = 2^n$ et $b = 2^m$ dans H , on a $a \times b^{-1} = 2^n \times 2^{-m} = 2^{n-m} \in H$ puisque $n - m \in \mathbb{Z}$. La caractérisation pratique conclut : $H \leq \mathbb{R}_+^*$.

Exemple 3 (Calcul d'ordre dans un groupe modulaire — Difficile). Calculer l'ordre de $\bar{3}$ dans $(\mathbb{Z}/14\mathbb{Z})^*, \times$.

Correction : on calcule les puissances modulo 14 : $3^1 = 3$, $3^2 = 9$, $3^3 = 27 \equiv 13 \equiv -1$, $3^4 \equiv -3 \equiv 11$, $3^5 \equiv 33 \equiv 5$, $3^6 \equiv 15 \equiv 1 \pmod{14}$. Donc $o(\bar{3}) = 6$. Le sous-groupe engendré est $\langle \bar{3} \rangle = \{1, 3, 9, 13, 11, 5\}$, qui contient les six éléments inversibles modulo 14. Donc $\bar{3}$ est un générateur de $(\mathbb{Z}/14\mathbb{Z})^*$ (qui est cyclique d'ordre $\varphi(14) = 6$).

4.1.9 Exercices pratiques avec Python

On peut tester sur machine la structure de groupe de $\mathbb{Z}/n\mathbb{Z}$: calculer l'ordre d'un élément, énumérer le sous-groupe engendré, lister tous les sous-groupes.

```

1 from math import gcd
2
3 def ordre_additif(a, n):
4     """Ordre de a dans (Z/nZ, +) : plus petit k>=1 tel que k*a == 0
5         mod n."""
6     return n // gcd(a, n)
7
8 def sous_groupe_engendre_additif(a, n):
9     """Sous-groupe engendre par a dans (Z/nZ, +)."""

```



```

9      H = set()
10     x = 0
11     while x not in H:
12         H.add(x)
13         x = (x + a) % n
14     return sorted(H)
15
16 def tous_les_sous_groupes_de_ZnZ(n):
17     """Les sous-groupes de  $\mathbb{Z}/n\mathbb{Z}$  sont en bijection avec les
18        diviseurs de  $n$ ."""
19     return {d: sous_groupe_engendre_additif(d, n) for d in range(1,
20         n+1) if n % d == 0}
21
22 # Test sur  $\mathbb{Z}/12\mathbb{Z}$ 
23 print("Ordres dans  $\mathbb{Z}/12\mathbb{Z}$ :")
24 for a in range(12):
25     print(f"o({a}) = {ordre_additif(a, 12) if a != 0 else 1}")
26
27 print("\nSous-groupes de  $\mathbb{Z}/12\mathbb{Z}$  (un par diviseur de 12):")
28 for d, H in tous_les_sous_groupes_de_ZnZ(12).items():
29     print(f"<{d}> = {H} (|H| = {len(H)})")

```

4.1.10 Exercices à faire

- Exercice 1.** Soit $G = \{1, -1, i, -i\} \subset \mathbb{C}^*$. Montrer que $(G, \times)$ est un groupe cyclique. Donner ses générateurs et tracer sa table.
- Exercice 2.** Soit $(G, \star)$ un groupe et H_1, H_2 deux sous-groupes de G . Montrer que $H_1 \cap H_2$ est un sous-groupe. La réunion $H_1 \cup H_2$ est-elle toujours un sous-groupe? (Donner un contre-exemple.)
- Exercice 3.** Lister tous les sous-groupes de $(\mathbb{Z}/18\mathbb{Z}, +)$ en justifiant. Combien y en a-t-il?
- Exercice 4.** Calculer l'ordre de $\bar{2}$, $\bar{3}$ et $\bar{5}$ dans $((\mathbb{Z}/11\mathbb{Z})^*, \times)$. Lequel est un générateur?
- Exercice 5 (synthèse).** Montrer que dans tout groupe G , si $a \in G$ vérifie $a^2 = e$, alors $\langle a \rangle = \{e, a\}$ et $o(a) \in \{1, 2\}$.

4.1.11 Théorème de Lagrange

Définition 4.1.7 (Classes à gauche modulo un sous-groupe). Soit $H \leq G$ et $a \in G$. La **classe à gauche** de a modulo H est l'ensemble

$$aH = \{a \star h : h \in H\}.$$

Proposition 4.1.5 (Partition par les classes). *La relation $\mathcal{R}_H$ définie par $a\mathcal{R}_H b \iff a^{-1}b \in H$ est une relation d'équivalence sur G , dont les classes sont précisément les ensembles aH . Toutes les classes ont le même cardinal $|H|$ (la translation par a est une bijection $H \rightarrow aH$).*

Démonstration. Réflexivité : $a^{-1}a = e \in H$. Symétrie : si $a^{-1}b \in H$ alors $(a^{-1}b)^{-1} = b^{-1}a \in H$. Transitivité : si $a^{-1}b \in H$ et $b^{-1}c \in H$, leur produit $a^{-1}c \in H$. La classe d'équivalence de a vaut $\{b \in G : a^{-1}b \in H\} = aH$. La translation à gauche $h \mapsto ah$ est bijective $H \rightarrow aH$. $\square$

Théorème 4.1.6 (Lagrange). *Soit G un groupe fini et H un sous-groupe de G . Alors $|H|$ divise $|G|$, et plus précisément*

$$|G| = [G : H] \cdot |H|,$$

où $[G : H]$, l'indice de H dans G , est le nombre de classes à gauche modulo H .

Démonstration. Les classes à gauche forment une partition de G et toutes ont pour cardinal $|H|$ (proposition ci-dessus). En notant $[G : H]$ leur nombre, on a $|G| = [G : H] \cdot |H|$. $\square$

Corollaire 4.1.7 (Ordre d'un élément). *Pour tout a dans un groupe fini G , $o(a)$ divise $|G|$. En particulier $a^{|G|} = e$.*

Démonstration. $o(a) = |\langle a \rangle|$ et $\langle a \rangle \leq G$. Lagrange donne $o(a) \mid |G|$. Posons $|G| = o(a) \cdot k$. Alors $a^{|G|} = (a^{o(a)})^k = e^k = e$. $\square$

Remarque 4.1.3 (Théorème d'Euler-Fermat). Appliqué au groupe multiplicatif $(\mathbb{Z}/n\mathbb{Z})^*$ d'ordre $\varphi(n)$, le corollaire donne le **théorème d'Euler** : pour tout entier a premier avec n , $a^{\varphi(n)} \equiv 1 \pmod{n}$. Lorsque $n = p$ premier, $\varphi(p) = p - 1$ et l'on retrouve le **petit théorème de Fermat** : $a^{p-1} \equiv 1 \pmod{p}$.

4.1.12 Morphismes de groupes

Définition 4.1.8 (Morphisme de groupes). Soient $(G, \star)$ et $(G', \cdot)$ deux groupes. Une application $f : G \rightarrow G'$ est un **morphisme de groupes** (ou *homomorphisme*) si

$$\forall a, b \in G, \quad f(a \star b) = f(a) \cdot f(b).$$

On dit que f est :

- un **isomorphisme** si f est bijectif (on note alors $G \simeq G'$) ;
- un **endomorphisme** si $G = G'$;
- un **automorphisme** si c'est à la fois un endomorphisme et un isomorphisme.

Proposition 4.1.8 (Propriétés élémentaires). *Soit $f : G \rightarrow G'$ un morphisme de groupes. Alors :*

1. $f(e_G) = e_{G'}$;
2. $\forall a \in G, f(a^{-1}) = f(a)^{-1}$;
3. $\forall a \in G, \forall n \in \mathbb{Z}, f(a^n) = f(a)^n$;
4. l'image directe d'un sous-groupe est un sous-groupe ; l'image réciproque d'un sous-groupe est un sous-groupe.

Démonstration. (1) $f(e_G) = f(e_G \star e_G) = f(e_G) \cdot f(e_G)$, donc en multipliant par $f(e_G)^{-1}$ on obtient $e_{G'} = f(e_G)$. (2) $f(a) \cdot f(a^{-1}) = f(a \star a^{-1}) = f(e_G) = e_{G'}$, donc $f(a^{-1}) = f(a)^{-1}$. (3) par récurrence à partir de (1) et (2). (4) vérification directe. $\square$

Définition 4.1.9 (Noyau et image). Soit $f : G \rightarrow G'$ un morphisme. Le **noyau** et l'**image** de f sont

$$\ker f = f^{-1}(\{e_{G'}\}) = \{a \in G : f(a) = e_{G'}\}, \quad \text{Im } f = f(G).$$

Ce sont respectivement des sous-groupes de G et de G' .

Proposition 4.1.9 (Caractérisation de l'injectivité). *Un morphisme $f : G \rightarrow G'$ est injectif si et seulement si $\ker f = \{e_G\}$.*

Démonstration. $(\Rightarrow)$ Si f est injectif et $f(a) = e_{G'} = f(e_G)$, alors $a = e_G$. $(\Leftarrow)$ Si $f(a) = f(b)$, alors $f(a \star b^{-1}) = f(a) \cdot f(b)^{-1} = e_{G'}$, donc $a \star b^{-1} \in \ker f = \{e_G\}$, soit $a = b$. $\square$

Exemple 4.1.4 (Exemples classiques de morphismes). — $\exp : (\mathbb{R}, +) \rightarrow (\mathbb{R}_+^*, \times)$ est un isomorphisme de réciproque $\ln$.

- $\det : (GL_n(\mathbb{R}), \times) \rightarrow (\mathbb{R}^*, \times)$ est un morphisme surjectif, de noyau $SL_n(\mathbb{R})$ (matrices de déterminant 1).
- $\pi : (\mathbb{Z}, +) \rightarrow (\mathbb{Z}/n\mathbb{Z}, +)$, $k \mapsto \bar{k}$ est un morphisme surjectif, de noyau $n\mathbb{Z}$.
- $\varepsilon : S_n \rightarrow (\{-1, 1\}, \times)$, la signature, est un morphisme surjectif (pour $n \geq 2$), de noyau le *groupe alterné* A_n .

4.1.13 Le groupe symétrique S_n

Définition 4.1.10 (Permutations). Soit $n \in \mathbb{N}^*$. Le **groupe symétrique** S_n est l'ensemble des bijections de $\{1, 2, \dots, n\}$ sur lui-même, muni de la composition. C'est un groupe d'ordre $|S_n| = n!$, non abélien dès que $n \geq 3$.

Notation à deux lignes. On représente $\sigma \in S_n$ par

$$\sigma = \begin{pmatrix} 1 & 2 & \cdots & n \\ \sigma(1) & \sigma(2) & \cdots & \sigma(n) \end{pmatrix}.$$

Définition 4.1.11 (Cycle, transposition). Un **cycle** de longueur ℓ (ou ℓ -cycle) est une permutation de la forme

$$(a_1 \ a_2 \ \dots \ a_\ell)$$

qui envoie $a_1 \mapsto a_2$, $a_2 \mapsto a_3$, $\dots$, $a_\ell \mapsto a_1$ et fixe les autres éléments. Un **2-cycle** $(a \ b)$ est appelé une **transposition**.

Théorème 4.1.10 (Décomposition en cycles disjoints). *Toute permutation $\sigma \in S_n$ se décompose de manière unique (à l'ordre près) en produit de cycles à supports deux à deux disjoints. Ces cycles commutent entre eux.*

Théorème 4.1.11 (Engendrement par les transpositions). *Tout cycle est produit de transpositions :*

$$(a_1 \ a_2 \ \dots \ a_\ell) = (a_1 \ a_\ell)(a_1 \ a_{\ell-1}) \cdots (a_1 \ a_2).$$

En conséquence, S_n est engendré par les transpositions.

Définition 4.1.12 (Signature). La **signature** d'une permutation σ , notée $\varepsilon(\sigma)$, vaut $+1$ si σ s'écrit comme produit d'un nombre pair de transpositions, et -1 sinon. Cette définition est cohérente : la parité du nombre de transpositions ne dépend pas de la décomposition.

Proposition 4.1.12 (Signature et cycles). *La signature est un morphisme $\varepsilon : S_n \rightarrow \{-1, 1\}$. La signature d'un ℓ -cycle vaut $(-1)^{\ell-1}$. Pour une permutation décomposée en cycles disjoints de longueurs $\ell_1, \dots, \ell_r$:*

$$\varepsilon(\sigma) = (-1)^{(\ell_1-1)+\dots+(\ell_r-1)} = (-1)^{n-r}.$$

Définition 4.1.13 (Groupe alterné). Le **groupe alterné** $A_n = \ker \varepsilon$ est le sous-groupe de S_n formé des permutations paires. On a $|A_n| = n!/2$ pour $n \geq 2$.

Exemple 4.1.5 (S_3). S_3 a 6 éléments :

$$S_3 = \{\text{id}, (1\ 2), (1\ 3), (2\ 3), (1\ 2\ 3), (1\ 3\ 2)\}.$$

Les trois transpositions sont impaires (signature -1). Les deux 3-cycles sont pairs (signature $+1$). $A_3 = \{\text{id}, (1\ 2\ 3), (1\ 3\ 2)\}$, cyclique d'ordre 3.

4.1.14 Le groupe $\mathbb{Z}/n\mathbb{Z}$

Définition 4.1.14 (Classes de congruence). Soit $n \in \mathbb{N}^*$. La relation de congruence modulo n , $a \equiv b \pmod{n} \iff n \mid (a-b)$, est une relation d'équivalence sur $\mathbb{Z}$. La classe de a est

$$\bar{a} = a + n\mathbb{Z} = \{a + nk : k \in \mathbb{Z}\}.$$

L'ensemble quotient $\mathbb{Z}/n\mathbb{Z} = \{\bar{0}, \bar{1}, \dots, \overline{n-1}\}$ a exactement n éléments.

Proposition 4.1.13 (Lois sur $\mathbb{Z}/n\mathbb{Z}$). *On définit l'addition et la multiplication sur $\mathbb{Z}/n\mathbb{Z}$ par*

$$\bar{a} + \bar{b} = \overline{a+b}, \quad \bar{a} \cdot \bar{b} = \overline{a \cdot b}.$$

Ces lois sont bien définies (indépendantes du choix des représentants), et $(\mathbb{Z}/n\mathbb{Z}, +)$ est un groupe abélien d'ordre n .

Théorème 4.1.14 ($\mathbb{Z}/n\mathbb{Z}$ est cyclique). *$(\mathbb{Z}/n\mathbb{Z}, +)$ est cyclique, engendré par $\bar{1}$. Plus précisément :*

$$\bar{a} \text{ est générateur de } \mathbb{Z}/n\mathbb{Z} \iff \text{pgcd}(a, n) = 1.$$

Ainsi $\mathbb{Z}/n\mathbb{Z}$ admet exactement $\varphi(n)$ générateurs.

Démonstration. $\langle \bar{a} \rangle = \{k\bar{a} : k \in \mathbb{Z}\}$ a pour cardinal $n/\text{pgcd}(a, n)$. C'est le groupe entier ssi ce cardinal vaut n , soit $\text{pgcd}(a, n) = 1$. $\square$

Théorème 4.1.15 (Groupe des inversibles). *L'ensemble $(\mathbb{Z}/n\mathbb{Z})^*$ des éléments inversibles pour la multiplication est*

$$(\mathbb{Z}/n\mathbb{Z})^* = \{\bar{a} : \text{pgcd}(a, n) = 1\}.$$

Muni de la multiplication, c'est un groupe abélien d'ordre $\varphi(n)$ (indicateur d'Euler).

Remarque 4.1.4 (Classification des groupes cycliques). Tout groupe cyclique d'ordre n est isomorphe à $(\mathbb{Z}/n\mathbb{Z}, +)$. En effet, si $G = \langle a \rangle$ avec $|G| = n$, l'application $\mathbb{Z}/n\mathbb{Z} \rightarrow G$, $\bar{k} \mapsto a^k$ est un isomorphisme bien défini.

4.1.15 Exemples résolus pour le §1.b (Difficulté ascendante)

Exemple 1 (Lagrange — Facile). Un groupe G est d'ordre 15. Quels sont les ordres possibles de ses éléments et de ses sous-groupes ?

Correction : par le corollaire de Lagrange, l'ordre d'un élément divise 15, donc appartient à $\{1, 3, 5, 15\}$. De même pour les sous-groupes. Si G contient un élément d'ordre 15, alors G est cyclique. (En fait, on montre que tout groupe d'ordre 15 est cyclique.)

Exemple 2 (Décomposition d'une permutation — Intermédiaire). Soit $\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 3 & 5 & 6 & 4 & 2 & 1 \end{pmatrix} \in S_6$. Décomposer σ en cycles disjoints, calculer son ordre et sa signature.

Correction : en suivant les images : $1 \rightarrow 3 \rightarrow 6 \rightarrow 1$ donne le cycle $(1\ 3\ 6)$. $2 \rightarrow 5 \rightarrow 2$ donne $(2\ 5)$. $4 \rightarrow 4$ est fixe. Donc $\sigma = (1\ 3\ 6)(2\ 5)$. L'ordre est $\text{ppcm}(3, 2) = 6$. La signature est $(-1)^{3-1} \cdot (-1)^{2-1} = 1 \cdot (-1) = -1$: σ est impaire.

Exemple 3 (Morphisme et noyau — Difficile). Montrer que $f : (\mathbb{Z}, +) \rightarrow (\mathbb{C}^*, \times)$, $k \mapsto e^{2i\pi k/n}$ est un morphisme de groupes. Déterminer son noyau et son image.

Correction : $f(k_1 + k_2) = e^{2i\pi(k_1+k_2)/n} = e^{2i\pi k_1/n} \cdot e^{2i\pi k_2/n} = f(k_1)f(k_2)$: c'est bien un morphisme. $\ker f = \{k \in \mathbb{Z} : e^{2i\pi k/n} = 1\} = \{k : n \mid k\} = n\mathbb{Z}$. $\text{Im } f = \{e^{2i\pi k/n} : k \in \mathbb{Z}\}$ est le groupe $\mathbb{U}_n$ des racines n -ièmes de l'unité. On peut alors « factoriser » f en un isomorphisme $\mathbb{Z}/n\mathbb{Z} \xrightarrow{\sim} \mathbb{U}_n$.

4.1.16 Exercices Python pour le §1.b

On expérimente S_n et $\mathbb{Z}/n\mathbb{Z}$ avec `sympy` et la bibliothèque standard.

```
1 from math import gcd
2 from sympy.combinatorics import Permutation
3 from sympy.combinatorics.named_groups import SymmetricGroup
4
5 # 1. Permutation : decomposition en cycles, ordre, signature
6 sigma = Permutation([2, 4, 5, 3, 1, 0]) # 0-indexed : envoie 1->3,
7     2->5, ...
8 print(f"sigma={sigma}")
9 print(f"cycles={sigma.cyclic_form}")
10 print(f"ordre={sigma.order()}")
11 print(f"signature={sigma.signature()}")
12
13 # 2. Verification du theoreme de Lagrange dans S_3
14 S3 = SymmetricGroup(3)
15 print(f"\n|S_3|={S3.order()}")
16 for h in S3.generate():
17     sub = h.cyclic_form
18     print(f"element {h} d'ordre {h.order()} (divise 6 : {6%h.order() == 0})")
19
20 # 3. Petit theoreme de Fermat : a^(p-1) = 1 mod p
21 def fermat_check(a, p):
22     return pow(a, p - 1, p) == 1
23
24 p = 13
```

```

24 print(f"\nFermat dans  $\mathbb{Z}/\{p\}\mathbb{Z}$  : ")
25 for a in range(1, p):
26     print(f" $a^{p-1} \bmod p = {pow(a, p-1, p)}$ ")
27
28 # 4. Générateurs de  $\mathbb{Z}/n\mathbb{Z}$  : les  $a$  tels que  $\text{pgcd}(a, n) = 1$ 
29 n = 12
30 generateurs = [a for a in range(1, n) if gcd(a, n) == 1]
31 print(f"\nGénérateurs de  $\mathbb{Z}/\{n\}\mathbb{Z}$  (additif) : {generateurs} (phi({n})) = {len(generateurs)}")

```

4.1.17 Exercices à faire (§1.b)

- Exercice 6.** Soit G un groupe d'ordre premier p . Montrer que les seuls sous-groupes de G sont $\{e\}$ et G . En déduire que G est cyclique et que tout élément $\neq e$ est générateur.
- Exercice 7.** Décomposer $\tau = (1\ 4\ 2)(3\ 5)(6\ 7\ 8\ 9) \in S_9$ en transpositions. Calculer $\varepsilon(\tau)$ et $o(\tau)$.
- Exercice 8.** Soit $f : (\mathbb{Z}, +) \rightarrow (\mathbb{Z}, +)$ un morphisme de groupes. Montrer qu'il existe un unique $a \in \mathbb{Z}$ tel que $f(k) = ak$ pour tout k . À quelle condition f est-il bijectif?
- Exercice 9 (Euler).** Calculer $7^{1000} \pmod{12}$ en utilisant le théorème d'Euler.
- Exercice 10 (synthèse).** Établir la table de $(\mathbb{Z}/8\mathbb{Z})^*$. Ce groupe est-il cyclique? À quel produit cartésien de groupes cycliques est-il isomorphe?

4.2 Structures d'anneau et de corps

4.2.1 Motivation

Un *groupe* capture une seule opération réversible. Mais beaucoup de structures usuelles — $\mathbb{Z}$, les polynômes, les matrices, $\mathbb{Z}/n\mathbb{Z}$, les complexes — en ont *deux* : addition et multiplication, reliées par la distributivité. Ce cadre commun est celui des **anneaux**. Lorsque la multiplication est en plus inversible (sauf pour 0), on obtient un **corps**, le terrain de jeu de l'algèbre linéaire et de toute la théorie des polynômes du chapitre suivant. La cryptographie à clé publique (RSA, ElGamal, courbes elliptiques) repose entièrement sur l'arithmétique d'anneaux et de corps finis.

4.2.2 Approche par problème : une anomalie dans $\mathbb{Z}/6\mathbb{Z}$

Problème : Dans $\mathbb{Z}$, le produit de deux entiers non nuls n'est jamais nul. Vérifier la même propriété dans $\mathbb{Z}/6\mathbb{Z}$. Combien l'équation $2x = 4$ admet-elle de solutions dans $\mathbb{Z}/6\mathbb{Z}$? Et dans $\mathbb{Z}/5\mathbb{Z}$?

Résolution : Dans $\mathbb{Z}/6\mathbb{Z}$, on a $\bar{2} \cdot \bar{3} = \bar{6} = \bar{0}$, alors que $\bar{2} \neq \bar{0}$ et $\bar{3} \neq \bar{0}$. C'est un phénomène de *diviseurs de zéro*, absent dans $\mathbb{Z}$. L'équation $2x = 4$ dans $\mathbb{Z}/6\mathbb{Z}$ admet *deux* solutions : $x = 2$ et $x = 5$ (car $2 \cdot 5 = 10 \equiv 4$). Dans $\mathbb{Z}/5\mathbb{Z}$, elle n'en a qu'une : $x = 2$ (car $\bar{2}$ est inversible d'inverse $\bar{3}$, donc $x = \bar{3} \cdot \bar{4} = \bar{12} = \bar{2}$). Cette différence motive la distinction *anneau intègre* / *corps*.

4.2.3 Définition d'un anneau

Définition 4.2.1 (Anneau). Un triplet $(A, +, \times)$ est un **anneau** si :

1. $(A, +)$ est un groupe abélien (de neutre noté 0_A , opposé noté $-a$) ;
2. $\times$ est une l.c.i *associative* sur A ;
3. $\times$ est *distributive* sur $+$: pour tous $a, b, c \in A$,

$$a \times (b + c) = a \times b + a \times c, \quad (a + b) \times c = a \times c + b \times c.$$

L'anneau est dit **unitaire** si $\times$ admet un neutre noté 1_A , avec la convention $1_A \neq 0_A$; **commutatif** si $\times$ est commutative.

Remarque 4.2.1. Dans tout ce cours, sauf mention contraire, le mot « anneau » désigne un *anneau unitaire*.

Exemple 4.2.1 (Anneaux usuels). — $(\mathbb{Z}, +, \times), (\mathbb{Q}, +, \times), (\mathbb{R}, +, \times), (\mathbb{C}, +, \times)$: commutatifs unitaires.

- $(\mathbb{Z}/n\mathbb{Z}, +, \times)$ pour tout $n \geq 1$: commutatif unitaire.
- $(K[X], +, \times)$ (polynômes à coefficients dans un corps K) : commutatif unitaire (chapitre suivant).
- $(M_n(K), +, \times)$ (matrices $n \times n$) : unitaire mais *non commutatif* dès $n \geq 2$.
- $(\mathcal{F}(\mathbb{R}, \mathbb{R}), +, \times)$ (fonctions réelles avec opérations point par point) : commutatif unitaire.

Proposition 4.2.1 (Règles de calcul dans un anneau). *Dans tout anneau A :*

1. $a \times 0_A = 0_A \times a = 0_A$ pour tout $a \in A$;
2. $a \times (-b) = (-a) \times b = -(a \times b)$;
3. $(-a) \times (-b) = a \times b$;
4. Si A est commutatif, la formule du binôme s'applique :

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k}.$$

4.2.4 Sous-anneau

Définition 4.2.2 (Sous-anneau). Une partie $B \subseteq A$ est un **sous-anneau** de A si :

1. $1_A \in B$;
2. B est stable par $+$ et par passage à l'opposé ;
3. B est stable par $\times$.

Caractérisation pratique : B est un sous-anneau ssi $1_A \in B$ et $\forall a, b \in B, a - b \in B$ et $a \times b \in B$.

Exemple 4.2.2. $\mathbb{Z}$ est sous-anneau de $\mathbb{Q}$, qui est sous-anneau de $\mathbb{R}$, qui est sous-anneau de $\mathbb{C}$. $\mathbb{Z}[i] = \{a + bi : a, b \in \mathbb{Z}\}$ (anneau des entiers de Gauss) est un sous-anneau de $\mathbb{C}$.

4.2.5 Anneau intègre, diviseurs de zéro

Définition 4.2.3 (Diviseur de zéro, anneau intègre). Soit A un anneau commutatif unitaire. Un élément $a \in A \setminus \{0_A\}$ est un **diviseur de zéro** s'il existe $b \in A \setminus \{0_A\}$ tel que $a \times b = 0_A$. On dit que A est **intègre** (ou que c'est un *domaine d'intégrité*) si A ne possède pas de diviseur de zéro, autrement dit :

$$\forall a, b \in A, \quad a \times b = 0_A \implies a = 0_A \text{ ou } b = 0_A.$$

Proposition 4.2.2 (Règle de simplification). Dans un anneau intègre A , on peut simplifier par tout élément non nul :

$$a \neq 0_A \text{ et } a \times b = a \times c \implies b = c.$$

Exemple 4.2.3. — $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}, K[X]$ sont intègres.

- $\mathbb{Z}/6\mathbb{Z}$ n'est pas intègre : $\bar{2} \cdot \bar{3} = \bar{0}$.
- Plus généralement, $\mathbb{Z}/n\mathbb{Z}$ est intègre si et seulement si n est premier.
- $M_n(K)$ pour $n \geq 2$ n'est pas intègre : il existe des matrices non nulles dont le produit est nul.

4.2.6 Idéaux (notion brève)

Définition 4.2.4 (Idéal). Soit A un anneau commutatif. Une partie $I \subseteq A$ est un **idéal** de A si :

1. $(I, +)$ est un sous-groupe de $(A, +)$;
2. $\forall a \in A, \forall x \in I, a \times x \in I$ (absorption).

Exemple 4.2.4. Les idéaux de $\mathbb{Z}$ sont exactement les $n\mathbb{Z}$ pour $n \in \mathbb{N}$ (ils coïncident avec les sous-groupes de $\mathbb{Z}$, propriété spéciale à $\mathbb{Z}$ liée à la division euclidienne). On dit que $\mathbb{Z}$ est un *anneau principal*.

4.2.7 Corps

Définition 4.2.5 (Corps). Un anneau K commutatif unitaire est un **corps** si tout élément non nul est inversible pour la multiplication. Autrement dit, $(K \setminus \{0_K\}, \times)$ est un groupe abélien.

Remarque 4.2.2. Certains auteurs ne demandent pas la commutativité dans la définition d'un corps. On parle alors de *corps gauche* ou *algèbre à division*. En suivant la convention française usuelle, nous incluons la commutativité.

Exemple 4.2.5 (Corps usuels). — $\mathbb{Q}, \mathbb{R}, \mathbb{C}$ sont des corps.

- $\mathbb{Z}/p\mathbb{Z}$ pour p premier est un corps fini, parfois noté $\mathbb{F}_p$.
- $\mathbb{Z}$ n'est pas un corps : 2 n'a pas d'inverse dans $\mathbb{Z}$.

Proposition 4.2.3 (Tout corps est intègre). Si K est un corps, alors K est un anneau intègre.

Démonstration. Soient $a, b \in K$ avec $a \times b = 0$. Si $a \neq 0$, a est inversible, donc $b = a^{-1} \times (a \times b) = a^{-1} \times 0 = 0$. Donc $a = 0$ ou $b = 0$. $\square$

Théorème 4.2.4 (Anneau intègre fini = corps). *Tout anneau commutatif unitaire intègre et fini est un corps.*

Démonstration. Soit A un anneau intègre fini et $a \in A \setminus \{0\}$. L'application $m_a : A \rightarrow A$, $x \mapsto a \times x$ est injective : si $a \times x = a \times y$, alors $a \times (x - y) = 0$, et l'intégrité donne $x = y$. Comme A est fini, m_a est aussi surjective. En particulier 1 a un antécédent : il existe $x \in A$ tel que $a \times x = 1$. Donc a est inversible. $\square$

Corollaire 4.2.5 (Caractérisation de $\mathbb{Z}/n\mathbb{Z}$ corps). *$\mathbb{Z}/n\mathbb{Z}$ est un corps si et seulement si n est un nombre premier.*

Démonstration. $\mathbb{Z}/n\mathbb{Z}$ est intègre ssi n est premier (vu plus haut). Étant fini, le théorème précédent donne le résultat. $\square$

4.2.8 Caractéristique d'un anneau

Définition 4.2.6 (Caractéristique). Soit A un anneau unitaire. La **caractéristique** de A , notée $\text{car}(A)$, est :

- le plus petit entier $n \geq 1$ tel que $\underbrace{1_A + 1_A + \cdots + 1_A}_{n \text{ fois}} = 0_A$, s'il existe ;
- 0 sinon.

Exemple 4.2.6. $\text{car}(\mathbb{Z}) = \text{car}(\mathbb{Q}) = \text{car}(\mathbb{R}) = \text{car}(\mathbb{C}) = 0$. $\text{car}(\mathbb{Z}/n\mathbb{Z}) = n$.

Proposition 4.2.6 (Caractéristique d'un anneau intègre). *La caractéristique d'un anneau intègre est soit 0, soit un nombre premier. En particulier, la caractéristique d'un corps est 0 ou un nombre premier.*

Démonstration. Si $\text{car}(A) = n \geq 2$ et $n = ab$ avec $1 < a, b < n$, alors $(a \cdot 1_A)(b \cdot 1_A) = (ab) \cdot 1_A = n \cdot 1_A = 0_A$. Par intégrité, $a \cdot 1_A = 0$ ou $b \cdot 1_A = 0$, ce qui contredit la minimalité de n . $\square$

4.2.9 Morphismes d'anneaux

Définition 4.2.7 (Morphisme d'anneaux). Soient A, B deux anneaux. Une application $f : A \rightarrow B$ est un **morphisme d'anneaux** si pour tous $a, b \in A$:

$$f(a + b) = f(a) + f(b), \quad f(a \times b) = f(a) \times f(b), \quad f(1_A) = 1_B.$$

Exemple 4.2.7. $\pi : \mathbb{Z} \rightarrow \mathbb{Z}/n\mathbb{Z}$, $k \mapsto \bar{k}$ est un morphisme d'anneaux surjectif. La conjugaison complexe $z \mapsto \bar{z}$ est un automorphisme du corps $\mathbb{C}$.

4.2.10 Exemples résolus (Difficulté ascendante)

Exemple 1 (Vérification d'un anneau — Facile). Montrer que $A = \mathbb{Z}[\sqrt{2}] = \{a + b\sqrt{2} : a, b \in \mathbb{Z}\}$ est un sous-anneau de $\mathbb{R}$.

Correction : $1 = 1 + 0 \cdot \sqrt{2} \in A$. Si $x = a + b\sqrt{2}$ et $y = c + d\sqrt{2}$ sont dans A , alors $x - y = (a - c) + (b - d)\sqrt{2} \in A$ et $x \times y = (ac + 2bd) + (ad + bc)\sqrt{2} \in A$. Par la caractérisation pratique, A est un sous-anneau de $\mathbb{R}$.

Exemple 2 (Inversibles dans $\mathbb{Z}/15\mathbb{Z}$ — Intermédiaire). Déterminer les éléments inversibles de l'anneau $\mathbb{Z}/15\mathbb{Z}$. Ce dernier est-il un corps ?

Correction : $\bar{a}$ est inversible ssi $\text{pgcd}(a, 15) = 1$. Les classes inversibles sont $\bar{1}, \bar{2}, \bar{4}, \bar{7}, \bar{8}, \bar{11}, \bar{13}, \bar{14}$, soit $\varphi(15) = 8$ éléments. Comme $15 = 3 \times 5$ n'est pas premier, $\mathbb{Z}/15\mathbb{Z}$ n'est pas un corps : par exemple $\bar{3} \cdot \bar{5} = \bar{0}$ avec $\bar{3}, \bar{5} \neq \bar{0}$.

Exemple 3 (Calcul dans un corps fini — Difficile). Dans le corps $\mathbb{F}_7 = \mathbb{Z}/7\mathbb{Z}$, calculer l'inverse de $\bar{3}$ et résoudre l'équation $3x + 5 = 1$.

Correction : on cherche $\bar{y}$ tel que $\bar{3} \cdot \bar{y} = \bar{1}$. Test : $\bar{3} \cdot \bar{5} = \bar{15} = \bar{1}$. Donc $\bar{3}^{-1} = \bar{5}$. L'équation devient $\bar{3} \cdot x = \bar{1} - \bar{5} = \bar{-4} = \bar{3}$. En multipliant par $\bar{3}^{-1} = \bar{5}$: $x = \bar{5} \cdot \bar{3} = \bar{15} = \bar{1}$. Vérification : $3 \cdot 1 + 5 = 8 \equiv 1 \pmod{7}$. ✓

4.2.11 Exercices pratiques avec Python

On exploite les fonctions natives de Python (`pow(a, -1, n)` disponible depuis Python 3.8) ou `sympy` pour calculer des inverses dans $\mathbb{Z}/n\mathbb{Z}$ et explorer la structure de corps fini.

```

1 from math import gcd
2 from sympy import isprime, totient
3
4 def inversibles_mod(n):
5     """Liste des classes inversibles dans Z/nZ."""
6     return [a for a in range(1, n) if gcd(a, n) == 1]
7
8 def inverse_mod(a, n):
9     """Inverse de a dans Z/nZ (None si non inversible)."""
10    return pow(a, -1, n) if gcd(a, n) == 1 else None
11
12 def est_corps(n):
13     """Z/nZ est un corps ssi n est premier."""
14     return isprime(n)
15
16 # Comparaison Z/15Z (anneau non integre) vs Z/13Z (corps)
17 for n in (15, 13):
18     inv = inversibles_mod(n)
19     print(f"Z/{n}Z : {len(inv)} éléments, phi = {totient(n)}")
20     print(f"est un corps : {est_corps(n)}")
21     if est_corps(n):
22         # Tableau des inverses dans le corps
23         print(f" inverses : {{a : a^(-1) for a in F_{n}*}} = " +
24               str({a: inverse_mod(a, n) for a in inv}))
25
26 # Verification des diviseurs de zero dans Z/12Z
27 print("\nDiviseurs de zero dans Z/12Z :")
28 for a in range(1, 12):
29     for b in range(a, 12):
30         if (a * b) % 12 == 0:
31             print(f" {a} * {b} = 0 mod 12")

```

4.2.12 Exercices à faire

1. **Exercice 1.** Soit A un anneau. Montrer que pour tout $a \in A$, $a \cdot 0 = 0$. (Indication : utiliser $0 = 0 + 0$ et la distributivité.)
2. **Exercice 2.** Lister tous les diviseurs de zéro et tous les inversibles de $\mathbb{Z}/10\mathbb{Z}$.
3. **Exercice 3.** Montrer que $\mathbb{Q}[\sqrt{3}] = \{a + b\sqrt{3} : a, b \in \mathbb{Q}\}$ est un corps. (Indication : justifier que $a + b\sqrt{3}$ admet un inverse en utilisant la quantité conjuguée $a - b\sqrt{3}$.)
4. **Exercice 4.** Résoudre l'équation $x^2 = \bar{1}$ dans $\mathbb{Z}/8\mathbb{Z}$. Combien de solutions ? Comparer avec le cas d'un corps.
5. **Exercice 5.** Soit A un anneau commutatif. Montrer que l'ensemble des éléments inversibles, noté $A^\times$, est un groupe pour la multiplication. Identifier $A^\times$ pour $A = \mathbb{Z}$, $A = \mathbb{Z}/n\mathbb{Z}$, $A = K[X]$ avec K corps.
6. **Exercice 6 (synthèse).** Donner la caractéristique des anneaux suivants : $\mathbb{Z}$, $\mathbb{Z}/12\mathbb{Z}$, $\mathbb{F}_5$, $\mathbb{F}_5[X]$, $\mathbb{F}_5 \times \mathbb{F}_3$ (produit direct).

Travaux Pratiques (TP) Python

L'objectif de ce TP est d'expérimenter sur machine les structures abstraites du chapitre : groupes finis $\mathbb{Z}/n\mathbb{Z}$ et S_n , théorème de Lagrange, anneaux et corps finis. On utilisera `itertools`, `sympy.combinatorics` et la fonction native `pow(a, k, n)` pour l'exponentiation modulaire.

Exercice 1 : Tables de Cayley dans $\mathbb{Z}/n\mathbb{Z}$

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire `table_cayley_addition(n)` et `table_cayley_multiplication(n)` qui retournent la table de Cayley sous forme d'une matrice $n \times n$ pour les lois de $\mathbb{Z}/n\mathbb{Z}$. Afficher joliment les tables pour $n = 6$.
2. **[Bloom : Analyse | Difficulté : Moyenne]**
Vérifier sur la table additive que chaque ligne et chaque colonne sont une permutation de $\{0, 1, \dots, n-1\}$. Pourquoi cette propriété caractérise-t-elle un groupe ? (Indication : penser à la fonction $x \mapsto a + x$.)

Exercice 2 : Vérification automatique des axiomes de groupe

1. **[Bloom : Application | Difficulté : Moyenne]**
Étant donné un ensemble fini G (liste) et une loi $\star : G \times G \rightarrow G$ (fonction Python), écrire `est_groupe(G, op)` qui retourne `True` ssi $(G, \star)$ est un groupe (test d'associativité, recherche de neutre, recherche d'inverses).
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester sur $(\mathbb{Z}/6\mathbb{Z}, +)$, $(\mathbb{Z}/6\mathbb{Z}, \times)$ (*n'est pas* un groupe), et $((\mathbb{Z}/7\mathbb{Z})^*, \times)$.
3. **[Bloom : Création | Difficulté : Difficile]**
Étendre en `est_abelien(G, op)`.

Exercice 3 : Ordre et sous-groupe engendré

1. **[Bloom : Application | Difficulté : Facile]**
Écrire `ordre(a, op, neutre, max_iter=1000)` qui calcule l'ordre d'un élément a dans un groupe (par itérations successives de la loi).

2. **[Bloom : Application | Difficulté : Moyenne]**
Écrire `sous_groupe_engendre(a, op)` qui retourne $\langle a \rangle$ comme un ensemble.
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Sur $((\mathbb{Z}/13\mathbb{Z})^*, \times)$, calculer l'ordre de chaque élément et identifier les générateurs (= éléments d'ordre 12 = $\varphi(13)$).

Exercice 4 : Énumération des sous-groupes d'un groupe cyclique

1. **[Bloom : Application | Difficulté : Moyenne]**
Les sous-groupes de $(\mathbb{Z}/n\mathbb{Z}, +)$ sont en bijection avec les diviseurs de n . Écrire `sous_groupes_de_ZnZ(n)` qui retourne un dictionnaire $\{d : H_d\}$ associant à chaque diviseur d de n le sous-groupe correspondant.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester pour $n = 12$. Vérifier que la somme des cardinaux des sous-groupes (par diviseur de n) vérifie une formule combinatoire connue.

Exercice 5 : Permutations, signatures et S_n

1. **[Bloom : Compréhension | Difficulté : Facile]**
Représenter une permutation comme un tuple $(\sigma(1), \sigma(2), \dots, \sigma(n))$. Écrire `compose(sigma, tau)` qui retourne $\sigma \circ \tau$.
2. **[Bloom : Application | Difficulté : Moyenne]**
Écrire `decomposition_cycles(sigma)` qui retourne la décomposition en cycles disjoints d'une permutation. Tester sur $(3, 5, 6, 4, 2, 1) \in S_6$.
3. **[Bloom : Synthèse | Difficulté : Moyenne]**
Écrire `signature(sigma)` qui calcule $\varepsilon(\sigma) = (-1)^{n-r}$ où r est le nombre de cycles (en comptant les points fixes comme des 1-cycles).
4. **[Bloom : Évaluation | Difficulté : Moyenne]**
Énumérer toutes les permutations de S_4 avec `itertools.permutations`, calculer leur signature, et vérifier que le groupe alterné A_4 a bien $4!/2 = 12$ éléments.

Exercice 6 : Vérification du théorème de Lagrange

1. **[Bloom : Synthèse | Difficulté : Difficile]**
Sur S_4 : pour chaque élément σ , calculer son ordre. Vérifier que tous les ordres divisent $|S_4| = 24$.
2. **[Bloom : Création | Difficulté : Difficile]**
Énumérer (programmatically) tous les sous-groupes de S_3 et vérifier que leurs cardinaux $(1, 2, 3, 6)$ divisent bien $|S_3| = 6$. Combien de sous-groupes au total ?

Exercice 7 : Application cryptographique — Diffie-Hellman

1. **[Bloom : Compréhension | Difficulté : Moyenne]**
Le protocole Diffie-Hellman repose sur la difficulté du *logarithme discret* dans $((\mathbb{Z}/p\mathbb{Z})^*, \times)$. Étant donné un nombre premier p et un générateur g de $(\mathbb{Z}/p\mathbb{Z})^*$, Alice choisit a et publie $A = g^a \bmod p$; Bob choisit b et publie $B = g^b \bmod p$. Le secret partagé est $K = g^{ab} \bmod p = A^b \bmod p = B^a \bmod p$.
Écrire `diffie_hellman(p, g, a, b)` qui retourne $(A, B, K_{\text{Alice}}, K_{\text{Bob}})$ et vérifie que les deux secrets coïncident. Tester avec $p = 23$, $g = 5$, $a = 6$, $b = 15$.

2. **[Bloom : Synthèse | Difficulté : Difficile]**

Écrire `est_generateur(g, p)` qui vérifie que g est un générateur de $(\mathbb{Z}/p\mathbb{Z})^*$ (i.e. son ordre vaut $p - 1$).

3. **[Bloom : Évaluation | Difficulté : Difficile]**

Pour $p = 23$, lister tous les générateurs. Vérifier que leur nombre vaut $\varphi(p - 1) = \varphi(22) = 10$.

Exercice 8 : Construction du corps fini $\mathbb{F}_4$

1. **[Bloom : Compréhension | Difficulté : Difficile]**

On construit $\mathbb{F}_4 = \mathbb{F}_2[X]/(X^2 + X + 1)$. Ses 4 éléments sont les classes de $0, 1, X, X + 1$, que l'on représente respectivement par les paires $(0, 0), (1, 0), (0, 1), (1, 1)$ (couples (c_0, c_1) pour $c_0 + c_1X$). Écrire `add_F4(p, q)` et `mul_F4(p, q)` qui implémentent l'addition et la multiplication dans $\mathbb{F}_4$ (avec la règle $X^2 = X + 1$ dans $\mathbb{F}_2$).

2. **[Bloom : Évaluation | Difficulté : Difficile]**

Construire les tables de Cayley additive et multiplicative de $\mathbb{F}_4$. Vérifier :

- $(\mathbb{F}_4, +)$ est un groupe abélien d'ordre 4 ;
- $(\mathbb{F}_4 \setminus \{0\}, \times)$ est un groupe abélien d'ordre 3 (donc cyclique) ;
- $\mathbb{F}_4$ est un corps (tout élément non nul est inversible).

3. **[Bloom : Création | Difficulté : Très Difficile]**

Vérifier que $\mathbb{F}_4 \neq \mathbb{Z}/4\mathbb{Z}$ comme anneaux : par exemple, dans $\mathbb{F}_4$ on a $1 + 1 = 0$ (caractéristique 2), alors que dans $\mathbb{Z}/4\mathbb{Z}$ on a $1 + 1 = 2 \neq 0$. La caractéristique distingue les deux.

Chapitre 5

Polynômes

Les polynômes constituent l'outil de calcul algébrique le plus polyvalent : interpolation et approximation numérique, traitement du signal, codes correcteurs (Reed-Solomon), cryptographie sur courbes elliptiques, méthodes à noyaux en apprentissage automatique. Ce chapitre développe leur arithmétique : structure d'anneau, racines, division euclidienne, factorisation et irréductibilité. Il prolonge directement le chapitre III (arithmétique dans $\mathbb{Z}$) et le chapitre IV (anneaux et corps).

Dans tout ce chapitre, K désigne un corps commutatif, le plus souvent $\mathbb{R}$ ou $\mathbb{C}$.

5.1 Anneaux $\mathbb{R}[X]$ et $\mathbb{C}[X]$

5.1.1 Motivation

Un polynôme « scolaire » comme $P(X) = X^2 - 3X + 2$ peut être vu de deux manières :

- Comme un *objet formel* (la suite des coefficients $(2, -3, 1, 0, 0, \dots)$) que l'on additionne et multiplie selon des règles algébriques.
- Comme une *fonction* $x \mapsto x^2 - 3x + 2$ que l'on évalue en chaque point.

Sur $\mathbb{R}$ ou $\mathbb{C}$, les deux points de vue sont équivalents : un polynôme est entièrement déterminé par sa fonction. Mais sur un corps fini, ce n'est plus vrai. Cette section pose les bases formelles : $K[X]$ comme *anneau abstrait*, indépendamment de l'évaluation. La distinction sera levée à la section suivante (§2).

5.1.2 Approche par problème : polynôme nul vs fonction nulle

Problème : Considérons sur le corps $\mathbb{F}_2 = \mathbb{Z}/2\mathbb{Z}$ le polynôme $P(X) = X^2 + X$. Calculer $P(\bar{0})$ et $P(\bar{1})$. Que peut-on dire ? Le polynôme $X^2 + X$ est-il « nul » ?

Résolution : $P(\bar{0}) = \bar{0}$ et $P(\bar{1}) = \bar{1}^2 + \bar{1} = \bar{1} + \bar{1} = \bar{0}$. La fonction $x \mapsto P(x)$ est donc identiquement nulle sur $\mathbb{F}_2$. Pourtant le polynôme $X^2 + X$ a des coefficients non tous nuls : il *n'est pas* le polynôme nul. Conclusion : sur $\mathbb{F}_2$, deux polynômes distincts peuvent définir la même fonction. Sur $\mathbb{R}$ ou $\mathbb{C}$, ce phénomène n'arrive pas. Il faut donc distinguer rigoureusement « polynôme » (objet algébrique formel) et « fonction polynomiale ».

5.1.3 Construction formelle de $K[X]$

Définition 5.1.1 (Polynôme à coefficients dans K). Un **polynôme** à coefficients dans K est une suite $(a_n)_{n \in \mathbb{N}}$ d'éléments de K *presque tous nuls* (i.e. tous nuls à partir d'un

certain rang). On note l'ensemble de ces suites $K[X]$.

Si $(a_n) \in K[X]$, on l'écrit :

$$P = \sum_{n=0}^{+\infty} a_n X^n = a_0 + a_1 X + a_2 X^2 + \dots$$

en utilisant le symbole X comme *indéterminée*. Les a_n sont les **coefficients** de P . Le **polynôme nul**, noté 0, est la suite identiquement nulle.

Définition 5.1.2 (Opérations sur $K[X]$). Soient $P = \sum a_n X^n$ et $Q = \sum b_n X^n$ deux polynômes de $K[X]$, et $\lambda \in K$.

$$P + Q = \sum_n (a_n + b_n) X^n,$$

$$\lambda P = \sum_n (\lambda a_n) X^n,$$

$$P \times Q = \sum_n c_n X^n \quad \text{où} \quad c_n = \sum_{k=0}^n a_k b_{n-k}.$$

La formule du produit est appelée *produit de Cauchy*. Elle correspond exactement à la multiplication usuelle des polynômes.

Théorème 5.1.1 ($K[X]$ est un anneau commutatif unitaire). *Muni de $+$ et $\times$ ci-dessus, $K[X]$ est un anneau commutatif unitaire. Le neutre additif est le polynôme nul, le neutre multiplicatif est le polynôme constant 1.*

5.1.4 Degré et valuation

Définition 5.1.3 (Degré). Soit $P = \sum a_n X^n \in K[X]$ non nul. Le **degré** de P , noté $\deg P$, est le plus grand entier n tel que $a_n \neq 0$. On pose par convention $\deg 0 = -\infty$.

Le coefficient $a_{\deg P}$ est le **coefficient dominant** de P . Si ce coefficient vaut 1, le polynôme est dit **unitaire** (ou monique).

Définition 5.1.4 (Valuation). La **valuation** de $P \neq 0$, notée $v(P)$, est le plus petit entier n tel que $a_n \neq 0$. Par convention, $v(0) = +\infty$.

Proposition 5.1.2 (Propriétés du degré). *Pour tous $P, Q \in K[X]$:*

1. $\deg(P + Q) \leq \max(\deg P, \deg Q)$, avec égalité si $\deg P \neq \deg Q$.
2. $\deg(P \times Q) = \deg P + \deg Q$.
3. $\deg(\lambda P) = \deg P$ pour tout $\lambda \in K^*$.

(Conventions : $-\infty + n = -\infty$, $\max(-\infty, n) = n$.)

Démonstration. (1) Le coefficient de X^n dans $P + Q$ est $a_n + b_n$, qui est nul si $n > \max(\deg P, \deg Q)$. Si $\deg P > \deg Q$, le coefficient de $X^{\deg P}$ dans $P + Q$ vaut $a_{\deg P} \neq 0$.

(2) Le coefficient dominant de PQ est $a_{\deg P} \cdot b_{\deg Q}$, non nul puisque K est un corps (donc intègre).

(3) Conséquence immédiate de (2) avec $Q = \lambda$ (constante non nulle). $\square$

Corollaire 5.1.3 (Intégrité de $K[X]$). *Pour tout corps K , l'anneau $K[X]$ est intègre. En particulier, $\mathbb{R}[X]$ et $\mathbb{C}[X]$ sont des anneaux commutatifs unitaires intègres.*

Démonstration. Si $P \neq 0$ et $Q \neq 0$, alors $\deg(PQ) = \deg P + \deg Q \geq 0$, donc $PQ \neq 0$. $\square$

5.1.5 Inversibles de $K[X]$

Proposition 5.1.4 (Inversibles). *Les éléments inversibles de $K[X]$ pour la multiplication sont exactement les constantes non nulles :*

$$K[X]^\times = K^* = K \setminus \{0\}.$$

Démonstration. Si $P \in K^*$, son inverse dans K est aussi son inverse dans $K[X]$. Réciproquement, si $PQ = 1$, alors $\deg P + \deg Q = \deg 1 = 0$. Comme les degrés sont ≥ 0 , on a $\deg P = \deg Q = 0$: P et Q sont des constantes non nulles. $\square$

Remarque 5.1.1. Ce résultat est analogue à $\mathbb{Z}^\times = \{-1, 1\}$: un anneau intègre n'est pas un corps en général. C'est précisément ce qui rend l'étude arithmétique de $K[X]$ riche (divisibilité, irréductibilité, factorisation), à l'image de l'arithmétique dans $\mathbb{Z}$.

5.1.6 Cas particuliers : $\mathbb{R}[X]$ et $\mathbb{C}[X]$

Les corps $\mathbb{R}$ et $\mathbb{C}$ sont les terrains de jeu privilégiés du chapitre.

- $\mathbb{R}[X]$ et $\mathbb{C}[X]$ sont tous deux des anneaux commutatifs unitaires intègres.
- L'inclusion $\mathbb{R} \subset \mathbb{C}$ induit une inclusion $\mathbb{R}[X] \subset \mathbb{C}[X]$ (un polynôme à coefficients réels est en particulier à coefficients complexes).
- La conjugaison complexe se prolonge en un automorphisme de $\mathbb{C}[X]$: si $P = \sum a_k X^k \in \mathbb{C}[X]$, on note $\bar{P} = \sum \bar{a}_k X^k$. On a $P \in \mathbb{R}[X] \iff \bar{P} = P$.
- Le théorème fondamental de l'algèbre (admis ici, prouvé en analyse complexe) affirme que tout polynôme non constant de $\mathbb{C}[X]$ admet au moins une racine dans $\mathbb{C}$. Cette propriété fera de $\mathbb{C}$ un corps *algébriquement clos*, et structurera toute la décomposition étudiée au §4.

5.1.7 Composition de polynômes

Définition 5.1.5 (Composition). Soient $P = \sum a_k X^k$ et $Q \in K[X]$. La **composée** de P par Q est :

$$P \circ Q = P(Q) = \sum_{k=0}^{\deg P} a_k Q^k \in K[X].$$

Proposition 5.1.5. *Si $\deg P, \deg Q \geq 1$, alors $\deg(P \circ Q) = \deg P \cdot \deg Q$.*

5.1.8 Exemples résolus (Difficulté ascendante)

Exemple 1 (Calcul direct — Facile). Soient $P = 2X^3 - X + 5$ et $Q = X^2 + 3$ dans $\mathbb{R}[X]$. Calculer $P + Q$, $P \times Q$ et leurs degrés.

Correction : $P+Q = 2X^3 + X^2 - X + 8$, de degré 3. $P \times Q = (2X^3)(X^2+3) + (-X)(X^2+3) + 5(X^2+3) = 2X^5 + 6X^3 - X^3 - 3X + 5X^2 + 15 = 2X^5 + 5X^3 + 5X^2 - 3X + 15$, de degré $5 = 3 + 2$. $\checkmark$

Exemple 2 (Conjugaison — Intermédiaire). Soit $P = X^3 + (2-i)X + 4i \in \mathbb{C}[X]$. Calculer $\bar{P}$, puis $P + \bar{P}$ et $P \cdot \bar{P}$. Lequel des deux résultats appartient à $\mathbb{R}[X]$?

Correction : $\bar{P} = X^3 + (2+i)X - 4i$. $P + \bar{P} = 2X^3 + 4X$: à coefficients réels, donc dans $\mathbb{R}[X]$. $\checkmark$ $P \cdot \bar{P}$ a aussi tous ses coefficients réels (on multiplie un polynôme par son conjugué). En général, $P + \bar{P}$, $P \cdot \bar{P}$ et $i(P - \bar{P})$ sont des polynômes réels, peu importe $P \in \mathbb{C}[X]$.

Exemple 3 (Identification de coefficients — Difficile). Trouver tous les polynômes $P \in \mathbb{R}[X]$ tels que $P(X+1) - P(X) = X$.

Correction : si $\deg P = n$, alors $\deg(P(X+1) - P(X)) = n-1$ (le coefficient dominant disparaît par soustraction). Comme le membre de droite a degré 1, on a $n = 2$. Posons $P = aX^2 + bX + c$. $P(X+1) - P(X) = a(X+1)^2 + b(X+1) + c - aX^2 - bX - c = a(2X+1) + b = 2aX + a + b$. Identification avec X : $2a = 1$ et $a + b = 0$, soit $a = 1/2$ et $b = -1/2$. La constante c reste libre. Donc $P(X) = \frac{X^2-X}{2} + c = \frac{X(X-1)}{2} + c$.

(On reconnaît $\binom{X}{2}$, lié au calcul de $\sum_{k=0}^{n-1} k$.)

5.1.9 Exercices pratiques avec Python

`numpy.polynomial.Polynomial` et `sympy` permettent de manipuler symboliquement et numériquement les polynômes.

```

1 import numpy as np
2 from numpy.polynomial import Polynomial
3 import sympy as sp
4
5 # 1. Numpy : polynomes a coefficients reels (ordre croissant des
6   coefficients)
7 P = Polynomial([5, -1, 0, 2])      # 5 - X + 2 X^3
8 Q = Polynomial([3, 0, 1])          # 3 + X^2
9 print(f"P={P}")
10 print(f"Q={Q}")
11 print(f"deg(P)={P.degree()}, deg(Q)={Q.degree()}")
12 print(f"P+Q={P+Q}")
13 print(f"P*Q={P*Q} (deg={ (P*Q).degree() })")
14
15 # 2. Sympy : calcul symbolique exact
16 X = sp.symbols('X')
17 P_s = 2*X**3 - X + 5
18 Q_s = X**2 + 3
19 print(f"\nP*Q (sympy)={sp.expand(P_s*Q_s)}")
20
21 # 3. Resolution de l'exemple 3 : trouver P tel que P(X+1) - P(X) =
22   X
23 P_inconnu = sum(sp.Symbol(f'a{i}') * X**i for i in range(3))
24 eq = sp.expand(P_inconnu.subs(X, X + 1) - P_inconnu - X)
25 sol = sp.solve(sp.Poly(eq, X).all_coeffs(), [sp.Symbol('a0'), sp.
26   Symbol('a1'), sp.Symbol('a2')])
27 print(f"\nSolution P={sol} (a0 reste libre)")

```

5.1.10 Exercices à faire

- Exercice 1.** Soit $P = 3X^4 - 2X^2 + X - 1$ et $Q = X^3 + X$ dans $\mathbb{Q}[X]$. Calculer $P + Q$, $P - Q$, PQ et leurs degrés.
- Exercice 2.** Démontrer que $\deg(P \circ Q) = \deg P \cdot \deg Q$ lorsque P, Q sont non constants. Que se passe-t-il si Q est une constante ?
- Exercice 3.** Trouver tous les $P \in \mathbb{R}[X]$ vérifiant $P(X^2) = (X^2 + 1)P(X)$. (Indication : commencer par étudier le degré, puis le coefficient dominant.)

4. **Exercice 4.** Combien y a-t-il de polynômes unitaires de degré 3 dans $\mathbb{F}_2[X]$? Les énumérer.
5. **Exercice 5 (synthèse).** Soit $P \in \mathbb{C}[X]$. Montrer les équivalences :
- $P \in \mathbb{R}[X] \iff \bar{P} = P$;
 - $P + \bar{P}$, $P \cdot \bar{P}$ et $i(P - \bar{P})$ sont toujours dans $\mathbb{R}[X]$.

5.2 Fonctions polynomiales et racines

5.2.1 Motivation

La résolution d'équations polynomiales est l'un des plus vieux problèmes des mathématiques. De la formule du second degré apprise au lycée à la résolution numérique en analyse, en passant par la cryptographie sur courbes elliptiques (où les opérations reposent sur des racines de polynômes de degré 3) et les codes correcteurs d'erreurs (Reed-Solomon détecte les erreurs en évaluant un polynôme), *les racines structurent toute l'algèbre du calcul*. Cette section formalise le lien entre un polynôme (objet abstrait) et la fonction qu'il définit, et caractérise ses racines par leur multiplicité.

5.2.2 Approche par problème : reconstruire un polynôme à partir de ses racines

Problème : Un polynôme $P \in \mathbb{R}[X]$ unitaire de degré 5 admet pour racines 1 (simple), 2 (double) et $1 + i$ (simple). Reconstruire P .

Résolution : Comme P est à coefficients réels, $\overline{1+i} = 1-i$ est aussi racine, et de même multiplicité que $1+i$ (théorème ci-dessous). On a donc 5 racines comptées avec multiplicité : 1, 2, 2, $1+i$, $1-i$. Comme $\deg P = 5$ et que P est unitaire :

$$P = (X-1)(X-2)^2(X-(1+i))(X-(1-i)) = (X-1)(X-2)^2(X^2-2X+2).$$

On vérifie que $(X-(1+i))(X-(1-i)) = X^2-2X+(1+i)(1-i) = X^2-2X+2$, à coefficients réels. ✓

5.2.3 Évaluation et fonction polynomiale

Définition 5.2.1 (Évaluation, fonction polynomiale). Soit $P = \sum_{k=0}^n a_k X^k \in K[X]$ et $a \in K$. L'évaluation de P en a est

$$P(a) = \sum_{k=0}^n a_k a^k \in K.$$

La **fonction polynomiale** associée à P est l'application $\tilde{P} : K \rightarrow K$, $x \mapsto P(x)$.

Proposition 5.2.1 (Morphisme d'évaluation). Pour tout $a \in K$, l'application $\text{ev}_a : K[X] \rightarrow K$, $P \mapsto P(a)$ est un morphisme d'anneaux. Autrement dit :

$$(P+Q)(a) = P(a) + Q(a), \quad (P \cdot Q)(a) = P(a) \cdot Q(a), \quad 1(a) = 1.$$

5.2.4 Racines et factorisation par $(X - a)$

Définition 5.2.2 (Racine). Un élément $a \in K$ est une **racine** (ou *zéro*) de $P \in K[X]$ si $P(a) = 0$.

Théorème 5.2.2 (Caractérisation par factorisation). Soit $P \in K[X]$ et $a \in K$. Alors a est racine de P si et seulement si $(X - a)$ divise P dans $K[X]$, c'est-à-dire qu'il existe $Q \in K[X]$ tel que $P = (X - a)Q$.

Démonstration. ($\Leftarrow$) Si $P = (X - a)Q$, alors $P(a) = 0 \cdot Q(a) = 0$.

($\Rightarrow$) On utilise l'identité $X^k - a^k = (X - a)(X^{k-1} + aX^{k-2} + \dots + a^{k-1})$. Pour $P = \sum a_k X^k$, on a

$$P(X) - P(a) = \sum_k a_k (X^k - a^k) = (X - a) \cdot \underbrace{\sum_k a_k (X^{k-1} + aX^{k-2} + \dots + a^{k-1})}_{=Q(X)}.$$

Si $P(a) = 0$, ceci s'écrit $P = (X - a)Q$. □

5.2.5 Multiplicité d'une racine

Définition 5.2.3 (Multiplicité). Soit a une racine de $P \in K[X]$, $P \neq 0$. La **multiplicité** de a dans P est l'entier $m \geq 1$ tel que :

$$(X - a)^m \mid P \quad \text{et} \quad (X - a)^{m+1} \nmid P.$$

On la note $\text{mult}_a(P)$. Si $m = 1$, on dit que a est une racine *simple*, si $m = 2$ *double*, et plus généralement de *multiplicité* m .

Si a n'est pas racine, on pose $\text{mult}_a(P) = 0$.

5.2.6 Polynôme dérivé

Définition 5.2.4 (Dérivée formelle). Soit $P = \sum_{k=0}^n a_k X^k \in K[X]$. Le **polynôme dérivé** de P est

$$P' = \sum_{k=1}^n k a_k X^{k-1} \in K[X].$$

On définit récursivement les dérivées d'ordre supérieur : $P^{(0)} = P$ et $P^{(m+1)} = (P^{(m)})'$.

Proposition 5.2.3 (Règles de dérivation). La dérivation $P \mapsto P'$ est K -linéaire et vérifie la règle de Leibniz :

$$(P + Q)' = P' + Q', \quad (\lambda P)' = \lambda P', \quad (PQ)' = P'Q + PQ'.$$

On en déduit la formule de la composée : $(P \circ Q)' = (P' \circ Q) \cdot Q'$. Si $\text{car}(K) = 0$ et $\deg P \geq 1$, alors $\deg P' = \deg P - 1$.

Théorème 5.2.4 (Caractérisation de la multiplicité par les dérivées). Soit K un corps de caractéristique 0 (en particulier $\mathbb{Q}, \mathbb{R}, \mathbb{C}$). Soit $P \in K[X]$ non nul et $a \in K$. La multiplicité de a dans P est m si et seulement si :

$$P(a) = P'(a) = \dots = P^{(m-1)}(a) = 0 \quad \text{et} \quad P^{(m)}(a) \neq 0.$$

Démonstration. Écrivons $P = (X - a)^m Q$ avec $Q(a) \neq 0$. Par la règle de Leibniz, on calcule successivement les dérivées : pour $k < m$, $P^{(k)}(a) = 0$, et pour $k = m$, $P^{(m)}(a) = m! Q(a) \neq 0$ (en caractéristique 0). $\square$

Remarque 5.2.1. Cette caractérisation est *fausse* en caractéristique non nulle. Sur $\mathbb{F}_p[X]$, le polynôme $(X - a)^p$ a pour dérivée $p(X - a)^{p-1} = 0$, ce qui invaliderait la formule. Dans nos applications usuelles $(\mathbb{R}, \mathbb{C})$, aucun problème.

5.2.7 Borne sur le nombre de racines

Théorème 5.2.5 (Nombre de racines dans un anneau intègre). *Soit K un corps (ou plus généralement un anneau intègre) et $P \in K[X]$ non nul. Le nombre de racines de P dans K , comptées avec multiplicité, est au plus $\deg P$.*

Démonstration. Par récurrence sur le degré. Si $\deg P = 0$, P est une constante non nulle, sans racine. Pour $\deg P \geq 1$: si P a une racine a , on factorise $P = (X - a)^m Q$ avec $Q(a) \neq 0$, où $m \geq 1$ est la multiplicité. Alors $\deg Q = \deg P - m$. Par récurrence, Q a au plus $\deg P - m$ racines, donc P en a au plus $m + (\deg P - m) = \deg P$. $\square$

Corollaire 5.2.6 (Polynôme nul à beaucoup de racines). *Si $P \in K[X]$ admet strictement plus de $\deg P$ racines distinctes (ou comptées avec multiplicité), alors $P = 0$.*

5.2.8 Identification polynôme-fonction

Théorème 5.2.7 (Polynôme $\leftrightarrow$ fonction sur un corps infini). *Si K est un corps infini (par exemple $\mathbb{Q}, \mathbb{R}, \mathbb{C}$), l'application $P \mapsto \tilde{P}$ est injective. Autrement dit :*

$$\tilde{P} = \tilde{Q} \iff P = Q.$$

Démonstration. Si $\tilde{P} = \tilde{Q}$, alors $\widetilde{P - Q} = 0$, donc $P - Q$ admet une infinité de racines (tout élément de K). Par le corollaire précédent, $P - Q = 0$. $\square$

Remarque 5.2.2 (Cas des corps finis). Sur $\mathbb{F}_p$, le théorème de Fermat dit que $a^p = a$ pour tout $a \in \mathbb{F}_p$. Donc le polynôme $X^p - X$ est nul comme fonction, mais non nul comme polynôme (de degré p). Cette distinction est cruciale en théorie des corps finis et en codage.

5.2.9 Polynômes à coefficients réels

Théorème 5.2.8 (Racines complexes conjuguées). *Soit $P \in \mathbb{R}[X]$ et $z \in \mathbb{C}$ une racine de P de multiplicité m . Alors $\bar{z}$ est aussi racine de P , avec la même multiplicité m .*

Démonstration. Comme $P \in \mathbb{R}[X]$, $\bar{P} = P$. Donc $P(\bar{z}) = \overline{P(z)} = \bar{0} = 0$. Pour la multiplicité : si $P = (X - z)^m Q$, alors en conjuguant $P = (X - \bar{z})^m \bar{Q}$, et la multiplicité de $\bar{z}$ est exactement m . $\square$

Corollaire 5.2.9 (Racine réelle d'un polynôme de degré impair). *Tout polynôme de $\mathbb{R}[X]$ de degré impair admet au moins une racine réelle.*

Démonstration. Sur $\mathbb{C}$, P admet $\deg P$ racines comptées avec multiplicité (théorème fondamental, §1). Les racines non réelles s'apparient en couples $(z, \bar{z})$ de même multiplicité, donc leur nombre total est pair. Comme $\deg P$ est impair, le nombre de racines réelles l'est aussi : il y en a au moins une.

(Argument analytique alternatif : $\lim_{x \rightarrow \pm\infty} P(x) = \pm\infty$ avec des signes opposés ; le théorème des valeurs intermédiaires donne une racine.) $\square$

5.2.10 Exemples résolus (Difficulté ascendante)

Exemple 1 (Factorisation par racines évidentes — Facile). Factoriser $P = X^3 - 6X^2 + 11X - 6$ dans $\mathbb{R}[X]$.

Correction : on teste les diviseurs entiers de 6. $P(1) = 1 - 6 + 11 - 6 = 0$, donc 1 est racine. Division : $P = (X - 1)(X^2 - 5X + 6) = (X - 1)(X - 2)(X - 3)$.

Exemple 2 (Multiplicité par dérivées — Intermédiaire). Soit $P = X^4 - 4X^3 + 6X^2 - 4X + 1$. Montrer que 1 est racine de multiplicité 4.

Correction : on remarque que $P = (X - 1)^4$ par le binôme. Vérification par dérivées : $P(1) = 0$. $P' = 4X^3 - 12X^2 + 12X - 4$, $P'(1) = 0$. $P'' = 12X^2 - 24X + 12$, $P''(1) = 0$. $P''' = 24X - 24$, $P'''(1) = 0$. $P^{(4)} = 24 \neq 0$. Donc $\text{mult}_1(P) = 4$. ✓

Exemple 3 (Polynôme à coefficients réels et racine complexe — Difficile). Le polynôme $P = X^4 + 1$ a-t-il des racines réelles ? Le factoriser dans $\mathbb{R}[X]$.

Correction : pour tout $x \in \mathbb{R}$, $x^4 + 1 \geq 1 > 0$, donc P n'a pas de racine réelle. Sur $\mathbb{C}$, les racines sont les solutions de $z^4 = -1 = e^{i\pi}$, soit $z_k = e^{i(\pi+2k\pi)/4}$ pour $k = 0, 1, 2, 3$:

$$z_0 = e^{i\pi/4}, z_1 = e^{3i\pi/4}, z_2 = e^{5i\pi/4} = \bar{z}_1, z_3 = e^{7i\pi/4} = \bar{z}_0.$$

Pour factoriser sur $\mathbb{R}$, on regroupe les racines conjuguées :

$$(X - z_0)(X - \bar{z}_0) = X^2 - 2\text{Re}(z_0)X + |z_0|^2 = X^2 - \sqrt{2}X + 1,$$

$$(X - z_1)(X - \bar{z}_1) = X^2 - 2\text{Re}(z_1)X + 1 = X^2 + \sqrt{2}X + 1.$$

Donc $P = (X^2 - \sqrt{2}X + 1)(X^2 + \sqrt{2}X + 1)$.

5.2.11 Exercices pratiques avec Python

sympy permet le calcul exact des racines avec multiplicité, la factorisation symbolique et la dérivation.

```
1 import sympy as sp
2
3 X = sp.symbols('X')
4
5 # 1. Factorisation et racines avec multiplicité
6 P = X**5 - 5*X**4 + 10*X**3 - 10*X**2 + 5*X - 1
7 print(f"P={P}")
8 print(f"factorise={sp.factor(P)}")
9 print(f"racines={sp.roots(P,X)}") # dictionnaire racine
   -> multiplicité
10
11 # 2. Caractérisation par dérivées : 1 est-il racine multiple ?
12 def multiplicite(poly, a, var):
13     """Multiplicité de a dans poly, par dérivées successives."""
14     m = 0
15     Q = sp.expand(poly)
16     while sp.simplify(Q.subs(var, a)) == 0:
17         m += 1
18         Q = sp.diff(Q, var)
19     return m
```

```

20
21 P_test = (X - 1)**3 * (X + 2)**2 * (X - 5)
22 print(f"\nP_test={sp.expand(P_test)}")
23 for a in [-2, 1, 5, 0]:
24     print(f"mult({a})={multiplicite(P_test, a, X)}")
25
26 # 3. Coefficients reels => racines complexes conjuguées
27 P_real = X**4 + 1
28 print(f"\nRacines complexes de X^4+1:")
29 for r in sp.solve(P_real, X):
30     print(f"{{r}}(numerique:{{complex(r):.4f}})")
31 print(f"Factorisation réelle:{{sp.factor(P_real, extension=[sp.
    sqrt(2)])}}")

```

5.2.12 Exercices à faire

1. **Exercice 1.** Factoriser $X^4 - 1$ dans $\mathbb{R}[X]$ et dans $\mathbb{C}[X]$. Combien de racines distinctes dans chaque corps ?
2. **Exercice 2.** Trouver le polynôme unitaire de $\mathbb{R}[X]$ de degré 4 admettant 1 pour racine simple, 2 pour racine double et -1 pour racine simple. Vérifier par développement.
3. **Exercice 3.** Démontrer que tout polynôme de $\mathbb{R}[X]$ se factorise comme produit (à un scalaire près) de polynômes de degré 1 et de polynômes de degré 2 à discriminant strictement négatif. (Utiliser la factorisation sur $\mathbb{C}$ et le théorème des racines conjuguées.)
4. **Exercice 4.** Soit $P \in \mathbb{R}[X]$ avec $\deg P = n \geq 1$. Montrer qu'entre deux racines réelles consécutives de P , P' admet au moins une racine. (Théorème de Rolle algébrique — on pourra invoquer Rolle de l'analyse.)
5. **Exercice 5 (synthèse).** Sur le corps $\mathbb{F}_5$, lister tous les polynômes unitaires de degré 2 qui n'ont aucune racine dans $\mathbb{F}_5$. (Indication : il y en a 10.)

5.3 Arithmétique dans $K[X]$

5.3.1 Motivation

L'anneau $K[X]$ partage avec l'anneau $\mathbb{Z}$ une propriété cruciale : il est muni d'une *division euclidienne*. Cette analogie n'est pas anecdotique — elle entraîne mécaniquement toute la théorie arithmétique du chapitre III (PGCD, algorithme d'Euclide, identité de Bézout, théorème de Gauss). En pratique, cela permet de *factoriser* les polynômes, de *simplifier les fractions rationnelles* (chapitre VI) et fonde l'arithmétique des codes correcteurs et des extensions de corps. Sur le plan algorithmique, le *procédé de Hörner* fournit une évaluation et une division par $(X - a)$ en temps linéaire, technique anciennement attribuée à Sharaf Eddine Al-Tusi (XII^e siècle).

5.3.2 Approche par problème : PGCD de deux polynômes

Problème : Calculer le PGCD de $A = X^4 - 1$ et $B = X^3 - X$ dans $\mathbb{R}[X]$.

Résolution : On peut deviner par factorisation : $A = (X - 1)(X + 1)(X^2 + 1)$ et $B = X(X - 1)(X + 1)$. Les facteurs communs sont $(X - 1)(X + 1) = X^2 - 1$. Mais en l'absence de factorisation évidente, comment procéder algorithmiquement ? Réponse : par divisions euclidiennes successives, exactement comme dans $\mathbb{Z}$. C'est l'algorithme d'Euclide pour les polynômes, que cette section formalise.

5.3.3 Divisibilité dans $K[X]$

Définition 5.3.1 (Divisibilité). Soient $A, B \in K[X]$. On dit que B **divise** A , et on note $B \mid A$, s'il existe $Q \in K[X]$ tel que $A = BQ$. Tout diviseur de A est aussi appelé un *facteur* de A .

Proposition 5.3.1 (Propriétés de la divisibilité). Pour $A, B, C \in K[X]$:

1. $A \mid A$ (réflexivité) ; $1 \mid A$; $A \mid 0$.
2. Si $A \mid B$ et $B \mid C$, alors $A \mid C$ (transitivité).
3. Si $A \mid B$ et $A \mid C$, alors $A \mid (\lambda B + \mu C)$ pour tous $\lambda, \mu \in K[X]$.
4. Si $A \mid B$ avec A, B non nuls, alors $\deg A \leq \deg B$.

Définition 5.3.2 (Polynômes associés). Deux polynômes A, B sont **associés** s'il existe $\lambda \in K^*$ tel que $A = \lambda B$. Cette relation est une équivalence ; deux polynômes associés ont les mêmes diviseurs.

Remarque 5.3.1. Pour avoir l'unicité du PGCD (à venir), on impose souvent qu'il soit *unitaire* (coefficient dominant 1). C'est l'analogue dans $K[X]$ du choix « positif » dans $\mathbb{Z}$.

5.3.4 Division euclidienne

Théorème 5.3.2 (Division euclidienne dans $K[X]$). Soient $A, B \in K[X]$ avec $B \neq 0$. Il existe un unique couple $(Q, R) \in K[X]^2$ tel que :

$$A = BQ + R \quad \text{avec} \quad \deg R < \deg B.$$

Q est appelé le **quotient** et R le **reste** de la division euclidienne de A par B .

Démonstration. Existence. Récurrence sur $\deg A$. Si $\deg A < \deg B$, on prend $Q = 0$ et $R = A$. Sinon, posons $A = a_n X^n + \dots$, $B = b_p X^p + \dots$ avec $n \geq p$. Alors $A_1 = A - \frac{a_n}{b_p} X^{n-p} B$ a un degré strictement inférieur à $\deg A$. Par hypothèse de récurrence, $A_1 = BQ_1 + R$ avec $\deg R < \deg B$, d'où $A = B(Q_1 + \frac{a_n}{b_p} X^{n-p}) + R$.

Unicité. Si $A = BQ_1 + R_1 = BQ_2 + R_2$ avec $\deg R_i < \deg B$, alors $B(Q_1 - Q_2) = R_2 - R_1$. Si $Q_1 \neq Q_2$, on aurait $\deg(B(Q_1 - Q_2)) \geq \deg B > \deg(R_2 - R_1)$, absurde. Donc $Q_1 = Q_2$ et $R_1 = R_2$. $\square$

Remarque 5.3.2. Cette division se pratique « à la main » de manière analogue à la division des entiers en colonne : on aligne les puissances de X et on élimine le terme dominant pas à pas.

5.3.5 Procédé de Hörner et division par $(X - a)$

Proposition 5.3.3 (Procédé de Hörner). Soit $P = a_n X^n + a_{n-1} X^{n-1} + \dots + a_0 \in K[X]$ et $a \in K$. Posons :

$$b_n = a_n, \quad b_{k-1} = a_{k-1} + a b_k \quad \text{pour } k = n, n-1, \dots, 1.$$

Alors :

- $P(a) = b_0$;
- $Q(X) = b_n X^{n-1} + b_{n-1} X^{n-2} + \dots + b_1$ est le quotient de la division euclidienne de P par $(X - a)$, et b_0 est le reste : $P(X) = (X - a)Q(X) + b_0$.

Le coût est de n multiplications et n additions, contre $\sim n^2/2$ pour une évaluation naïve.

Démonstration. On procède par identification des coefficients dans $P(X) = (X - a)(b_n X^{n-1} + \dots + b_1) + b_0$. Le coefficient de X^k donne $a_k = b_k - a b_{k+1}$, soit $b_{k+1} = (a_k + b_k - \dots)$. La récurrence ascendante mène aux formules ci-dessus. $\square$

Note historique : Sharaf Eddine Al-Tusi (vers 1135–1213). Mathématicien et astronome persan né à Tus (Iran), Sharaf al-Dīn al-Tūsī a développé dans son traité sur les équations cubiques (*Risāla fī al-mu'ādalāt*) une méthode tabulaire d'évaluation et de division de polynômes équivalente au procédé que William G. Horner publiera en 1819. Sa technique est aussi utilisée pour calculer numériquement les racines d'une équation polynomiale par approximations successives. Cette même méthode était également connue des mathématiciens chinois (Qin Jiushao, XIII^e siècle) et fut redécouverte par l'italien Paolo Ruffini en 1804. La dénomination plus juste serait donc *procédé d'Al-Tusi-Ruffini-Horner*.

5.3.6 PGCD dans $K[X]$

Définition 5.3.3 (Diviseur commun, PGCD). Soient $A, B \in K[X]$ non tous deux nuls. Un *diviseur commun* de A et B est un polynôme D tel que $D \mid A$ et $D \mid B$.

Le **plus grand commun diviseur** (PGCD) de A et B est l'unique diviseur commun unitaire de degré maximal. On le note $A \wedge B$ ou $\text{pgcd}(A, B)$.

Théorème 5.3.4 (Algorithme d'Euclide pour les polynômes). Soient $A, B \in K[X]$ avec $B \neq 0$. La suite des restes obtenue par divisions euclidiennes successives :

$$A = BQ_1 + R_1, \quad B = R_1Q_2 + R_2, \quad R_1 = R_2Q_3 + R_3, \quad \dots$$

avec $\deg R_1 > \deg R_2 > \dots$ se termine en un nombre fini d'étapes par un reste $R_N = 0$. Le dernier reste non nul R_{N-1} , normalisé pour être unitaire, est le PGCD de A et B .

Démonstration. La suite des degrés $\deg R_k$ est strictement décroissante, donc l'algorithme s'arrête. À chaque étape, l'égalité $R_{k-1} = R_k Q_{k+1} + R_{k+1}$ montre que $\{\text{diviseurs communs de } R_{k-1}, R_k\} = \{\text{diviseurs communs de } R_k, R_{k+1}\}$. Par récurrence, $\text{pgcd}(A, B) = \text{pgcd}(R_{N-1}, 0) = R_{N-1}$ (à un scalaire près). $\square$

5.3.7 Identité de Bézout

Théorème 5.3.5 (Bézout pour $K[X]$). Soient $A, B \in K[X]$ non tous deux nuls. Il existe $U, V \in K[X]$ tels que :

$$AU + BV = A \wedge B.$$

Démonstration. Considérons l'ensemble $I = \{AU + BV : U, V \in K[X]\}$. C'est un idéal de $K[X]$ contenant A et B . Comme $K[X]$ est principal (chaque idéal est engendré par un polynôme), il existe $D \in K[X]$ tel que $I = (D)$. Ce D divise A et B , et tout autre diviseur commun divise D : c'est le PGCD (à association près). De plus $D \in I$ s'écrit $D = AU + BV$. $\square$

Corollaire 5.3.6 (Polynômes premiers entre eux). A, B sont **premiers entre eux**, noté $A \wedge B = 1$, si et seulement si :

$$\exists U, V \in K[X], \quad AU + BV = 1.$$

5.3.8 Théorème de Gauss

Théorème 5.3.7 (Gauss). Soient $A, B, C \in K[X]$. Si $A \mid BC$ et $A \wedge B = 1$, alors $A \mid C$.

Démonstration. Bézout : $AU + BV = 1$, donc $C = AUC + BVC$. Comme $A \mid AUC$ et $A \mid BC$ (puisque $A \mid BC$), on a $A \mid C$. $\square$

Corollaire 5.3.8. 1. Si $A \wedge B = 1$ et $A \mid C$ et $B \mid C$, alors $AB \mid C$.

2. Si $A \wedge B_1 = 1$ et $A \wedge B_2 = 1$, alors $A \wedge (B_1 B_2) = 1$.

5.3.9 PPCM

Définition 5.3.4 (PPCM). Le **plus petit commun multiple** de A et B non nuls, noté $A \vee B$, est l'unique polynôme unitaire de degré minimal tel que $A \mid M$ et $B \mid M$.

Proposition 5.3.9 (Lien PGCD/PPCM). Pour A, B unitaires non nuls : $A \cdot B = (A \wedge B) \cdot (A \vee B)$.

5.3.10 Exemples résolus (Difficulté ascendante)

Exemple 1 (Division euclidienne directe — Facile). Effectuer la division euclidienne de $A = X^4 + 2X^3 - X + 1$ par $B = X^2 + X - 1$ dans $\mathbb{R}[X]$.

Correction : on procède en colonnes. $X^4 \div X^2 = X^2$; $X^2 \cdot B = X^4 + X^3 - X^2$. Reste partiel : $A - X^2 B = X^3 + X^2 - X + 1$. $X^3 \div X^2 = X$; $X \cdot B = X^3 + X^2 - X$. Reste : 1. Le quotient est $Q = X^2 + X$ et le reste est $R = 1$. Vérification : $(X^2 + X)(X^2 + X - 1) + 1 = X^4 + 2X^3 - X + 1$. $\checkmark$

Exemple 2 (Hörner — Intermédiaire). Évaluer $P = 3X^4 - 2X^3 + 5X - 7$ en $a = 2$, et donner le quotient de P par $(X - 2)$.

Correction : on tabule les coefficients $a_4 = 3$, $a_3 = -2$, $a_2 = 0$, $a_1 = 5$, $a_0 = -7$.

	a_4	a_3	a_2	a_1	a_0
	3	-2	0	5	-7
$\times 2 :$		6	8	16	42
$b_k :$	3	4	8	21	35

Donc $P(2) = 35$ et $Q(X) = 3X^3 + 4X^2 + 8X + 21$, c'est-à-dire $P(X) = (X - 2)(3X^3 + 4X^2 + 8X + 21) + 35$.

Exemple 3 (Algorithme d'Euclide étendu — Difficile). Calculer $A \wedge B$ pour $A = X^4 + X^3 - X - 1$ et $B = X^3 - 1$ dans $\mathbb{R}[X]$, puis trouver une identité de Bézout.

Correction : divisions successives.

$$A = X^4 + X^3 - X - 1 = (X+1)(X^3-1) + 0 \cdot X^2 + 0 \cdot X + 0 \quad ? \text{ Vérifions : } (X+1)(X^3-1) = X^4 - X + X^3 - 1 =$$

Donc $A = (X+1)B + 0$, le reste est nul, et $A \wedge B =$ partie unitaire de $B = X^3 - 1$.

Identité de Bézout : $A \cdot 1 + B \cdot (-(X+1)) = 0$ ne fonctionne pas. Comme $B \mid A$, le PGCD est B (unitaire) et l'on peut écrire $B = A \cdot 0 + B \cdot 1$, soit $U = 0, V = 1$.

Variante. Pour $A = X^3 + 1$ et $B = X^2 + X$:

$$A = B \cdot X + (-X^2 + 1) \quad (\text{reste } R_1 = -X^2 + 1)$$

$$B = R_1 \cdot (-1) + (X + 1) \quad (\text{reste } R_2 = X + 1)$$

$$R_1 = R_2 \cdot (-X + 1) + 0$$

Donc $A \wedge B = X + 1$ (déjà unitaire). En remontant les égalités : $X + 1 = B + R_1 = B + (A - XB) = A - (X - 1)B$. Soit $U = 1$ et $V = -(X - 1) = 1 - X$: $A \cdot 1 + B \cdot (1 - X) = X + 1$.

5.3.11 Exercices pratiques avec Python

sympy fournit `div`, `gcd`, `gcdex` (Bézout étendu) sur les polynômes.

```

1 import sympy as sp
2
3 X = sp.symbols('X')
4
5 # 1. Division euclidienne
6 A = sp.Poly(X**4 + 2*X**3 - X + 1, X)
7 B = sp.Poly(X**2 + X - 1, X)
8 Q, R = sp.div(A, B, X)
9 print(f"A = B*Q + R : {Q.as_expr()}, {R.as_expr()}")
10
11 # 2. Procédé de Horner (évaluation + division par (X - a))
12 def horner(coeffs, a):
13     """coeffs : ordre décroissant des coefficients. Retourne (P(a),
14         coeffs du quotient)."""
15     bs = []
16     b = 0
17     for c in coeffs:
18         b = b * a + c
19         bs.append(b)
20     return bs[-1], bs[:-1] # (P(a), coefficients du quotient en
21                             ordre décroissant)
22
23 Pa, coeffs_Q = horner([3, -2, 0, 5, -7], 2)
24 print(f"\nHorner sur P=3X^4-2X^3+5X-7 en a=2 :")
25 print(f"P(2) = {Pa}")
26 print(f"P(X) coefficients (ordre décroissant) = {coeffs_Q}")
27
28 # 3. Algorithme d'Euclide étendu : Bezout
29 A = sp.Poly(X**3 + 1, X)
30 B = sp.Poly(X**2 + X, X)
31 g = sp.gcd(A, B)

```

```

30 U, V, d = sp.gcdex(A.as_expr(), B.as_expr(), X)
31 print(f"\nA={A.as_expr()}, B={B.as_expr()}")
32 print(f"\nPGCD={g.as_expr()}")
33 print(f"\nBezout: ({U}) * A + ({V}) * B = {sp.expand(U*A.as_expr() + V*B.as_expr())}")
34
35 # 4. PPCM
36 print(f"\nPPCM(A, B) = {sp.lcm(A, B).as_expr()}")
37 print(f"Verification A*B / pgcd = {sp.expand(A.as_expr() * B.as_expr() / g.as_expr())}")

```

5.3.12 Exercices à faire

- Exercice 1.** Effectuer la division euclidienne de $A = X^5 - X + 1$ par $B = X^2 + 1$ dans $\mathbb{R}[X]$.
- Exercice 2.** Évaluer $P = 2X^4 - 3X^3 + X^2 - 5X + 6$ en $a = -1$ par le procédé de Hörner. En déduire le quotient de P par $(X + 1)$ et $P(-1)$.
- Exercice 3.** Calculer $A \wedge B$ pour $A = X^5 + X^4 + X^3 + X^2 + X + 1$ et $B = X^4 - 1$. Donner une identité de Bézout.
- Exercice 4.** Démontrer que pour tous $m, n \in \mathbb{N}^*$, $\text{pgcd}(X^m - 1, X^n - 1) = X^{\text{pgcd}(m, n)} - 1$ dans $K[X]$. (Indication : utiliser la division euclidienne sur les exposants, à l'image du calcul du PGCD entier.)
- Exercice 5 (synthèse).** Soit $A, B, C \in K[X]$. Montrer que si $A \wedge B = 1$ et $A \mid C^k$ pour un certain $k \geq 1$, et si A est irréductible, alors $A \mid C$. (Variante du théorème de Gauss — démontrée en §4 dans le cas général.)

5.4 Polynômes irréductibles, décomposition, relations racines-coefficients

5.4.1 Motivation

Dans $\mathbb{Z}$, les nombres premiers sont les briques élémentaires : tout entier se décompose de manière unique en facteurs premiers. Dans $K[X]$, ce rôle est joué par les **polynômes irréductibles**, et la décomposition unique en irréductibles permet de comprendre la structure complète d'un polynôme : ses racines, ses facteurs réels, ses simplifications. Les polynômes irréductibles sur les corps finis $\mathbb{F}_p$ servent à construire les corps $\mathbb{F}_{p^n}$, fondement des codes correcteurs et de la cryptographie sur courbes elliptiques. Pour finir, les *relations racines-coefficients* (formules de Viète) donnent un dictionnaire entre les coefficients d'un polynôme et les fonctions symétriques de ses racines.

5.4.2 Approche par problème : où $X^2 + 1$ est-il irréductible ?

Problème : Le polynôme $P = X^2 + 1$ est-il irréductible dans $\mathbb{Q}[X]$? dans $\mathbb{R}[X]$? dans $\mathbb{C}[X]$?

Résolution :

— Sur $\mathbb{C}$: $X^2 + 1 = (X - i)(X + i)$. Réductible.

- Sur $\mathbb{R}$: aucune racine réelle (puisque $x^2 + 1 \geq 1$). Si $P = AB$ avec $A, B \in \mathbb{R}[X]$ non constants, alors $\deg A = \deg B = 1$, donc P aurait deux racines réelles — contradiction. *Irréductible*.
- Sur $\mathbb{Q}$: aucune racine rationnelle (a fortiori, pas de racine réelle). Le même argument donne l'irréductibilité. *Irréductible*.

L'irréductibilité *dépend du corps de base*. Cette section caractérise les irréductibles dans $\mathbb{C}[X]$ et $\mathbb{R}[X]$.

5.4.3 Polynômes irréductibles

Définition 5.4.1 (Polynôme irréductible). Un polynôme $P \in K[X]$ est **irréductible** sur K si :

1. $\deg P \geq 1$ (on exclut les constantes) ;
2. dans toute factorisation $P = A \cdot B$, l'un des facteurs A ou B est inversible dans $K[X]$, c'est-à-dire constant non nul.

Un polynôme non constant qui n'est pas irréductible est dit **réductible**.

Exemple 5.4.1. — Tout polynôme de degré 1 est irréductible (sur n'importe quel corps).

- $X^2 - 2$ est irréductible sur $\mathbb{Q}$ mais réductible sur $\mathbb{R}$ ($X^2 - 2 = (X - \sqrt{2})(X + \sqrt{2})$).
- $X^2 + 1$ est irréductible sur $\mathbb{R}$ mais réductible sur $\mathbb{C}$.
- Sur $\mathbb{F}_2$, le polynôme $X^2 + X + 1$ est irréductible (pas de racine dans $\mathbb{F}_2$). Il sert à construire $\mathbb{F}_4$.

5.4.4 Décomposition unique en facteurs irréductibles

Théorème 5.4.1 (Théorème fondamental de l'arithmétique dans $K[X]$). *Tout polynôme non constant $P \in K[X]$ se factorise sous la forme*

$$P = c \cdot P_1^{m_1} \cdot P_2^{m_2} \cdots P_r^{m_r},$$

où $c \in K^*$ et les P_i sont des polynômes irréductibles unitaires deux à deux distincts. Cette décomposition est unique à l'ordre près des facteurs.

Esquisse. Existence : par récurrence sur $\deg P$. Si P est irréductible, c'est terminé. Sinon $P = AB$ avec $\deg A, \deg B < \deg P$, et chaque facteur se décompose par récurrence.

Unicité : utilise le théorème de Gauss. Si $P_1 \cdots P_r = Q_1 \cdots Q_s$ avec tous les facteurs irréductibles unitaires, alors P_1 divise $Q_1 \cdots Q_s$. Comme P_1 est irréductible et premier avec tout Q_j qui ne lui est pas associé, Gauss force P_1 à être associé à un certain Q_j . On simplifie et l'on conclut par récurrence. $\square$

5.4.5 Irréductibles de $\mathbb{C}[X]$

Théorème 5.4.2 (Théorème fondamental de l'algèbre, admis). *Tout polynôme non constant de $\mathbb{C}[X]$ admet au moins une racine dans $\mathbb{C}$. Autrement dit, $\mathbb{C}$ est **algébriquement clos**.*

Remarque 5.4.1. Ce théorème, conjecturé par d'Alembert et démontré rigoureusement par Gauss en 1799 (sa thèse), nécessite des outils d'analyse. Il sera démontré en cours de fonctions complexes.

Corollaire 5.4.3 (Irréductibles de $\mathbb{C}[X]$). *Les polynômes irréductibles de $\mathbb{C}[X]$ sont exactement ceux de degré 1.*

Corollaire 5.4.4 (Décomposition dans $\mathbb{C}[X]$). *Tout $P \in \mathbb{C}[X]$ de degré $n \geq 1$ et de coefficient dominant a_n se factorise de manière unique :*

$$P = a_n \prod_{k=1}^r (X - z_k)^{m_k}, \quad \sum_{k=1}^r m_k = n,$$

où les $z_k \in \mathbb{C}$ sont les racines distinctes de P et m_k leurs multiplicités.

5.4.6 Irréductibles de $\mathbb{R}[X]$

Théorème 5.4.5 (Irréductibles de $\mathbb{R}[X]$). *Les polynômes irréductibles de $\mathbb{R}[X]$ sont :*

- les polynômes de degré 1 ;
- les polynômes de degré 2 à discriminant strictement négatif, c'est-à-dire de la forme $aX^2 + bX + c$ avec $a \neq 0$ et $b^2 - 4ac < 0$.

Démonstration. Les polynômes de degré 1 sont irréductibles. Pour le degré 2 : un trinôme $aX^2 + bX + c$ a une racine réelle ssi $b^2 - 4ac \geq 0$. S'il en a une, il se factorise en $a(X - r_1)(X - r_2)$, donc réductible. Sinon, ses deux racines complexes sont conjuguées et il est irréductible sur $\mathbb{R}$.

Réciproquement, soit $P \in \mathbb{R}[X]$ de degré ≥ 3 . Sur $\mathbb{C}$, il admet une racine z . Si $z \in \mathbb{R}$, alors $(X - z) \mid P$ dans $\mathbb{R}[X]$, donc P est réductible. Sinon, $\bar{z}$ est aussi racine (conjugaison), et $(X - z)(X - \bar{z}) = X^2 - 2\operatorname{Re}(z)X + |z|^2 \in \mathbb{R}[X]$ divise P dans $\mathbb{R}[X]$; encore réductible. $\square$

Corollaire 5.4.6 (Décomposition dans $\mathbb{R}[X]$). *Tout $P \in \mathbb{R}[X]$ de degré $n \geq 1$ se factorise de manière unique sous la forme*

$$P = a_n \prod_{i=1}^p (X - a_i)^{\alpha_i} \prod_{j=1}^q (X^2 + b_jX + c_j)^{\beta_j},$$

où les $a_i \in \mathbb{R}$ sont les racines réelles, α_i leurs multiplicités, et les facteurs $X^2 + b_jX + c_j$ sont irréductibles ($b_j^2 - 4c_j < 0$).

5.4.7 Relations entre coefficients et racines (formules de Viète)

Soient $z_1, \dots, z_n$ des indéterminées. Les **polynômes symétriques élémentaires** sont définis par :

$$\begin{aligned} \sigma_1 &= z_1 + z_2 + \dots + z_n, \\ \sigma_2 &= \sum_{i < j} z_i z_j, \\ \sigma_k &= \sum_{i_1 < i_2 < \dots < i_k} z_{i_1} z_{i_2} \dots z_{i_k}, \\ \sigma_n &= z_1 z_2 \dots z_n. \end{aligned}$$

Théorème 5.4.7 (Formules de Viète). *Soit $P = a_n X^n + a_{n-1} X^{n-1} + \dots + a_0 \in K[X]$ un polynôme scindé sur K , c'est-à-dire admettant n racines $z_1, \dots, z_n \in K$ (comptées avec multiplicité). Alors :*

$$\boxed{\sigma_k(z_1, \dots, z_n) = (-1)^k \frac{a_{n-k}}{a_n}} \quad \text{pour } k = 1, 2, \dots, n.$$

Démonstration. On part de $P = a_n(X - z_1)(X - z_2) \cdots (X - z_n)$ et l'on développe. Le coefficient de X^{n-k} dans le produit est $(-1)^k \sigma_k$ multiplié par a_n , soit $a_{n-k} = a_n \cdot (-1)^k \sigma_k$. $\square$

Remarque 5.4.2 (Cas usuels). Pour le degré 2 ($P = aX^2 + bX + c$ de racines z_1, z_2) :

$$z_1 + z_2 = -\frac{b}{a}, \quad z_1 z_2 = \frac{c}{a}.$$

Pour le degré 3 ($P = X^3 + pX^2 + qX + r$ unitaire de racines z_1, z_2, z_3) :

$$z_1 + z_2 + z_3 = -p, \quad z_1 z_2 + z_1 z_3 + z_2 z_3 = q, \quad z_1 z_2 z_3 = -r.$$

Remarque 5.4.3. Sur $\mathbb{C}$, tout polynôme est scindé (par le théorème fondamental). Les formules de Viète s'appliquent donc toujours dans $\mathbb{C}[X]$.

5.4.8 Exemples résolus (Difficulté ascendante)

Exemple 1 (Décomposition dans $\mathbb{R}$ et $\mathbb{C}$ — Facile). Décomposer $P = X^4 - 1$ dans $\mathbb{R}[X]$ et dans $\mathbb{C}[X]$.

Correction : $P = (X^2 - 1)(X^2 + 1) = (X - 1)(X + 1)(X^2 + 1)$ dans $\mathbb{R}[X]$: $X^2 + 1$ est irréductible sur $\mathbb{R}$ (discriminant $-4 < 0$). Sur $\mathbb{C}$: $X^2 + 1 = (X - i)(X + i)$, d'où $P = (X - 1)(X + 1)(X - i)(X + i)$. Quatre facteurs de degré 1.

Exemple 2 (Application de Viète — Intermédiaire). Soient z_1, z_2, z_3 les racines complexes de $P = X^3 - 6X^2 + 11X - 6$. Calculer $z_1 + z_2 + z_3$, $z_1 z_2 z_3$, et $z_1^2 + z_2^2 + z_3^2$.

Correction : Viète donne directement $\sigma_1 = z_1 + z_2 + z_3 = 6$ et $\sigma_3 = z_1 z_2 z_3 = 6$, ainsi que $\sigma_2 = z_1 z_2 + z_1 z_3 + z_2 z_3 = 11$. Pour la somme des carrés, on utilise $z_1^2 + z_2^2 + z_3^2 = \sigma_1^2 - 2\sigma_2 = 36 - 22 = 14$.

(Vérification : on peut factoriser $P = (X - 1)(X - 2)(X - 3)$, donc les racines sont 1, 2, 3 avec $1 + 4 + 9 = 14$. $\checkmark$)

Exemple 3 (Décomposition réelle complète — Difficile). Décomposer $P = X^6 - 1$ dans $\mathbb{R}[X]$ et dans $\mathbb{C}[X]$.

Correction : sur $\mathbb{C}$, les racines sont les racines 6-ièmes de l'unité : $\omega^k = e^{2ik\pi/6}$ pour $k = 0, 1, \dots, 5$. Donc

$$X^6 - 1 = \prod_{k=0}^5 (X - e^{ik\pi/3}).$$

On les liste : 1, $e^{i\pi/3}$, $e^{2i\pi/3}$, -1 , $e^{4i\pi/3}$, $e^{5i\pi/3}$. Les conjuguées s'apparient : $(e^{i\pi/3}, e^{5i\pi/3})$ et $(e^{2i\pi/3}, e^{4i\pi/3})$. Sur $\mathbb{R}$, on regroupe les conjuguées :

$$\begin{aligned} (X - e^{i\pi/3})(X - e^{-i\pi/3}) &= X^2 - 2\cos(\pi/3)X + 1 = X^2 - X + 1, \\ (X - e^{2i\pi/3})(X - e^{-2i\pi/3}) &= X^2 - 2\cos(2\pi/3)X + 1 = X^2 + X + 1. \end{aligned}$$

D'où :

$$X^6 - 1 = (X - 1)(X + 1)(X^2 - X + 1)(X^2 + X + 1).$$

On vérifie que les deux trinômes ont des discriminants $1 - 4 = -3 < 0$: ils sont bien irréductibles sur $\mathbb{R}$.

5.4.9 Exercices pratiques avec Python

`sympy.factor` effectue la factorisation symbolique sur $\mathbb{Q}$, et accepte des extensions algébriques pour factoriser sur $\mathbb{R}$ ou $\mathbb{C}$. Les formules de Viète se vérifient via `sympy.elementary_symmetric`

```

1 import sympy as sp
2 from sympy import I, sqrt, factor, roots, expand
3
4 X = sp.symbols('X')
5
6 # 1. Décomposition selon le corps de base
7 P = X**6 - 1
8 print(f"X^6_{-1}_{sur}_{Q}_{:}_{factor}(P)")
9 print(f"_{sur}_{Q}(sqrt(3))_{:}_{factor}(P,extension=[sqrt(3)])")
10 print(f"_{sur}_{C}_{:}_{factor}(P,extension=[I,sqrt(3)])")
11
12 # 2. Verification des formules de Viète
13 P = X**3 - 6*X**2 + 11*X - 6
14 zs = list(roots(P).keys())
15 print(f"\nRacines_{de}_{P}_{:}_{zs}")
16 print(f"_{sigma}_1_{:}_{sum(zs)}_{:}_{(attendu_{:}_{6})}")
17 print(f"_{sigma}_2_{:}_{sum(z1*z2_{for}_{i}_{z1}_{in}_{enumerate(zs)}_{for}_{z2}_{in}_{zs[i+1:]})}_{:}_{(attendu_{:}_{11})}")
18 print(f"_{sigma}_3_{:}_{sp.prod(zs)}_{:}_{(attendu_{:}_{6})}")
19
20 # 3. Somme des carres a partir de sigma_1, sigma_2 (sans calculer
    les racines)
21 P = X**3 - 6*X**2 + 11*X - 6
22 coeffs = sp.Poly(P, X).all_coeffs()
23 a3, a2, a1, a0 = coeffs
24 sigma1 = -a2 / a3
25 sigma2 = a1 / a3
26 somme_carres = sigma1**2 - 2 * sigma2
27 print(f"\nz1^2_{+}_{z2^2}_{+}_{z3^2}_{=}_{sigma}_1^2_{-}_{2}_{sigma}_2_{=}_{somme_carres}")
28
29 # 4. Test d'irréductibilité sur Q (sans factoriser explicitement)
30 for poly in [X**2 + 1, X**2 - 2, X**3 - 2, X**4 + 1, X**2 + X + 1]:
31     print(f"{poly}_{irréductible}_{sur}_{Q}_{?}_{sp.Poly(poly,X).is_irreducible}")

```

5.4.10 Exercices à faire

- Exercice 1.** Décomposer $P = X^4 + 1$ dans $\mathbb{Q}[X]$, $\mathbb{R}[X]$ et $\mathbb{C}[X]$. Vérifier que sur $\mathbb{Q}$, P est irréductible (par exemple par le critère d'Eisenstein appliqué à $P(X+1)$, ou par considération de degré).
- Exercice 2.** Soient a, b, c les racines complexes de $P = X^3 + 2X^2 - X + 5$. Calculer $a+b+c$, $ab+ac+bc$, abc , puis $a^2+b^2+c^2$ et $\frac{1}{a} + \frac{1}{b} + \frac{1}{c}$ (cette dernière sans expliciter les racines).
- Exercice 3.** Trouver un polynôme unitaire $P \in \mathbb{R}[X]$ de degré 3 dont les racines sont 2 , $1+i$ et $1-i$. Vérifier les formules de Viète.

4. **Exercice 4.** Décomposer $P = X^4 + X^2 + 1$ dans $\mathbb{R}[X]$. (Indication : remarquer que $P = (X^2 + X + 1)(X^2 - X + 1)$ par identité algébrique ; vérifier en développant.)
5. **Exercice 5 (synthèse).** Soit $P \in \mathbb{R}[X]$ unitaire de degré n , scindé sur $\mathbb{R}$ avec racines réelles $a_1 \leq a_2 \leq \dots \leq a_n$. Démontrer que $\frac{P'}{P} = \sum_{i=1}^n \frac{1}{X-a_i}$ (en tant qu'identité de fractions rationnelles — voir chapitre VI).

Travaux Pratiques (TP) Python

L'objectif de ce TP est de construire pas à pas une bibliothèque *maison* de manipulation de polynômes : représentation, arithmétique, racines, factorisation. Ce travail prépare à la décomposition en éléments simples (chapitre VI) et à des applications concrètes en codes correcteurs et en cryptographie. On comparera systématiquement à `numpy.polynomial` et `sympy`.

Exercice 1 : Classe Polynome

1. **[Bloom : Compréhension | Difficulté : Moyenne]**
Créer une classe `Polynome` dont l'attribut principal est la liste des coefficients en ordre croissant (`[a0, a1, ..., an]`). Implémenter :
— `__init__(self, coeffs)` avec normalisation (suppression des zéros de tête) ;
— `degre(self)` ;
— `__repr__(self)` pour afficher joliment le polynôme.
2. **[Bloom : Application | Difficulté : Moyenne]**
Surcharger les opérateurs `__add__`, `__sub__`, `__mul__` pour la somme, la différence et le produit (produit de Cauchy).
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester votre classe sur $P = 2X^3 - X + 5$ et $Q = X^2 + 3$: vérifier $\deg(PQ) = \deg P + \deg Q$.

Exercice 2 : Évaluation par le procédé de Hörner

1. **[Bloom : Application | Difficulté : Moyenne]**
Ajouter une méthode `evaluer(self, a)` qui calcule $P(a)$ par le procédé de Hörner (en n multiplications et n additions).
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Comparer le temps d'exécution de `evaluer_Horner` et d'une évaluation naïve $\sum a_k a^k$ (avec puissances explicites) pour un polynôme de degré 1000 évalué 10000 fois. Conclusion ?
3. **[Bloom : Synthèse | Difficulté : Moyenne]**
Modifier le procédé pour qu'il retourne en plus le quotient Q tel que $P = (X - a)Q + P(a)$ (division par $(X - a)$ par Hörner).

Exercice 3 : Division euclidienne dans $K[X]$

1. **[Bloom : Synthèse | Difficulté : Difficile]**
Implémenter `divmod_poly(A, B)` qui retourne le couple (Q, R) de la division euclidienne $A = BQ + R$ avec $\deg R < \deg B$. Suivre la démonstration du cours : éliminer pas à pas le terme dominant.

2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier sur $A = X^4 + 2X^3 - X + 1$ et $B = X^2 + X - 1$ que $Q = X^2 + X$ et $R = 1$.
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Comparer avec `sympy.div(A, B, X)`.

Exercice 4 : PGCD de polynômes (algorithme d'Euclide)

1. **[Bloom : Application | Difficulté : Moyenne]**
En utilisant `divmod_poly`, écrire `pgcd_poly(A, B)` qui retourne le PGCD de A et B (unitaire) par divisions successives.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester `pgcd( $X^4 - 1, X^3 - X$ ) =  $X^2 - 1$` .
3. **[Bloom : Synthèse | Difficulté : Difficile]**
Vérifier numériquement la propriété :

$$\text{pgcd}(X^m - 1, X^n - 1) = X^{\text{pgcd}(m,n)} - 1$$

pour 10 couples (m, n) aléatoires avec $m, n \leq 20$.

Exercice 5 : Bézout étendu pour polynômes

1. **[Bloom : Synthèse | Difficulté : Difficile]**
Étendre l'algorithme d'Euclide en `bezout_poly(A, B)` qui retourne (D, U, V) tel que $D = A \wedge B$ et $AU + BV = D$. (Adapter l'algorithme d'Euclide étendu de l'arithmétique entière.)
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester sur $A = X^3 + 1$ et $B = X^2 + X$: vérifier que $A \cdot 1 + B \cdot (1 - X) = X + 1$ (le PGCD).
3. **[Bloom : Évaluation | Difficulté : Moyenne]**
Comparer avec `sympy.gcdex(A, B, X)`.

Exercice 6 : Recherche de racines rationnelles

1. **[Bloom : Compréhension | Difficulté : Moyenne]**
Théorème des racines rationnelles : si $P = a_n X^n + \dots + a_0 \in \mathbb{Z}[X]$ admet une racine rationnelle p/q écrite sous forme irréductible, alors $p \mid a_0$ et $q \mid a_n$. Écrire `racines_rationnelles(P)` qui retourne la liste de toutes les racines rationnelles d'un polynôme à coefficients entiers, en énumérant les couples (p, q) candidats.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Tester sur $P = 6X^3 - 11X^2 + 6X - 1$ (racines $1, 1/2, 1/3$).
3. **[Bloom : Création | Difficulté : Difficile]**
Améliorer en factorisant P par chaque racine trouvée (Hörner) et en répétant la recherche sur le quotient.

Exercice 7 : Multiplicité d'une racine par dérivation

1. **[Bloom : Application | Difficulté : Moyenne]**
Ajouter à votre classe `Polynome` la méthode `derivee(self)` qui retourne P' (formule formelle).

2. **[Bloom : Synthèse | Difficulté : Moyenne]**

Écrire `multiplicite(P, a)` qui retourne la multiplicité de a comme racine de P , en utilisant la caractérisation $P(a) = P'(a) = \dots = P^{(m-1)}(a) = 0$ et $P^{(m)}(a) \neq 0$.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

Tester sur $P = (X - 1)^3(X + 2)^2(X - 5)$ développé : vérifier $\text{mult}_1 = 3$, $\text{mult}_{-2} = 2$, $\text{mult}_5 = 1$.

Exercice 8 : Application — Codes correcteurs Reed-Solomon (encodage)

1. **[Bloom : Compréhension | Difficulté : Difficile]**

Le principe simple de Reed-Solomon : on encode un message $(m_0, m_1, \dots, m_{k-1}) \in \mathbb{F}_q^k$ comme le polynôme $M(X) = \sum m_i X^i$, puis on l'évalue en n points distincts $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ (avec $n > k$). Le mot de code est $(M(\alpha_1), \dots, M(\alpha_n))$.

Écrire `encoder_RS(message, points)` en travaillant dans $\mathbb{Z}/p\mathbb{Z}$ avec p premier (par exemple $p = 11$).

2. **[Bloom : Évaluation | Difficulté : Moyenne]**

Encoder le message $(3, 1, 4)$ avec les points d'évaluation $\{1, 2, 3, 4, 5\}$ dans $\mathbb{F}_{11}$. Donner le mot de code obtenu.

3. **[Bloom : Synthèse | Difficulté : Difficile]**

Décodage simple (sans erreur) : étant donné un mot de code $(y_1, \dots, y_n)$ supposé sans erreur et les points $(\alpha_1, \dots, \alpha_n)$, on retrouve le polynôme M par interpolation de Lagrange. Implémenter `decoder_RS_sans_erreur(mot, points, k)` qui retourne le message original.

Chapitre 6

Fractions rationnelles

Ce chapitre construit le corps $K(X)$ des fractions rationnelles, puis en étudie l'arithmétique fine : forme irréductible, pôles, zéros, multiplicités, et enfin la *décomposition en éléments simples*, outil central pour le calcul intégral, le calcul de séries télescopiques, le filtrage en traitement du signal et l'analyse des systèmes linéaires en automatique. Ce chapitre est l'aboutissement naturel du chapitre V : on quitte l'anneau $K[X]$ pour son *corps des fractions*, par analogie au passage de $\mathbb{Z}$ à $\mathbb{Q}$.

6.1 Corps des fractions $K(X)$

6.1.1 Motivation

L'anneau $K[X]$ est intègre mais n'est *pas* un corps : seuls les polynômes constants non nuls sont inversibles. On ne peut pas diviser librement par n'importe quel polynôme non nul. C'est exactement la situation rencontrée dans $\mathbb{Z}$: pour pouvoir diviser, on a construit $\mathbb{Q}$, le corps des fractions de $\mathbb{Z}$. La même construction appliquée à $K[X]$ donne le corps $K(X)$ des **fractions rationnelles**, où l'expression $1/(X^2+1)$ devient un objet algébrique parfaitement défini.

Le passage à $K(X)$ ouvre la porte au calcul des primitives de fractions rationnelles, à l'analyse des transformées de Laplace en automatique, et plus généralement à toute l'analyse réelle et complexe d'expressions algébriques.

6.1.2 Approche par problème : à quelles conditions $A/B = C/D$?

Problème : Quand peut-on dire que $\frac{X-1}{X^2-1}$ et $\frac{1}{X+1}$ représentent la même fraction rationnelle ? Plus généralement, quelle relation $\sim$ identifier sur les couples (A, B) avec $B \neq 0$ pour obtenir un cadre algébrique cohérent ?

Résolution : Dans $\mathbb{Q}$, $\frac{a}{b} = \frac{c}{d} \iff ad = bc$. La même règle s'applique aux polynômes :

$$\frac{X-1}{X^2-1} = \frac{1}{X+1} \iff (X-1)(X+1) = (X^2-1) \cdot 1,$$

ce qui est vrai. Donc on définit $(A, B) \sim (C, D) \iff AD = BC$ et l'on note A/B la classe d'équivalence de (A, B) . Cette classe d'équivalence est notre *fraction rationnelle*.

6.1.3 Construction de $K(X)$

Définition 6.1.1 (Fraction rationnelle, ensemble $K(X)$). Sur l'ensemble $K[X] \times (K[X] \setminus \{0\})$, on définit la relation $\sim$ par :

$$(A, B) \sim (C, D) \iff AD = BC.$$

C'est une relation d'équivalence (vérification directe, exploitant l'intégrité de $K[X]$). L'ensemble **des fractions rationnelles** à coefficients dans K est l'ensemble quotient

$$K(X) = (K[X] \times (K[X] \setminus \{0\})) / \sim,$$

et l'on note la classe de (A, B) par $\frac{A}{B}$.

Remarque 6.1.1. La condition $B \neq 0$ est essentielle : on ne peut pas diviser par le polynôme nul, exactement comme dans $\mathbb{Q}$.

6.1.4 Opérations sur $K(X)$

Définition 6.1.2 (Addition, multiplication, opposé, inverse). Soient $\frac{A}{B}$ et $\frac{C}{D}$ deux fractions de $K(X)$. On pose :

$$\begin{aligned} \frac{A}{B} + \frac{C}{D} &= \frac{AD + BC}{BD}, \\ \frac{A}{B} \cdot \frac{C}{D} &= \frac{AC}{BD}, \\ -\frac{A}{B} &= \frac{-A}{B}. \end{aligned}$$

Si de plus $A \neq 0$, on définit l'inverse multiplicatif :

$$\left(\frac{A}{B}\right)^{-1} = \frac{B}{A}.$$

Proposition 6.1.1 (Bonne définition). *Les opérations ci-dessus sont indépendantes du choix des représentants des classes : si $\frac{A}{B} = \frac{A'}{B'}$ et $\frac{C}{D} = \frac{C'}{D'}$, alors $\frac{AD+BC}{BD} = \frac{A'D'+B'C'}{B'D'}$ et de même pour le produit.*

Démonstration. Vérification par calcul direct utilisant les égalités $AB' = A'B$ et $CD' = C'D$ et l'intégrité de $K[X]$. $\square$

6.1.5 Structure de corps

Théorème 6.1.2 ($K(X)$ est un corps commutatif). $(K(X), +, \times)$ est un corps commutatif :

- l'élément neutre additif est $\frac{0}{1}$, noté 0 ;
- l'élément neutre multiplicatif est $\frac{1}{1}$, noté 1 ;
- toute fraction non nulle $\frac{A}{B}$ admet pour inverse $\frac{B}{A}$.

Démonstration. Les axiomes d'anneau commutatif (associativité, commutativité, distributivité) se vérifient directement par calcul sur les représentants. L'inverse de $\frac{A}{B}$ est $\frac{B}{A}$ lorsque $A \neq 0$, car

$$\frac{A}{B} \cdot \frac{B}{A} = \frac{AB}{BA} = \frac{AB}{AB} = \frac{1}{1} = 1.$$

□

6.1.6 Inclusion $K[X] \hookrightarrow K(X)$

Proposition 6.1.3 (Plongement de $K[X]$ dans $K(X)$). *L'application $j : K[X] \rightarrow K(X)$, $P \mapsto \frac{P}{1}$ est un morphisme d'anneaux injectif. On identifie $K[X]$ à son image dans $K(X)$: tout polynôme est une fraction rationnelle particulière (de dénominateur 1).*

Démonstration. j est un morphisme : $j(P + Q) = \frac{P+Q}{1} = \frac{P}{1} + \frac{Q}{1} = j(P) + j(Q)$; idem pour le produit ; $j(1) = 1$. Injectivité : $j(P) = 0 \iff \frac{P}{1} = \frac{0}{1} \iff P = 0$. □

Remarque 6.1.2 (Universalité du corps des fractions). $K(X)$ est le *plus petit* corps contenant $K[X]$: si L est un corps et $\varphi : K[X] \rightarrow L$ un morphisme injectif d'anneaux, alors φ se prolonge de manière unique en un morphisme injectif $K(X) \rightarrow L$. Cette propriété universelle caractérise $K(X)$ à isomorphisme près.

6.1.7 Cas particuliers : $\mathbb{R}(X)$ et $\mathbb{C}(X)$

- $\mathbb{R}(X)$ est le corps des fractions rationnelles à coefficients réels.
- $\mathbb{C}(X)$ est le corps des fractions rationnelles à coefficients complexes.
- L'inclusion $\mathbb{R}[X] \subset \mathbb{C}[X]$ induit $\mathbb{R}(X) \subset \mathbb{C}(X)$.
- La conjugaison complexe se prolonge à $\mathbb{C}(X)$: si $F = A/B$, on pose $\bar{F} = \bar{A}/\bar{B}$; c'est un automorphisme de corps de $\mathbb{C}(X)$, dont les invariants sont exactement $\mathbb{R}(X)$.

6.1.8 Fonction rationnelle associée

Définition 6.1.3 (Fonction rationnelle). Soit $F = A/B \in K(X)$. La **fonction rationnelle** associée est l'application

$$\tilde{F} : K \setminus Z(B) \longrightarrow K, \quad x \longmapsto \frac{A(x)}{B(x)},$$

où $Z(B) = \{x \in K : B(x) = 0\}$ est l'ensemble des racines (zéros) de B .

Remarque 6.1.3. Un même $F \in K(X)$ peut s'écrire avec différents représentants A/B . Le domaine de définition de $\tilde{F}$ peut donc varier selon l'écriture choisie. C'est pourquoi on travaille avec la *forme irréductible* (§2), où le domaine est uniquement déterminé par la fraction rationnelle.

6.1.9 Exemples résolus (Difficulté ascendante)

Exemple 1 (Simplification — Facile). Vérifier que $\frac{X^2 - 1}{X^2 - 2X + 1} = \frac{X + 1}{X - 1}$ dans $\mathbb{R}(X)$.

Correction : on factorise numérateur et dénominateur : $X^2 - 1 = (X - 1)(X + 1)$ et

$$X^2 - 2X + 1 = (X - 1)^2 \cdot \frac{(X - 1)(X + 1)}{(X - 1)^2} = \frac{X + 1}{X - 1} \cdot \checkmark$$

(Vérification par produit en croix : $(X^2 - 1)(X - 1) = (X - 1)(X + 1)(X - 1) = (X^2 - 2X + 1)(X + 1)$. ✓)

Exemple 2 (Addition de fractions — Intermédiaire). Calculer $\frac{1}{X} + \frac{1}{X+1} - \frac{1}{X^2+X}$.

Correction : le dénominateur commun est $X(X+1) = X^2 + X$.

$$\frac{1}{X} + \frac{1}{X+1} - \frac{1}{X^2+X} = \frac{(X+1)+X-1}{X(X+1)} = \frac{2X}{X(X+1)} = \frac{2}{X+1}.$$

Exemple 3 (Inverse et calcul — Difficile). Soit $F = \frac{X^2 + 1}{X - 2}$. Calculer $F + F^{-1}$ et donner le résultat sous forme A/B avec $A, B \in \mathbb{R}[X]$.

$$\text{Correction : } F^{-1} = \frac{X-2}{X^2+1}.$$

$$F + F^{-1} = \frac{X^2 + 1}{X - 2} + \frac{X - 2}{X^2 + 1} = \frac{(X^2 + 1)^2 + (X - 2)^2}{(X - 2)(X^2 + 1)}.$$

Développement du numérateur : $(X^2 + 1)^2 + (X - 2)^2 = X^4 + 2X^2 + 1 + X^2 - 4X + 4 = X^4 + 3X^2 - 4X + 5$. Développement du dénominateur : $(X - 2)(X^2 + 1) = X^3 - 2X^2 + X - 2$.

$$F + F^{-1} = \frac{X^4 + 3X^2 - 4X + 5}{X^3 - 2X^2 + X - 2}.$$

6.1.10 Exercices pratiques avec Python

`sympy` traite nativement les fractions rationnelles : addition, multiplication, simplification, mise au même dénominateur via `together` et `cancel`.

```

1 import sympy as sp
2
3 X = sp.symbols('X')
4
5 # 1. Operations elementaires
6 F1 = (X**2 + 1) / (X - 2)
7 F2 = (X - 2) / (X**2 + 1)
8 print(f"F1_{}_{}={F1}")
9 print(f"F2_{}_{}=1/F1_{}_{}={F2}")
10 print(f"F1_{}_{}F2_{}_{}={sp.simplify(F1_{}_{}F2_{}_{})}_{}_{}(attendu_{}_{}1)")
11
12 # 2. Addition et mise au meme denominateur
13 G = 1/X + 1/(X + 1) - 1/(X**2 + X)

```

```

14 print(f"\n1/X+1/(X+1)-1/(X^2+X)={sp.together(G)}")
15 print(f"\nsimplifie={sp.simplify(G)}")
16
17 # 3. F + 1/F : verification de l'exemple 3
18 H = F1 + F2
19 print(f"\nF+1/F={sp.together(H)}")
20 print(f"\nsimplifie={sp.simplify(H)}")
21
22 # 4. Test : deux fractions sont-elles egales ?
23 A1 = (X**2 - 1) / (X**2 - 2*X + 1)
24 A2 = (X + 1) / (X - 1)
25 print(f"\n(X^2-1)/(X^2-2X+1)-(X+1)/(X-1)={sp.simplify(A1-A2)}")

```

6.1.11 Exercices à faire

- Exercice 1.** Vérifier que $\frac{X^3 - 1}{X - 1} = X^2 + X + 1$ dans $\mathbb{R}(X)$, en explicitant le sens donné à cette égalité.
- Exercice 2.** Effectuer la somme $\frac{1}{X-1} + \frac{1}{X+1} + \frac{2}{X^2-1}$ et la simplifier au maximum.
- Exercice 3.** Soit $F = \frac{X^2 + 2X + 1}{X^3 + X^2}$ dans $\mathbb{R}(X)$. Donner deux écritures de F avec des dénominateurs différents.
- Exercice 4.** Démontrer que la relation $\sim$ définie sur $K[X] \times (K[X] \setminus \{0\})$ par $(A, B) \sim (C, D) \iff AD = BC$ est bien une relation d'équivalence. Où utilise-t-on l'intégrité de $K[X]$?
- Exercice 5 (synthèse).** Soit $F \in \mathbb{C}(X)$. Démontrer que $F \in \mathbb{R}(X)$ si et seulement si $\bar{F} = F$, où $\bar{F}$ est obtenu en conjuguant les coefficients d'un représentant de F . (Indication : commencer par vérifier que la conjugaison est bien définie sur les classes.)

6.2 Forme irréductible et fonctions rationnelles

6.2.1 Motivation

Une même fraction rationnelle $F \in K(X)$ admet une infinité d'écritures A/B . Pour calculer, factoriser, étudier des limites ou simplifier des expressions, on a besoin d'une *écriture canonique*, libre de tout facteur commun parasite : c'est la **forme irréductible**. Elle est l'analogue de la forme « fraction réduite » d'un rationnel ($\frac{6}{8} = \frac{3}{4}$). Au-delà de la commodité, la forme irréductible permet de définir intrinsèquement le domaine de définition d'une fonction rationnelle, ses pôles, ses zéros (étudiés en §3).

6.2.2 Approche par problème : trouver la « plus simple » écriture

Problème : Réduire la fraction rationnelle $F = \frac{X^4 - 1}{X^3 - X^2 + X - 1}$ à sa forme la plus simple.

Résolution : factorisons numérateur et dénominateur. $X^4 - 1 = (X^2 - 1)(X^2 + 1) = (X - 1)(X + 1)(X^2 + 1)$. $X^3 - X^2 + X - 1 = X^2(X - 1) + (X - 1) = (X - 1)(X^2 + 1)$. Le PGCD est $(X - 1)(X^2 + 1)$. En simplifiant :

$$F = \frac{(X - 1)(X + 1)(X^2 + 1)}{(X - 1)(X^2 + 1)} = X + 1.$$

Cette écriture est minimale : aucun facteur commun ne subsiste entre numérateur $(X + 1)$ et dénominateur (1) . Elle s'obtient algorithmiquement en divisant numérateur et dénominateur par leur PGCD.

6.2.3 Forme irréductible

Définition 6.2.1 (Forme irréductible). Une fraction rationnelle $F = \frac{A}{B} \in K(X)$ est dite sous **forme irréductible** si $A \wedge B = 1$, c'est-à-dire si A et B sont premiers entre eux dans $K[X]$.

Une forme irréductible A/B est dite *normalisée* si de plus B est unitaire (coefficient dominant égal à 1).

Théorème 6.2.1 (Existence et unicité de la forme irréductible). *Toute fraction rationnelle $F \in K(X)$, $F \neq 0$, admet :*

1. *au moins une forme irréductible ;*
2. *une unique forme irréductible normalisée.*

Démonstration. Existence. Soit A/B un représentant quelconque de F . Notons $D = A \wedge B$. Alors $A = DA'$ et $B = DB'$ avec $A' \wedge B' = 1$. Donc $\frac{A}{B} = \frac{A'}{B'}$ est une forme irréductible.

Unicité de la forme normalisée. Supposons $\frac{A_1}{B_1} = \frac{A_2}{B_2}$ avec $A_i \wedge B_i = 1$ et B_i unitaire. L'égalité $A_1 B_2 = A_2 B_1$ et le théorème de Gauss donnent : comme $B_1 \mid A_1 B_2$ et $B_1 \wedge A_1 = 1$, alors $B_1 \mid B_2$. De même $B_2 \mid B_1$. Comme B_1, B_2 sont unitaires, $B_1 = B_2$. Ensuite $A_1 = A_2$. $\square$

Remarque 6.2.1. Sans la condition de normalisation, la forme irréductible n'est unique qu'à *multiplication par une constante non nulle près* : A/B et $(\lambda A)/(\lambda B)$ sont deux formes irréductibles équivalentes pour tout $\lambda \in K^*$.

6.2.4 Algorithme de réduction

L'algorithme suivant, basé sur l'algorithme d'Euclide vu au chapitre V, calcule la forme irréductible normalisée d'une fraction A/B donnée :

1. Calculer $D = A \wedge B$ par l'algorithme d'Euclide.
2. Effectuer les divisions : $A' = A/D$, $B' = B/D$.
3. Diviser A' et B' par le coefficient dominant de B' pour rendre B' unitaire.

Le résultat A''/B'' est l'unique forme irréductible normalisée de F .

6.2.5 Domaine de la fonction rationnelle

Proposition 6.2.2 (Domaine intrinsèque). *Soit $F \in K(X)$ et A/B sa forme irréductible. La fonction rationnelle $\tilde{F} : x \mapsto A(x)/B(x)$ est définie sur $K \setminus Z(B)$, où $Z(B)$ est l'ensemble des racines de B . Cet ensemble ne dépend pas du choix de la forme irréductible : il est intrinsèque à F .*

Démonstration. Si A/B et A'/B' sont deux formes irréductibles de la même fraction F , alors B et B' sont associés (par l'unicité ci-dessus à un scalaire près). Donc $Z(B) = Z(B')$. $\square$

Définition 6.2.2 (Pôle, zéro — définitions provisoires). Soit $F \in K(X)$ donnée par sa forme irréductible A/B .

- Un **zéro** de F est une racine de A dans K (ou son extension $\mathbb{C}$).
- Un **pôle** de F est une racine de B dans K (ou son extension $\mathbb{C}$).

Les multiplicités de zéros et pôles seront formalisées au §3.

Remarque 6.2.2. La nécessité de la forme irréductible apparaît clairement ici : sans simplification, la fraction $\frac{X(X-1)}{X-1}$ semblerait avoir un pôle en 1, mais une fois réduite à X on voit qu'il n'en est rien. Les pôles et zéros sont des invariants algébriques de F , lisibles seulement sur la forme irréductible.

6.2.6 Opérations sur les formes irréductibles

Les opérations $+, \times, /$ ne préservent pas la forme irréductible : la somme de deux fractions sous forme irréductible doit en général être réduite à nouveau. Toutefois :

Proposition 6.2.3 (Multiplication et inverse). *Soient $F = A/B$ et $G = C/D$ deux fractions rationnelles non nulles sous forme irréductible.*

- Si $A \wedge D = 1$ et $C \wedge B = 1$, alors $\frac{AC}{BD}$ est sous forme irréductible.
- L'inverse de A/B est B/A , et il est sous forme irréductible (puisque $A \wedge B = 1$).

Démonstration. Pour la multiplication : un facteur commun à AC et BD devrait, par Gauss, diviser un des facteurs ; les hypothèses excluent toutes les combinaisons non triviales. Pour l'inverse : la condition $B \wedge A = 1$ est symétrique. $\square$

6.2.7 Exemples résolus (Difficulté ascendante)

Exemple 1 (Réduction directe — Facile). Mettre sous forme irréductible $F = \frac{2X^2 - 8}{4X - 8}$ dans $\mathbb{R}(X)$.

Correction : on factorise $2X^2 - 8 = 2(X - 2)(X + 2)$ et $4X - 8 = 4(X - 2)$.

$$F = \frac{2(X - 2)(X + 2)}{4(X - 2)} = \frac{2(X + 2)}{4} = \frac{X + 2}{2}.$$

La forme normalisée demande un dénominateur unitaire : ici 2 n'est pas unitaire, mais on accepte $1/2 \cdot (X + 2)$ ou bien on écrit $F = \frac{X/2 + 1}{1}$: c'est en fait un polynôme.

(Remarque : les fractions « polynomiales » ont pour forme irréductible normalisée $P/1$ avec P polynôme.)

Exemple 2 (Réduction par algorithme d'Euclide — Intermédiaire). Mettre sous forme irréductible $F = \frac{X^3 - X}{X^2 - 1}$ dans $\mathbb{R}(X)$.

Correction : $A = X^3 - X = X(X^2 - 1)$ et $B = X^2 - 1$. Par Euclide : $A = X \cdot B + 0$, donc $A \wedge B = B = X^2 - 1$. $A/D = X(X^2 - 1)/(X^2 - 1) = X$. $B/D = (X^2 - 1)/(X^2 - 1) = 1$.
Forme irréductible normalisée : $F = X$.

Exemple 3 (Domaine de définition — Difficile). Donner la forme irréductible et le domaine de définition de la fonction $x \mapsto \frac{x^4 - 5x^2 + 4}{x^4 - 1}$ sur $\mathbb{R}$.

Correction : factorisons. $X^4 - 5X^2 + 4 = (X^2 - 1)(X^2 - 4) = (X - 1)(X + 1)(X - 2)(X + 2)$. $X^4 - 1 = (X^2 - 1)(X^2 + 1) = (X - 1)(X + 1)(X^2 + 1)$. PGCD = $(X - 1)(X + 1) = X^2 - 1$.
Après simplification :

$$F = \frac{(X - 2)(X + 2)}{X^2 + 1} = \frac{X^2 - 4}{X^2 + 1}.$$

Comme $X^2 + 1$ n'a pas de racine réelle, le domaine de définition de $\tilde{F}$ sur $\mathbb{R}$ est $\mathbb{R}$ tout entier.

Attention à un piège fréquent : la fonction $x \mapsto \frac{x^4 - 5x^2 + 4}{x^4 - 1}$ telle qu'écrite n'est pas définie en $x = \pm 1$. La forme irréductible donne une fonction *prolongée par continuité* en ces points (avec valeur $-3/2$ par exemple en $x = 1$). C'est précisément l'intérêt de la forme irréductible : elle révèle les vrais pôles, en éliminant les « fausses singularités » dues à des facteurs communs.

6.2.8 Exercices pratiques avec Python

`sympy.cancel` renvoie la forme irréductible d'une fraction rationnelle.

```

1 import sympy as sp
2
3 X = sp.symbols('X')
4
5 # 1. Reduction explicite
6 F = (X**3 - X) / (X**2 - 1)
7 print(f"F===== {F}")
8 print(f"forme irréductible : {sp.cancel(F)}")
9
10 # 2. Algorithme manuel : pgcd numérateur/dénominateur
11 def forme_irreductible(num, den, var):
12     g = sp.gcd(num, den)
13     return sp.simplify(num / g), sp.simplify(den / g)
14
15 num = X**4 - 5*X**2 + 4
16 den = X**4 - 1
17 A, B = forme_irreductible(num, den, X)
18 print(f"\n{num}/_{den}")
19 print(f"pgcd===== {sp.gcd(num, den)}")
20 print(f"forme irred. normalisee : ({A})/_({B})")
21
22 # 3. Domaine reel d'une fonction rationnelle
23 def domaine_reel(num, den, var):

```

```

24     """Retourne les valeurs reelles a exclure (poles)."""
25     A, B = forme_irreductible(num, den, var)
26     return sp.solve(B, var)
27
28 print(f"\nDomaine de F = ({num})/({den}) :")
29 print(f"poles reels (apres simplification) : {domaine_reel(num, den, X)}")
30 print(f"poles apparents si on n'avait pas simplifie : {sp.solve(den, X)}")

```

6.2.9 Exercices à faire

- Exercice 1.** Mettre sous forme irréductible normalisée chacune des fractions suivantes dans $\mathbb{R}(X)$:
 - $F_1 = \frac{3X^2 - 12}{6X + 12}$;
 - $F_2 = \frac{X^4 + X^3}{X^3 + X^2}$;
 - $F_3 = \frac{X^3 - 1}{X^2 + X + 1}$.
- Exercice 2.** Soient $F = \frac{X^2 + 1}{X - 1}$ et $G = \frac{X^2 - 1}{X + 2}$ dans $\mathbb{R}(X)$, sous forme irréductible. Calculer $F + G$ et donner le résultat sous forme irréductible normalisée.
- Exercice 3.** Soit $F = \frac{X^2 - 4}{X^3 - 2X^2 - X + 2}$. Mettre F sous forme irréductible et déterminer son domaine de définition sur $\mathbb{R}$.
- Exercice 4.** Démontrer que si $F = A/B$ est sous forme irréductible avec $\deg A < \deg B$, alors la même fraction multipliée par tout polynôme non nul P (i.e. $F \cdot P$) reste *strictement propre* (degré du numérateur strictement inférieur à celui du dénominateur) si et seulement si $\deg P < \deg B - \deg A$.
- Exercice 5 (synthèse).** Soit $F \in \mathbb{R}(X)$ sous forme irréductible A/B . Montrer que F et $-F$ ont la même forme irréductible normalisée à un signe près sur le numérateur. Que dire de F et $1/F$ (lorsque $F \neq 0$) ?

6.3 Degré, pôles, zéros, multiplicités

6.3.1 Motivation

Une fraction rationnelle est entièrement caractérisée localement par trois invariants : son *degré* (qui contrôle son comportement à l'infini), la *multiplicité de ses pôles* (qui contrôle l'explosion près des points singuliers) et la *multiplicité de ses zéros* (qui contrôle la nullité). Ces invariants sont à la base de toute analyse de fractions rationnelles : calcul de primitives ($\int dx/(x-a)^m$), étude asymptotique en automatique ($1/(p^2 + 2\zeta\omega p + \omega^2)$), théorème des résidus en analyse complexe, et finalement la décomposition en éléments simples (§4).

6.3.2 Approche par problème

Problème : On considère $F = \frac{X^3 - X}{(X-1)^2(X^2+4)}$ dans $\mathbb{R}(X)$, sous forme *déjà irréductible* (à vérifier).

1. Quel est le comportement de $\tilde{F}(x)$ lorsque $x \rightarrow +\infty$?
2. Combien de pôles réels ? Avec quelle multiplicité chacun ?
3. Combien de zéros réels ? Avec quelle multiplicité ?

Résolution : $\deg(X^3 - X) = 3$, $\deg((X-1)^2(X^2+4)) = 4$. Donc $\deg F = 3 - 4 = -1$: à l'infini, $F(x) \sim c/x$ et tend vers 0. Le numérateur factorisé : $X^3 - X = X(X-1)(X+1)$. PGCD avec $(X-1)^2(X^2+4)$: facteur commun $X-1$. Donc F *n'était pas* sous forme irréductible. Forme irréductible : $F = \frac{X(X+1)}{(X-1)(X^2+4)}$. Pôles réels : $X-1=0$, soit $x=1$, multiplicité 1. Le facteur X^2+4 n'a pas de racine réelle (mais a deux pôles complexes $\pm 2i$ de multiplicité 1). Zéros réels : $X(X+1)=0$, soit $x=0$ (multiplicité 1) et $x=-1$ (multiplicité 1).

Cette section formalise ces calculs.

6.3.3 Degré d'une fraction rationnelle

Définition 6.3.1 (Degré). Soit $F = A/B \in K(X)$ avec $A \neq 0$ et $B \neq 0$. Le **degré** de F est :

$$\deg F = \deg A - \deg B \in \mathbb{Z}.$$

Par convention, $\deg 0 = -\infty$.

Proposition 6.3.1 (Bonne définition). $\deg F$ ne dépend pas du représentant choisi de F .

Démonstration. Si $A/B = A'/B'$, alors $AB' = A'B$, donc $\deg A + \deg B' = \deg A' + \deg B$, soit $\deg A - \deg B = \deg A' - \deg B'$. $\square$

Proposition 6.3.2 (Propriétés du degré). Pour $F, G \in K(X)$:

1. $\deg(FG) = \deg F + \deg G$;
2. $\deg(F+G) \leq \max(\deg F, \deg G)$, avec égalité si $\deg F \neq \deg G$;
3. $\deg(1/F) = -\deg F$ pour $F \neq 0$;
4. Si $F \in K[X] \subset K(X)$, le degré comme fraction coïncide avec le degré comme polynôme.

6.3.4 Fractions propres et partie entière

Définition 6.3.2 (Fraction propre, strictement propre). $F = A/B \in K(X)$ est dite :

- **propre** si $\deg F \leq 0$ (i.e. $\deg A \leq \deg B$) ;
- **strictement propre** si $\deg F < 0$ (i.e. $\deg A < \deg B$) ;
- **impropre** si $\deg F > 0$.

Théorème 6.3.3 (Décomposition partie entière + reste strictement propre). Pour toute $F = A/B \in K(X)$ avec $B \neq 0$, il existe un unique couple (E, R) tel que :

$$F = E + \frac{R}{B}, \quad E \in K[X], \quad \deg R < \deg B.$$

E est appelée la **partie entière** (ou *partie polynomiale*) de F , et R/B est la partie strictement propre. Lorsque F est strictement propre, $E = 0$.

Démonstration. Existence : la division euclidienne dans $K[X]$ donne $A = BQ + R$ avec $\deg R < \deg B$. Alors $F = A/B = Q + R/B$, donc $E = Q$ convient. Unicité : si $E + R/B = E' + R'/B$, alors $E - E' = (R' - R)/B$; le membre de gauche est polynomial, donc $B \mid R' - R$. Comme $\deg(R' - R) < \deg B$, on a $R' = R$, puis $E = E'$. $\square$

Remarque 6.3.1. La partie entière E donne le *comportement asymptotique* : pour $|x| \rightarrow \infty$, $F(x) \sim E(x)$. C'est l'analogie du « polynôme dominant » en analyse réelle.

6.3.5 Multiplicité d'un zéro et d'un pôle

Définition 6.3.3 (Multiplicité d'un zéro et d'un pôle). Soit $F \in K(X)$ non nulle, sous forme irréductible A/B , et a un élément de K (ou plus généralement de $\mathbb{C}$ pour $F \in \mathbb{R}(X)$).

- a est un **zéro** de F de multiplicité $m \geq 1$ si $(X - a)^m \mid A$ et $(X - a)^{m+1} \nmid A$. On note $\text{mult}_a^0(F) = m$.
- a est un **pôle** de F de multiplicité $m \geq 1$ si $(X - a)^m \mid B$ et $(X - a)^{m+1} \nmid B$. On note $\text{mult}_a^\infty(F) = m$.

Si a n'est ni zéro ni pôle, on pose $\text{mult}_a^0(F) = 0$ et $\text{mult}_a^\infty(F) = 0$.

Remarque 6.3.2. Comme $A \wedge B = 1$, un même a ne peut pas être à la fois zéro et pôle : au plus l'un des deux est non nul. C'est précisément l'intérêt de la forme irréductible.

6.3.6 Comportement local près d'un pôle

Proposition 6.3.4 (Asymptotique au voisinage d'un pôle). Si $a \in K$ est un pôle de F de multiplicité m , alors il existe une fraction $G \in K(X)$, définie en a et telle que $G(a) \neq 0$, vérifiant

$$F = \frac{G}{(X - a)^m}.$$

En particulier, lorsque $K = \mathbb{R}$ et a est un pôle réel, on a $\tilde{F}(x) \underset{x \rightarrow a}{\sim} \frac{G(a)}{(x - a)^m}$.

Démonstration. Sous forme irréductible A/B , on factorise $B = (X - a)^m B'$ avec $B'(a) \neq 0$, puis $G = A/B' \in K(X)$. $\square$

6.3.7 Cas $\mathbb{C}(X)$: relation entre degré, zéros et pôles

Théorème 6.3.5 (Bilan zéros-pôles dans $\mathbb{C}(X)$). Soit $F \in \mathbb{C}(X)$ non nulle, sous forme irréductible A/B . La somme des multiplicités des zéros (dans $\mathbb{C}$) moins la somme des multiplicités des pôles (dans $\mathbb{C}$) est égale à $\deg F$:

$$\sum_{a \in \mathbb{C}} \text{mult}_a^0(F) - \sum_{a \in \mathbb{C}} \text{mult}_a^\infty(F) = \deg A - \deg B = \deg F.$$

Démonstration. Sur $\mathbb{C}$, A et B sont scindés. La somme des multiplicités des zéros vaut $\deg A$, celle des pôles $\deg B$. $\square$

Corollaire 6.3.6. Une fraction rationnelle non constante de $\mathbb{C}(X)$ admet (en comptant multiplicités) autant de zéros que de pôles si et seulement si $\deg F = 0$ (i.e. $\deg A = \deg B$).

6.3.8 Exemples résolus (Difficulté ascendante)

Exemple 1 (Degré et partie entière — Facile). Soit $F = \frac{X^4 + 1}{X^2 - 1}$. Calculer $\deg F$ et donner la décomposition partie entière + reste.

Correction : $\deg F = 4 - 2 = 2$, donc F est impropre. Division euclidienne : $X^4 + 1 = (X^2 - 1)(X^2 + 1) + 2$, donc

$$F = X^2 + 1 + \frac{2}{X^2 - 1}.$$

Partie entière : $E = X^2 + 1$. Reste strictement propre : $\frac{2}{X^2 - 1}$.

Exemple 2 (Pôles et zéros avec multiplicités — Intermédiaire). Soit $F = \frac{(X - 2)^3(X + 1)}{(X - 1)^2(X + 1)^2}$ dans $\mathbb{R}(X)$. Mettre sous forme irréductible et déterminer les pôles et zéros avec leurs multiplicités.

Correction : on simplifie le facteur commun $(X + 1)$:

$$F = \frac{(X - 2)^3}{(X - 1)^2(X + 1)}.$$

Cette écriture est irréductible (les facteurs $(X - 2)$, $(X - 1)$, $(X + 1)$ sont premiers entre eux deux à deux).

— Zéro : $a = 2$ de multiplicité 3.

— Pôles : $a = 1$ de multiplicité 2 ; $a = -1$ de multiplicité 1.

Vérification du bilan : $3 - (2 + 1) = 0 = \deg F = 3 - 3$. ✓

Exemple 3 (Comportement asymptotique — Difficile). Étudier le comportement de $\tilde{F}(x)$ près de chaque pôle réel et à l'infini, pour $F = \frac{X^2 + 1}{(X - 1)^3(X + 2)}$ dans $\mathbb{R}(X)$ (forme déjà irréductible).

Correction :

— À l'infini : $\deg F = 2 - 4 = -2$, donc $\tilde{F}(x) \underset{x \rightarrow \pm\infty}{\sim} \frac{1}{x^2}$. La fonction tend vers 0 « comme $1/x^2$ ».

— Pôle $a = 1$ de multiplicité 3 : on écrit $F = \frac{(X^2 + 1)/(X + 2)}{(X - 1)^3}$. Au voisinage de 1, le numérateur vaut $G(1) = 2/3$. Donc $\tilde{F}(x) \underset{x \rightarrow 1}{\sim} \frac{2/3}{(x - 1)^3}$. La fonction « explose » comme $\pm\infty$ selon le signe de $x - 1$.

— Pôle $a = -2$ de multiplicité 1 : $F = \frac{(X^2 + 1)/(X - 1)^3}{X + 2}$. Numérateur en -2 vaut $5/(-3)^3 = -5/27$. Donc $\tilde{F}(x) \underset{x \rightarrow -2}{\sim} \frac{-5/27}{x + 2}$.

6.3.9 Exercices pratiques avec Python

`sympy` fournit des fonctions pour décomposer en partie entière + reste (`div` sur les polynômes), trouver les pôles (zéros du dénominateur après simplification) et leur multiplicité.

```
1 import sympy as sp
2
```

```

3 X = sp.symbols('X')
4
5 # 1. Degré d'une fraction et partie entière
6 F = (X**4 + 1) / (X**2 - 1)
7 F_simpl = sp.cancel(F)
8 num, den = sp.fraction(F_simpl)
9 print(f"F={F}")
10 print(f"deg F={sp.degree(num)}-sp.degree(den)")
11
12 # Partie entière + reste : division euclidienne
13 Q, R = sp.div(num, den, X)
14 print(f"partie entière E={Q}")
15 print(f"reste R={R}")
16 print(f"R->F=({Q})+({R})/({den})")
17
18 # 2. Poles et zeros avec multiplicites
19 F = (X - 2)**3 * (X + 1) / ((X - 1)**2 * (X + 1)**2)
20 F_irred = sp.cancel(F)
21 num, den = sp.fraction(F_irred)
22 print(f"\nF(forme irred.)={F_irred}")
23 print(f"zeros(avec multiplicites):{sp.roots(num, X)}")
24 print(f"poles(avec multiplicites):{sp.roots(den, X)}")
25
26 # 3. Bilan zeros/poles = deg F (sur C)
27 def bilan_zeros_poles(F, var):
28     F_irr = sp.cancel(F)
29     n, d = sp.fraction(F_irr)
30     z = sum(sp.roots(n, var).values())
31     p = sum(sp.roots(d, var).values())
32     return z, p, z - p
33
34 F = (X**3 - 1) / (X**4 + X**2)
35 z, p, diff = bilan_zeros_poles(F, X)
36 print(f"\nF={F}")
37 print(f"somme multiplicites zeros:{z}")
38 print(f"somme multiplicites poles:{p}")
39 print(f"z-p={diff} (attendu: deg F={sp.degree(sp.numer(
    sp.cancel(F)))-sp.degree(sp.denom(sp.cancel(F)))}")

```

6.3.10 Exercices à faire

1. **Exercice 1.** Calculer $\deg F$ pour chacune des fractions :

$$\begin{aligned}
 \text{--- } F_1 &= \frac{X^5 + 3X}{X^2 + 1}; \\
 \text{--- } F_2 &= \frac{X + 1}{X^3 - X}; \\
 \text{--- } F_3 &= \frac{X^4 - 1}{X^4 + 1}.
 \end{aligned}$$

Indiquer dans chaque cas si la fraction est propre, strictement propre, ou impropre.

2. **Exercice 2.** Donner la partie entière et le reste strictement propre de $F =$

$$\frac{X^4 - 2X^3 + X - 1}{X^2 + X + 1}.$$

3. **Exercice 3.** Déterminer les pôles et zéros (avec multiplicités) dans $\mathbb{C}$ de chacune des fractions suivantes, après mise sous forme irréductible :

$$\begin{aligned} \text{--- } F_1 &= \frac{X^3 - X}{X^2 - 1}; \\ \text{--- } F_2 &= \frac{X^2 + 1}{(X^2 - 1)^2}; \\ \text{--- } F_3 &= \frac{X^4 + 1}{X^4 - 1}. \end{aligned}$$

4. **Exercice 4.** Soit $F \in \mathbb{R}(X)$ non nulle. Démontrer que les pôles complexes non réels de F apparaissent par couples conjugués, avec la même multiplicité.
5. **Exercice 5 (synthèse).** Soit $F \in \mathbb{C}(X)$ avec $\deg F = -2$, ayant un unique pôle a de multiplicité 2. Montrer qu'il existe $\lambda \in \mathbb{C}^*$ tel que $F = \frac{\lambda}{(X - a)^2}$. Plus généralement, énoncer et démontrer le résultat pour $\deg F = -m$ avec un unique pôle de multiplicité m .

6.4 Décomposition en éléments simples

6.4.1 Motivation

La **décomposition en éléments simples** (DES) écrit toute fraction rationnelle comme somme d'un polynôme et de termes élémentaires de la forme $\frac{1}{(X - a)^k}$ (et, sur $\mathbb{R}$, de termes $\frac{\alpha X + \beta}{(X^2 + pX + q)^k}$). C'est l'outil universel pour :

- le *calcul de primitives* de fractions rationnelles ($\int dx/(x - a)^k$, $\int dx/(x^2 + 1)$ se calculent immédiatement) ;
- l'*inversion des transformées de Laplace et en z* en automatique et en traitement du signal ;
- le *calcul de séries télescopiques* et de sommes infinies ;
- le *théorème des résidus* en analyse complexe.

On démontre ici l'existence et l'unicité de la DES, puis l'on présente les méthodes de calcul des coefficients.

6.4.2 Approche par problème : intégrale d'une fraction simple

Problème : Calculer $\int_2^3 \frac{dx}{x^2 - 1}$.

Résolution : on écrit $\frac{1}{x^2 - 1} = \frac{1}{(x - 1)(x + 1)}$ et l'on cherche $\alpha, \beta \in \mathbb{R}$ tels que

$$\frac{1}{(x - 1)(x + 1)} = \frac{\alpha}{x - 1} + \frac{\beta}{x + 1}.$$

Méthode de multiplication par $(x - 1)$ puis évaluation en $x = 1$: $\alpha = \frac{1}{1 + 1} = \frac{1}{2}$.

Multiplication par $(x + 1)$ puis évaluation en $x = -1$: $\beta = \frac{1}{-1 - 1} = -\frac{1}{2}$. Donc

$$\frac{1}{x^2 - 1} = \frac{1}{2} \left(\frac{1}{x - 1} - \frac{1}{x + 1} \right),$$

et

$$\int_2^3 \frac{dx}{x^2 - 1} = \frac{1}{2} [\ln |x - 1| - \ln |x + 1|]_2^3 = \frac{1}{2} (\ln 2 - \ln 4 - \ln 1 + \ln 3) = \frac{1}{2} \ln \frac{3}{2}.$$

La décomposition en éléments simples a transformé un calcul difficile en deux primitives évidentes. Cette section systématise la technique.

6.4.3 Théorème de décomposition sur $\mathbb{C}$

Théorème 6.4.1 (DES sur $\mathbb{C}(X)$). *Soit $F = A/B \in \mathbb{C}(X)$ sous forme irréductible, avec $B = b \prod_{i=1}^r (X - a_i)^{m_i}$ (les a_i pôles distincts de multiplicités m_i , $b \in \mathbb{C}^*$). Alors F s'écrit de manière unique sous la forme*

$$F = E(X) + \sum_{i=1}^r \sum_{k=1}^{m_i} \frac{\lambda_{i,k}}{(X - a_i)^k}, \quad \lambda_{i,k} \in \mathbb{C},$$

où $E \in \mathbb{C}[X]$ est la partie entière de F (le polynôme tel que $\deg(F - E) < 0$).

Idée de la démonstration. Existence. Par récurrence sur le nombre de pôles distincts. On commence par soustraire la partie entière E . Pour le pôle a_1 de multiplicité m_1 , on extrait par multiplication par $(X - a_1)^{m_1}$ et division euclidienne tous les termes $\lambda_{1,k}/(X - a_1)^k$; la fraction restante a un pôle de moins.

Unicité. Si deux décompositions coïncident, leur différence est un polynôme strictement propre nul. L'unicité des coefficients de la « partie singulière » en chaque pôle découle de la minimalité du dénominateur. $\square$

6.4.4 Théorème de décomposition sur $\mathbb{R}$

Théorème 6.4.2 (DES sur $\mathbb{R}(X)$). *Soit $F = A/B \in \mathbb{R}(X)$ sous forme irréductible. Soit*

$$B = b \prod_{i=1}^r (X - a_i)^{m_i} \cdot \prod_{j=1}^s (X^2 + p_j X + q_j)^{n_j}$$

la factorisation de B en irréductibles réels (où les facteurs de degré 2 ont un discriminant $p_j^2 - 4q_j < 0$). Alors F se décompose de manière unique :

$$F = E(X) + \sum_{i=1}^r \sum_{k=1}^{m_i} \frac{\alpha_{i,k}}{(X - a_i)^k} + \sum_{j=1}^s \sum_{l=1}^{n_j} \frac{\beta_{j,l} X + \gamma_{j,l}}{(X^2 + p_j X + q_j)^l}$$

avec $E \in \mathbb{R}[X]$, $\alpha_{i,k}, \beta_{j,l}, \gamma_{j,l} \in \mathbb{R}$.

Les fractions du premier type sont appelées éléments simples de première espèce, celles du second type éléments simples de seconde espèce.

Idée. On part de la DES sur $\mathbb{C}(X)$. Les pôles complexes non réels apparaissent par couples conjugués $(a, \bar{a})$ de même multiplicité. On combine les termes $\frac{\lambda}{(X - a)^k}$ et $\frac{\bar{\lambda}}{(X - \bar{a})^k}$: leur somme est un polynôme à coefficients réels divisé par $((X - a)(X - \bar{a}))^k = (X^2 - 2\operatorname{Re}(a)X + |a|^2)^k$. En réduisant dans $\mathbb{R}[X]$, on obtient les éléments simples de seconde espèce. $\square$

6.4.5 Méthodes pratiques de calcul des coefficients

Méthode 1 : multiplication par $(X - a)^m$ pour le coefficient de plus haute multiplicité. Soit a pôle de multiplicité m dans la DES, contribuant le terme $\frac{\lambda_m}{(X - a)^m} + \dots + \frac{\lambda_1}{X - a}$. On calcule

$$\lambda_m = [(X - a)^m \cdot F(X)]_{X=a}.$$

C'est l'unique coefficient extractible directement par évaluation : les autres exigent un travail supplémentaire (méthode 2 ou 3).

Méthode 2 : valeurs particulières. On écrit la DES avec coefficients inconnus, puis l'on évalue F en des valeurs commodes (par exemple $X = 0$ ou $X = 2$ si 0 et 2 ne sont pas pôles), pour obtenir un système linéaire.

Méthode 3 : identification par développement asymptotique. On multiplie l'identité par le dénominateur, on développe les deux membres comme polynômes, et l'on identifie les coefficients de chaque puissance de X .

Méthode 4 : formule du résidu pour pôles simples. Si a est pôle simple ($m = 1$) et $F = A/B$ avec $A(a), B'(a) \neq 0$, alors le coefficient de $1/(X - a)$ est :

$$\lambda_a = \frac{A(a)}{B'(a)}.$$

Démonstration : on écrit $B(X) = (X - a)Q(X)$ avec $Q(a) \neq 0$. Alors $B'(a) = Q(a)$ et $\lambda_a = A(a)/Q(a) = A(a)/B'(a)$.

Méthode 5 : dérivation pour les pôles multiples. Si a est pôle de multiplicité m , en notant $G(X) = (X - a)^m F(X)$, on a

$$\lambda_{m-k} = \frac{1}{k!} G^{(k)}(a) \quad \text{pour } k = 0, 1, \dots, m - 1.$$

C'est la formule de Taylor appliquée à G en a .

6.4.6 Exemples résolus (Difficulté ascendante)

Exemple 1 (Pôles simples — Facile). Décomposer $F = \frac{1}{(X - 1)(X - 2)}$ en éléments simples sur $\mathbb{R}$.

Correction : forme cherchée : $F = \frac{\alpha}{X - 1} + \frac{\beta}{X - 2}$. Multiplication par $(X - 1)$ et évaluation en $X = 1$: $\alpha = \frac{1}{1 - 2} = -1$. Multiplication par $(X - 2)$ et évaluation en $X = 2$: $\beta = \frac{1}{2 - 1} = 1$.

$$F = \frac{-1}{X - 1} + \frac{1}{X - 2}.$$

Exemple 2 (Pôle multiple — Intermédiaire). Décomposer $F = \frac{X+1}{(X-1)^2(X-2)}$ sur $\mathbb{R}$.

Correction : forme cherchée : $F = \frac{\alpha_2}{(X-1)^2} + \frac{\alpha_1}{X-1} + \frac{\beta}{X-2}$.

Calcul de α_2 : multiplier par $(X-1)^2$ et évaluer en $X=1$:

$$\alpha_2 = \left[\frac{X+1}{X-2} \right]_{X=1} = \frac{2}{-1} = -2.$$

Calcul de β : multiplier par $(X-2)$ et évaluer en $X=2$:

$$\beta = \left[\frac{X+1}{(X-1)^2} \right]_{X=2} = \frac{3}{1} = 3.$$

Calcul de α_1 : on évalue en une valeur simple, par exemple $X=0$:

$$F(0) = \frac{1}{(-1)^2(-2)} = -\frac{1}{2}.$$

D'autre part, la DES en $X=0$ donne :

$$\frac{\alpha_2}{1} + \frac{\alpha_1}{-1} + \frac{\beta}{-2} = -2 - \alpha_1 - \frac{3}{2} = -\frac{7}{2} - \alpha_1.$$

Identification : $-\frac{7}{2} - \alpha_1 = -\frac{1}{2}$, soit $\alpha_1 = -3$.

Conclusion :

$$\frac{X+1}{(X-1)^2(X-2)} = \frac{-2}{(X-1)^2} + \frac{-3}{X-1} + \frac{3}{X-2}.$$

Exemple 3 (Élément simple de seconde espèce — Difficile). Décomposer $F = \frac{1}{(X^2+1)(X-1)}$ sur $\mathbb{R}$.

Correction : le facteur X^2+1 est irréductible sur $\mathbb{R}$ (discriminant -4). La forme cherchée est :

$$F = \frac{\beta X + \gamma}{X^2+1} + \frac{\alpha}{X-1}.$$

Calcul de α : multiplication par $(X-1)$ et évaluation en $X=1$:

$$\alpha = \frac{1}{1+1} = \frac{1}{2}.$$

Calcul de β, γ : par identification après mise au même dénominateur $(X^2+1)(X-1)$:

$$1 = (\beta X + \gamma)(X-1) + \alpha(X^2+1) = \beta X^2 - \beta X + \gamma X - \gamma + \frac{1}{2}X^2 + \frac{1}{2}.$$

On regroupe :

$$1 = \left(\beta + \frac{1}{2}\right) X^2 + (\gamma - \beta)X + \left(\frac{1}{2} - \gamma\right).$$

Identification : $\beta + \frac{1}{2} = 0$ donne $\beta = -\frac{1}{2}$; $\frac{1}{2} - \gamma = 1$ donne $\gamma = -\frac{1}{2}$; vérification : $\gamma - \beta = -\frac{1}{2} - (-\frac{1}{2}) = 0$. ✓

Conclusion :

$$\frac{1}{(X^2+1)(X-1)} = \frac{-\frac{1}{2}X - \frac{1}{2}}{X^2+1} + \frac{\frac{1}{2}}{X-1} = \frac{1}{2} \left(\frac{1}{X-1} - \frac{X+1}{X^2+1} \right).$$

6.4.7 Application : calcul de primitives

Une fois la DES obtenue, l'intégration est immédiate sur chaque terme.

Éléments de première espèce.

$$\int \frac{dx}{(x-a)} = \ln|x-a| + C, \quad \int \frac{dx}{(x-a)^k} = \frac{-1}{(k-1)(x-a)^{k-1}} + C \quad (k \geq 2).$$

Éléments de seconde espèce ($x^2 + px + q$ **irréductible**). On complète le carré : $x^2 + px + q = (x + p/2)^2 + (q - p^2/4)$, avec $q - p^2/4 > 0$. On pose $u = x + p/2$, et l'on est ramené à $\int \frac{\alpha u + \beta}{(u^2 + c^2)^k} du$ qui se traite par décomposition $\alpha u du / (u^2 + c^2)^k$ (substitution $v = u^2 + c^2$) et $\beta du / (u^2 + c^2)^k$ (formule de récurrence ou arctan).

6.4.8 Exercices pratiques avec Python

`sympy.apart` effectue la décomposition en éléments simples automatiquement, sur $\mathbb{Q}$, $\mathbb{R}$ ou $\mathbb{C}$ selon l'extension fournie.

```

1 import sympy as sp
2
3 X = sp.symbols('X')
4
5 # 1. DES sur R (default = factorisation sur Q)
6 F = (X + 1) / ((X - 1)**2 * (X - 2))
7 print(f"F_□=□{F}")
8 print(f"□□DES_□sur_□R_□:□{sp.apart(F,□X)}")
9
10 # 2. DES avec un facteur de seconde espece
11 F = 1 / ((X**2 + 1) * (X - 1))
12 print(f"\nF_□=□{F}")
13 print(f"□□DES_□sur_□R_□:□{sp.apart(F,□X)}")
14
15 # 3. DES sur C : meme fraction, avec extension par I
16 print(f"□□DES_□sur_□C_□:□{sp.apart(F,□X,□full=True).doit()}")
17
18 # 4. Application au calcul de primitive
19 F = 1 / (X**2 - 1)
20 des = sp.apart(F, X)
21 print(f"\nF_□=□{F}")
22 print(f"□□DES_□□□□□□□□□□:□{des}")
23 print(f"□□primitive_□□□:□{sp.integrate(F,□X)}")
24
25 # 5. Verification : la somme des residus est-elle = 0 quand deg F <
    -1 ?
26 F = (X**2 + 1) / ((X - 1)*(X - 2)*(X - 3))
27 des = sp.apart(F, X)
28 print(f"\nF_□=□{F},□deg_□F_□=□-1")
29 print(f"□□DES_□:□{des}")

```

6.4.9 Exercices à faire

1. **Exercice 1.** Décomposer en éléments simples sur $\mathbb{R}$:

$$F = \frac{1}{(X-1)(X-2)(X-3)}.$$

2. **Exercice 2.** Décomposer en éléments simples sur $\mathbb{R}$:

$$F = \frac{X^2 + 2}{(X-1)^3}.$$

(Indication : utiliser la dérivation, ou poser $Y = X - 1$ et développer le numérateur en puissances de Y .)

3. **Exercice 3.** Décomposer en éléments simples sur $\mathbb{R}$:

$$F = \frac{X}{(X^2 + 1)^2}.$$

4. **Exercice 4.** En utilisant la décomposition en éléments simples, calculer la primitive

$$\int \frac{x}{(x-1)(x+2)} dx.$$

5. **Exercice 5 (synthèse, séries télescopiques).** En décomposant $\frac{1}{n(n+1)}$ en éléments simples, calculer

$$S_N = \sum_{n=1}^N \frac{1}{n(n+1)}.$$

En déduire $\lim_{N \rightarrow \infty} S_N$.

Travaux Pratiques (TP) Python

L'objectif de ce TP est de construire une bibliothèque *maison* pour les fractions rationnelles : classe `FractionRationnelle`, mise sous forme irréductible, recherche de pôles et zéros, décomposition en éléments simples, et applications au calcul de primitives et de séries télescopiques. On supposera disponible la classe `Polynome` construite au TP du chapitre V (ou `sympy` pour aller plus vite).

Exercice 1 : Classe `FractionRationnelle`

1. **[Bloom : Compréhension | Difficulté : Moyenne]**

Créer une classe `FractionRationnelle` avec deux attributs (`num`, `den`) qui sont des objets `Polynome` (ou des polynômes `sympy`). Le constructeur doit :

- refuser un dénominateur nul ;
- mettre la fraction sous *forme irréductible normalisée* (division par le PGCD, dénominateur unitaire).

2. **[Bloom : Application | Difficulté : Moyenne]**

Surcharger `__repr__` pour un affichage lisible du type `(num) / (den)`.

Exercice 2 : Arithmétique des fractions rationnelles

1. [Bloom : Application | Difficulté : Moyenne]

Surcharger `__add__`, `__sub__`, `__mul__`, `__truediv__` pour implémenter $\frac{A}{B} + \frac{C}{D} = \frac{AD + BC}{BD}$ et les autres opérations. Toujours retourner le résultat sous forme irréductible normalisée.

2. [Bloom : Évaluation | Difficulté : Moyenne]

Vérifier numériquement les axiomes de corps sur 5 fractions aléatoires : $F + (-F) = 0$, $F \cdot (1/F) = 1$, distributivité $F(G + H) = FG + FH$.

Exercice 3 : Partie entière et reste strictement propre

1. [Bloom : Application | Difficulté : Moyenne]

Ajouter une méthode `partie_entiere(self)` qui retourne le couple $(E, R/B)$ où E est la partie polynomiale et R/B la partie strictement propre, obtenus par division euclidienne $A = BQ + R$.

2. [Bloom : Évaluation | Difficulté : Moyenne]

Tester sur $F = \frac{X^4 + 1}{X^2 - 1}$: on doit retrouver $E = X^2 + 1$ et $R/B = \frac{2}{X^2 - 1}$.

Exercice 4 : Pôles et zéros avec multiplicités

1. [Bloom : Application | Difficulté : Moyenne]

Ajouter `poles(self)` et `zeros(self)` qui retournent un dictionnaire $\{a : m\}$ associant à chaque pôle (resp. zéro) sa multiplicité. Travailler sur la forme irréductible.

2. [Bloom : Évaluation | Difficulté : Moyenne]

Tester sur $F = \frac{(X - 2)^3(X + 1)}{(X - 1)^2(X + 1)^2}$. Après simplification, identifier zéros et pôles.

3. [Bloom : Synthèse | Difficulté : Moyenne]

Vérifier numériquement le bilan zéros-pôles dans $\mathbb{C}$: $\sum \text{mult}(\text{zéros}) - \sum \text{mult}(\text{pôles}) = \deg F$.

Exercice 5 : Décomposition en éléments simples (pôles simples)

1. [Bloom : Synthèse | Difficulté : Moyenne]

Soit $F = A/B$ sous forme irréductible avec B scindé à pôles simples sur $\mathbb{C}$. Implémenter `des_poles_simples(F)` qui calcule les coefficients $\lambda_a = \frac{A(a)}{B'(a)}$ pour chaque pôle simple a .

2. [Bloom : Évaluation | Difficulté : Moyenne]

Tester sur $F = \frac{1}{(X - 1)(X - 2)(X - 3)}$. On doit obtenir $\frac{1/2}{X - 1} + \frac{-1}{X - 2} + \frac{1/2}{X - 3}$.

3. [Bloom : Évaluation | Difficulté : Moyenne]

Comparer avec `sympy.apart(F, X)`.

Exercice 6 : DES pour pôle multiple par dérivation (Taylor)

1. [Bloom : Synthèse | Difficulté : Difficile]

Soit a un pôle de multiplicité m de $F = A/B$. En posant $G(X) = (X - a)^m F(X)$

(qui est un polynôme), les coefficients de $1/(X-a)^k$ dans la DES s'obtiennent par les dérivées :

$$\lambda_{m-k} = \frac{G^{(k)}(a)}{k!}, \quad k = 0, 1, \dots, m-1.$$

Implémenter `coefficients_pole_multiple(F, a, m)` qui retourne la liste $[\lambda_1, \lambda_2, \dots, \lambda_m]$.

2. **[Bloom : Évaluation | Difficulté : Difficile]**

Tester sur $F = \frac{X^2+2}{(X-1)^3}$. Calculer la DES complète et vérifier en additionnant les éléments simples.

Exercice 7 : Application — Sommation de séries télescopiques

1. **[Bloom : Application | Difficulté : Moyenne]**

Calculer $S_N = \sum_{n=1}^N \frac{1}{n(n+1)(n+2)}$ par DES. La fraction $\frac{1}{X(X+1)(X+2)}$ se décompose en :

$$\frac{1}{X(X+1)(X+2)} = \frac{1/2}{X} - \frac{1}{X+1} + \frac{1/2}{X+2}.$$

Vérifier cette décomposition (par DES Python ou par identification).

2. **[Bloom : Synthèse | Difficulté : Difficile]**

En utilisant la décomposition, montrer que la somme se télescope et calculer une formule fermée pour S_N . Vérifier sur $N = 10, 100, 1000$ que la formule fermée et la sommation directe coïncident.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

En déduire $\sum_{n=1}^{\infty} \frac{1}{n(n+1)(n+2)} = \frac{1}{4}$. Vérifier numériquement avec $N = 10^6$.

Exercice 8 : Application — Calcul de primitives

1. **[Bloom : Application | Difficulté : Moyenne]**

En utilisant la DES, calculer à la main les primitives suivantes :

$$I_1 = \int \frac{dx}{x^2-1}, \quad I_2 = \int \frac{x dx}{(x-1)(x-2)}, \quad I_3 = \int \frac{dx}{x(x^2+1)}.$$

2. **[Bloom : Évaluation | Difficulté : Moyenne]**

Comparer avec `sympy.integrate` : les expressions doivent coïncider à une constante près (utiliser `sympy.simplify` sur la différence).

3. **[Bloom : Création | Difficulté : Difficile]**

Écrire `integrer_fraction_rationnelle(F, X)` qui automatise le calcul : DES, intégration de chaque élément simple (ln pour 1^{re} espèce simple, arctan + ln pour 2^{nde} espèce). Tester sur $\frac{1}{(X^2+1)(X-1)}$ et comparer à `sympy.integrate`.